

Pixels to Convolution

A complete, plain-English course for everything covered in your Digital Image Processing lectures so far: what an image really is, how pixels relate to each other, how 2D convolution works, the whole of intensity transformation and histogram processing, and every spatial filter from the averaging mask to Sobel. Written to be read once, slowly, and then used again before the exam.



This strip is the whole subject in one picture. An 8-bit gray image is nothing but a table of numbers taken from this scale.

BEFORE YOU START

How to use these notes

These notes are built from your three lecture handouts and your syllabus. Nothing extra has been added and nothing that was taught has been left out. Read the chapters in order — each one uses words explained in the chapter before it.

Three kinds of boxes appear in the text:

IN SIMPLE WORDS

The idea explained again in the easiest language possible. If the main paragraph feels heavy, read this instead.

FOR THE EXAM

Something that is very likely to be asked, or a mistake students usually make. Do not skip these.

YOU HAVE ALREADY DONE THIS

Points to one of your own assignments, so you can connect the theory to code you have already written and run.

What is covered, and what is not

Your syllabus has six units and 45 hours. Your handouts stop part way through. Here is the honest map, so you know exactly where you stand.

SYLLABUS UNIT	HOURS	IN YOUR HANDOUTS?	WHERE IN THESE NOTES
Digital Image Fundamentals	06	Yes, fully	Chapters 1–10
2D convolution, Z-transform (taught as extra groundwork)	—	Yes, fully	Chapters 11–14
Image Enhancement	12	Partly — all of the spatial-domain point processing and histogram work	Chapters 15–22
...the rest of that unit: mask processing, the 2-D Fourier transform and frequency-domain filtering		Not yet	—
Image Restoration	10	Not yet	—
Image Compression	04	Not yet	—
Morphological Image Processing	04	Not yet	—
Image Segmentation	09	Not yet	—

So these notes cover **Unit 1 completely**, the 2D convolution and Z-transform work your teacher added on top of it, and **the spatial-domain half of Unit 2** — every intensity transformation and all of the histogram processing, up to the end of Lecture 20. The next topic in class is mask processing, and the notes stop exactly where your lectures do.

CHAPTER 1 HANDOUT 1, LECTURE 1 UNIT 1

What a digital image really is

Start with the one definition the whole course is built on.

$$f(x, y)$$

x and y are the position of a point in the picture. f is the brightness at that point.

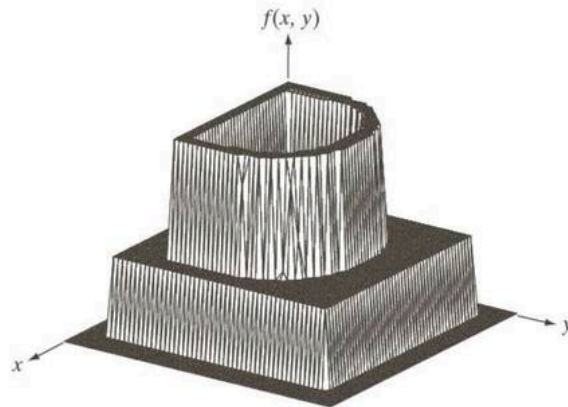
An image is a function of two things. You give it a position, it gives you back a brightness. That is all. When x , y and the value of f are all whole numbers instead of continuous quantities, we call it a **digital image**, and each element of it is called a **pixel** (short for "picture element").

FROM YOUR LECTURE SLIDES

141
of
36

Image Representation

Images taken from Gonzalez & Woods, Digital Image Processing (2002).



The same idea drawn as a surface: the height at each point is the brightness $f(x, y)$. A bright patch is a hill, a dark patch is a valley. The picture on the left is the letter D, and the plot behind it is exactly its pixel values.

IN SIMPLE WORDS

A digital image is a table of numbers. Small numbers are dark spots, big numbers are bright spots. If you print that table and stand far away, your eye joins the numbers together and you see a photograph.

The number itself

In an 8-bit gray image every number is stored in one byte, so it can be anything from 0 to 255. By convention **0 is black** and **255 is white**, and everything between is a shade of gray. This value is called the **intensity** or the **gray level** of the pixel.

Look at a real example. Below is a tiny 4×4 piece of a gray image, first as numbers and then as the shades those numbers mean.

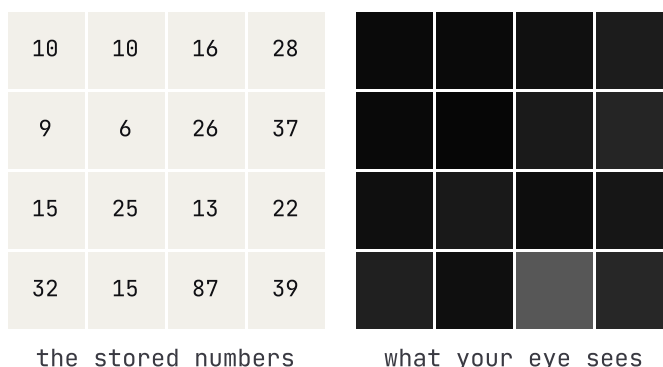


Figure 1.1 — The same 4×4 patch twice. All these values are small, so the patch is a dark region of the picture. The single 87 is a bright speck.

Where the origin sits

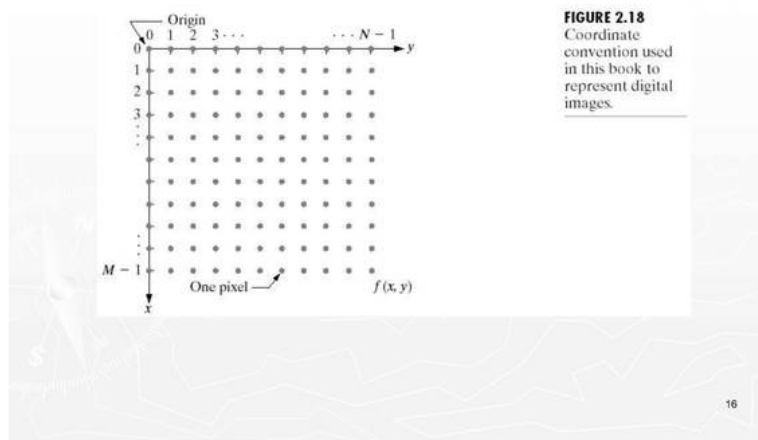
The book (Gonzalez and Woods) and your slides put the **origin at the top-left corner**. x runs *downwards* along the rows and y runs to the right along the columns. This is different from the graphs you drew in maths class, and it catches people out.

$f(0,0)$ $f(0,1)$... $f(0, N-1)$ $f(1,0)$ $f(1,1)$... $f(1, N-1)$ $f(M-1,0)$
 $f(M-1,1)$... $f(M-1, N-1)$

An image with M rows and N columns. Indices start at 0.

FROM YOUR LECTURE SLIDES

Representing Digital Images



The coordinate convention used throughout the course. The origin sits at the top-left, x counts rows downwards, y counts columns to the right, and one dot is one pixel.

FOR THE EXAM

MATLAB counts from 1, not from 0. In MATLAB the same image is written $f(1,1)$ to $f(M,N)$. If a question uses MATLAB notation, remember that $f(1,1)$ is the top-left pixel, and there is no $f(0,0)$. Python and OpenCV count from 0, like the book.

Gray or colour

How many numbers do we keep per point?

- **One number per point** — a gray (also called monochrome or black-and-white) image.
 The array is 2-dimensional: rows $\times$ columns.
- **Three numbers per point** — a colour image, holding Red, Green and Blue for that point.
 The array is 3-dimensional: rows $\times$ columns $\times$ 3.

Most of this course uses gray images, because everything important can be shown with one number per pixel, and colour is just the same work repeated three times.

FROM YOUR LECTURE SLIDES

What is a Digital Image? (cont...)

Common image formats include:

- 1 sample per point (B&W or Grayscale)
- 3 samples per point (Red, Green, and Blue)



For most of this course we will focus on grey-scale images

One sample per point gives the gray picture on the left; three samples per point (red, green, blue) give the colour picture on the right.

YOU HAVE ALREADY DONE THIS

In Assignment 1 your `my_imread()` returned a `(512, 512)` array for the gray baboon and a `(rows, cols, 3)` array for a colour file. That is exactly this difference, in code.

A digital image is only an approximation

The real scene in front of the camera is smooth and continuous. The moment we store it as a table of whole numbers, two things are thrown away for ever:

1. everything that happened *between* the sample points, and
2. every brightness that fell *between* two allowed numbers.

So a digital image is always an approximation of reality. How good the approximation is depends on how many samples you took and how many brightness levels you allowed — which is the whole subject of Chapter 6.

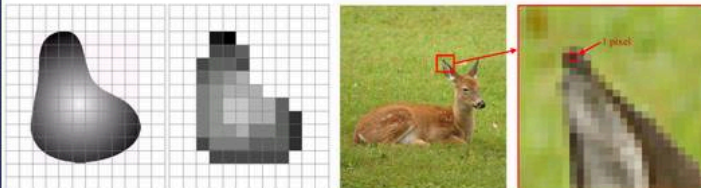
FROM YOUR LECTURE SLIDES

What is a Digital Image? (cont...)

Images taken from Gonzalez & Woods, Digital Image Processing (2002)

Pixel values typically represent gray levels, colours, heights etc

Remember *digitization* implies that a digital image is an *approximation* of a real scene



Why a digital image is only an approximation. Zoom far enough into any photograph and the smooth shape breaks up into the square blocks that were actually stored.

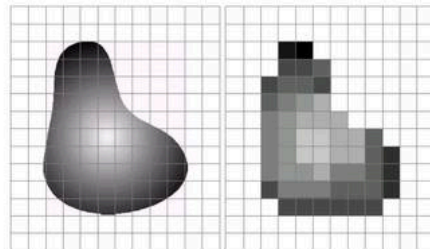
FROM YOUR LECTURE SLIDES

 138
of
36

Image Sampling And Quantisation (cont...)

Images taken from Gonzalez & Woods, Digital Image Processing (2002)

Remember that a digital image is always only an **approximation** of a real world scene



The same point again: the smooth object on the left, and what the sensor really recorded on the right.

Check yourself: an image is stored as a 512×512 array of type uint8. How many numbers is that, and how many bytes?

$512 \times 512 = 262,144$ numbers. "uint8" means one byte each, so 262,144 bytes, about 256 KB. If it were a colour image it would be three times that.

Check yourself: in $f(x, y)$, which one is the row?

x is the row and y is the column, with the origin at the top-left. Writing $f(2, 5)$ means row 2, column 5.

CHAPTER 2 HANDOUT 1, LECTURES 1-2 UNIT 1

What image processing is, and where it stops

Digital image processing means processing digital images with a digital computer. That sounds obvious, but it is worth saying, because the pictures in the first slides of your handout are digital pictures that were *not* made by image processing — a scanned photo is a digital image, but scanning it is not image processing.

Low, mid and high level

People argue about where image processing ends and computer vision begins. The useful way to settle it is to look at what goes *in* and what comes *out*.

LEVEL	INPUT	OUTPUT	EXAMPLES
Low	image	image	noise removal, sharpening, contrast improvement
Mid	image	attributes	segmentation, finding edges, recognising objects
High	attributes	understanding	scene understanding, self-driving navigation

Your course covers the low level fully and stops in the middle of the mid level. The high level is artificial intelligence, and is not part of EC341.

IN SIMPLE WORDS

If a picture goes in and a picture comes out, it is image processing. If a picture goes in and a *fact* comes out ("there are 4 cars"), you have crossed into image analysis. If facts go in and a *decision* comes out ("brake now"), that is computer vision.

Why we bother

There are only three real reasons to process an image, and every application in your handout is one of them:

1. **To help a human see better.** Removing noise, increasing contrast, removing blur, so that a doctor or an operator can see what matters.
2. **To help a machine decide.** Preparing the picture so that a program can count, measure, sort or recognise.
3. **To store or send it cheaply.** Images are enormous; compression makes them fit and travel.

A short history, in the order it happened

1826	The first photograph in the world, by Joseph Nicéphore Niépce ("View from the Window at Le Gras"). Not digital, but the start of everything.
Early 1920s	The Bartlane cable picture transmission system . Newspaper pictures were coded, sent by submarine cable between London and New York, and printed again by a telegraph printer at the other end. Sending a picture dropped from about a week to about three hours. Only 5 gray levels were used.
By 1929	Better reproduction methods raised the Bartlane system to 15 gray levels .
1948 onwards	The transistor (Bell Labs, 1948), the languages COBOL and FORTRAN in the 1950s and 60s, the integrated circuit (Texas Instruments, 1958), operating systems in the early 1960s, the microprocessor (Intel, early 1970s), the IBM personal computer (1981), then LSI, VLSI and ULSI.
1960s	The first pictures we would honestly call digital image processing: computers powerful enough to correct the distortion in the Moon pictures sent back by <i>Ranger 7</i> , and the beginning of medical imaging such as CT.

FOR THE EXAM

The Bartlane system is a favourite one-mark question. Remember three facts: newspaper industry, 1920s, submarine cable between London and New York — and that it started with 5 gray levels and improved to 15 by the end of 1929. Note the real point of the story: digital image processing had to wait for *digital computers*, because images need a lot of storage and computing power.

FROM YOUR LECTURE SLIDES

A Historical Overview of DIP



Newspaper industry used Bartlane cable picture transmission system to send pictures by submarine cable between London and New York in 1920s

27

A picture sent by the Bartlane cable system in the 1920s, printed at the receiving end by a telegraph printer. This is one of the first digital pictures ever transmitted.

Check yourself: is removing noise from an X-ray a low, mid or high level process?

Low level. A noisy image goes in and a cleaner image comes out — input and output are both images.

CHAPTER 3 HANDOUT 1, LECTURES 2-4 UNIT 1

Light, the EM spectrum, and where images come from

Images are made by energy hitting a sensor. Most of the time that energy is **electromagnetic (EM) radiation**. An EM wave can be thought of in two ways: as a wave of a certain wavelength, or as a stream of massless particles called **photons**, each carrying a fixed bundle of energy, all travelling at the speed of light.

If we line the spectrum up by energy per photon, we get the order you must remember:



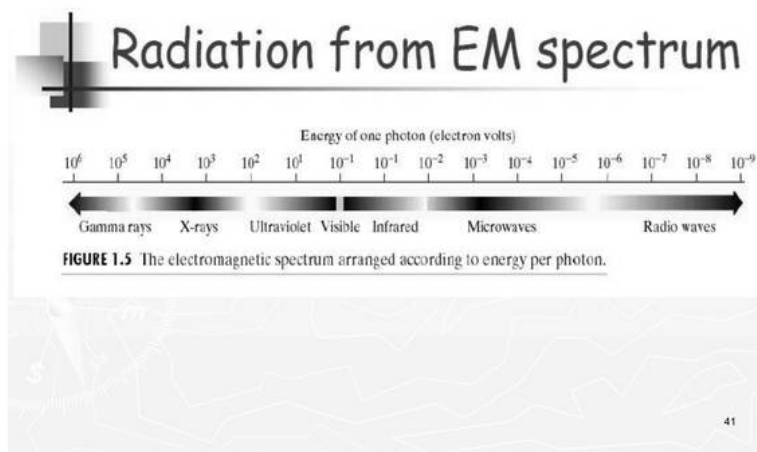
Figure 3.1 — Highest energy per photon on the left, lowest on the right. Wavelength goes the other way: gamma rays are the shortest, radio waves the longest.

IN SIMPLE WORDS

One line, memorised left to right: **gamma, X-ray, ultraviolet, visible, infrared, microwave, radio**. Energy falls as you go right, wavelength grows as you go right.

The visible band, the only part your eyes can see, is a very thin slice in the middle: roughly **0.43 μm to 0.79 μm** .

FROM YOUR LECTURE SLIDES



The textbook version of the same spectrum, arranged by energy per photon. Note the energy scale along the top, from 10^6 eV for gamma rays down to 10^{-9} eV for radio waves.

Which band is used for what

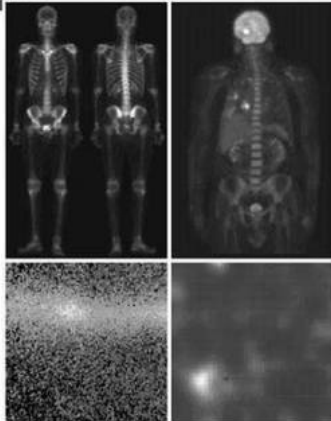
BAND	HOW THE IMAGE IS MADE	USED FOR
Gamma ray	A radioactive isotope is injected into the patient; detectors collect the gamma rays it gives off as it decays.	Nuclear medicine: bone scans to find infection or tumours; PET (positron emission tomography); astronomy; checking nuclear reactors.
X-ray	X-rays pass through the body and are partly absorbed; what is left falls on a film or detector, exactly like light developing a photograph.	Chest X-ray; angiography (a catheter injects a contrast medium into a blood vessel so blockages show up); CT scans; circuit board inspection; astronomy.
Ultraviolet	Often fluorescence: the material is made to glow when UV falls on it.	Lithography, industrial inspection, fluorescence microscopy (for example healthy corn against corn infected by smut), lasers, biological imaging, astronomy.
Visible & infrared	Ordinary reflected light, or heat radiation for infrared.	Light microscopy, remote sensing from satellites, weather prediction, law enforcement (number plates, fingerprints, CCTV), industrial inspection.
Microwave	Radar. The system sends its own pulse and listens for the echo, so it needs no sunlight at all and can see through cloud.	Mapping ground from aircraft and satellites.
Radio	Radio waves plus strong magnetic fields.	MRI, and radio astronomy.

Sources that are not EM waves at all

FROM YOUR LECTURE SLIDES

Gamma-Ray Imaging

a	b
c	d



- Nuclear Image
 - (a) Bone scan
 - (b) PET (Positron emission tomography) image
- Astronomical Observations.
 - (c) Cygnus Loop
- Nuclear Reaction
 - (d) Gamma radiation from a reactor valve

An area of strong radiation is seen in the lower left side of the image.

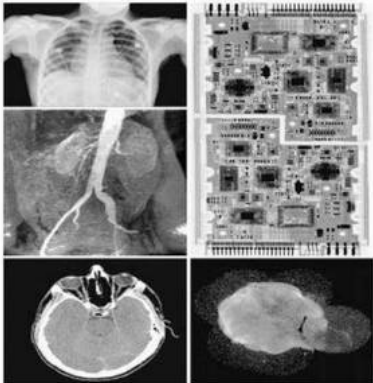
17

Gamma ray. (a) bone scan, (b) PET image, (c) the Cygnus Loop, (d) gamma radiation from a reactor valve.

FROM YOUR LECTURE SLIDES

X-ray Imaging

a	d
b	
c	e

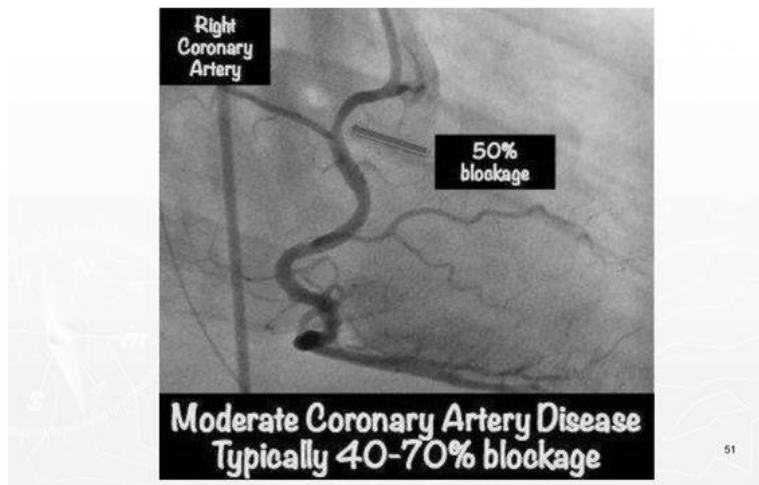


- Medical diagnostics
 - (a) chest X-ray (familiar)
 - (b) aortic angiogram
 - (c) head CT
- Industrial imaging
 - (d) Circuit board
- Astronomy
 - (e) Cygnus Loop

49

X-ray. (a) chest X-ray, (b) aortic angiogram, (c) head CT, (d) circuit board, (e) Cygnus Loop.

FROM YOUR LECTURE SLIDES



Angiography. A contrast medium injected through a catheter makes the blockage in the artery visible.

FROM YOUR LECTURE SLIDES

a	b
c	

Imaging in Ultraviolet Band

- Lithography
- Industrial inspection
- Microscopy (fluorescence)
 - (a) Normal corn
 - (b) Smut corn
- Lasers
- Biological imaging
- Astronomical observations
 - (c) Cygnus Loop

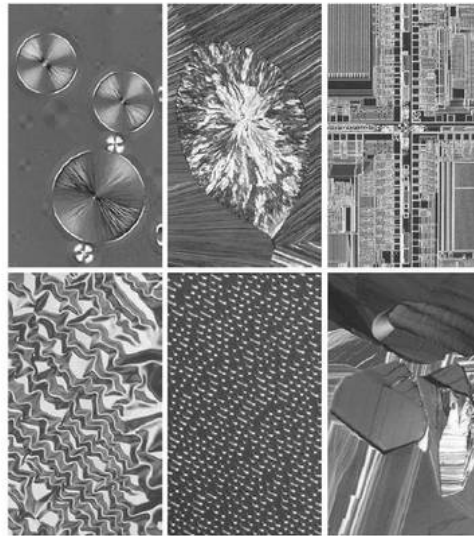
Ultraviolet. Fluorescence microscopy: (a) normal corn, (b) corn infected by smut.

FROM YOUR LECTURE SLIDES

a b c
d e f

FIGURE 1.9

Examples of light microscopy images.
 (a) Taxol (anticancer agent), magnified 250 $\times$.
 (b) Cholesterol—40 $\times$.
 (c) Microprocessor—60 $\times$.
 (d) Nickel oxide thin film—600 $\times$.
 (e) Surface of audio CD—1750 $\times$.
 (f) Organic superconductor—450 $\times$.
 (Images courtesy of Dr. Michael W. Davidson, Florida State University.)



Visible band, light microscopy. From pharmaceuticals to a microprocessor and the surface of an audio CD.

FROM YOUR LECTURE SLIDES

TABLE 1.1
Thematic bands
of NASA's
LANDSAT
satellite.

Band No.	Name	Wavelength (μm)	Characteristics and Uses
1	Visible blue	0.45–0.52	Maximum water penetration
2	Visible green	0.53–0.61	Measures plant vigor
3	Visible red	0.63–0.69	Vegetation discrimination
4	Near infrared	0.78–0.90	Biomass and shoreline mapping
5	Middle infrared	1.55–1.75	Moisture content; soil/vegetation
6	Thermal infrared	10.4–12.5	Soil moisture; thermal mapping
7	Short-wave infrared	2.09–2.35	Mineral mapping

The area imaged is Washington D.C., which includes features such as buildings, roads, vegetation, and a major river (the Potomac) going through the city.

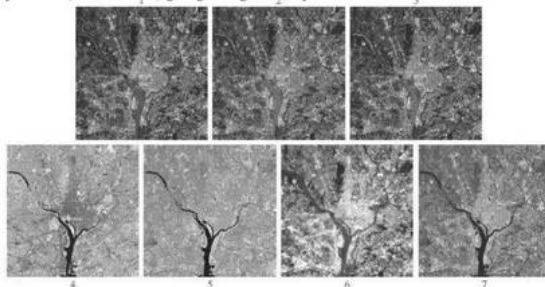
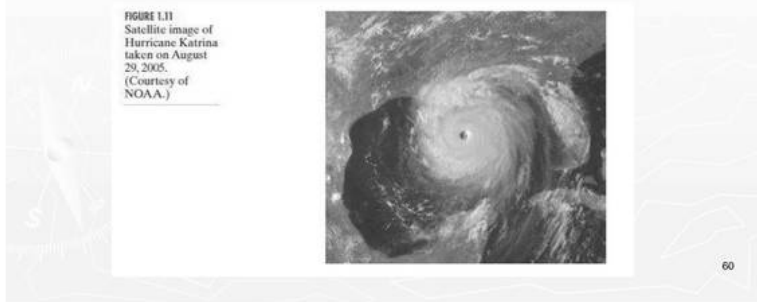


FIGURE 1.10 LANDSAT satellite images of the Washington, D.C. area. The numbers refer to the thematic bands in Table 1.1. (Images courtesy of NASA.)

Remote sensing. The seven thematic bands of the LANDSAT satellite and what each one is used for.

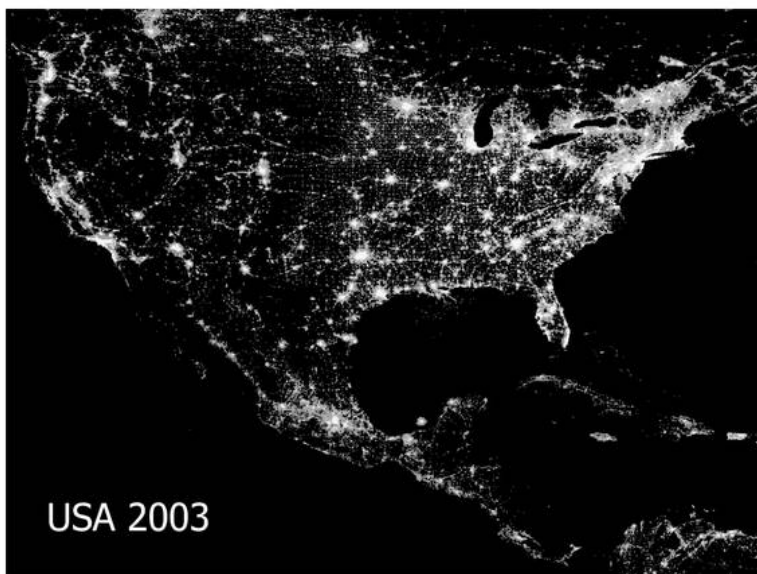
FROM YOUR LECTURE SLIDES

- ▶ Weather observation and prediction also are major applications of multispectral imaging from satellites. For example, Fig. shown below is an image of Hurricane Katrina, one of the most devastating storms in recent memory in the Western Hemisphere.
- ▶ This image was taken by a National Oceanographic and Atmospheric Administration (NOAA) satellite using sensors in the visible and infrared bands.
- ▶ **The eye of the hurricane is clearly visible in this image.**



Weather. Hurricane Katrina, imaged by a NOAA satellite in the visible and infrared bands.

FROM YOUR LECTURE SLIDES

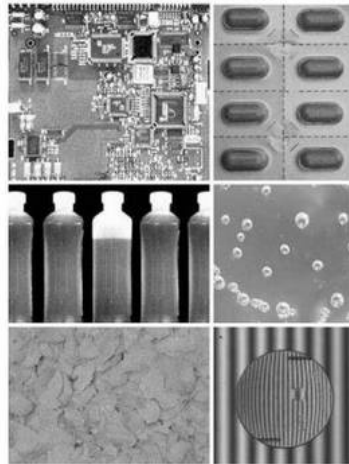


Infrared at night. The Nighttime Lights of the World data set, used to estimate electrical energy use.

FROM YOUR LECTURE SLIDES

■ Industry : visual spectrum

(automated visual inspection of manufactured goods)

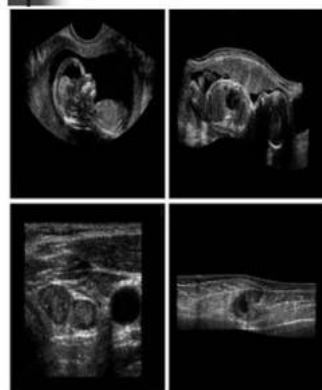


- | | |
|---|---|
| a | d |
| b | e |
| c | f |
- (a). A circuit board: inspect them for missing parts
 (b). Pill container: look for missing pills
 (c). Bottles : look for bottles that are not filled up to an acceptable level
 (d). Bubbles in clear-plastic product : detect unacceptable number of air pockets
 (e). Cereal : inspection for color and the presence of anomalies such as burned flake.
 (f). Image of replacement lens for human eye : inspection of damaged or incorrectly manufactured implants

Industrial inspection. Missing pills, half-filled bottles, air pockets in plastic, burned cereal flakes.

FROM YOUR LECTURE SLIDES

Ultrasound Imaging



- | | |
|---|---|
| a | b |
| c | d |
- Manufacturing
 - Medicine
 - (a) Baby
 - (b) Another view of baby
 - (c) Thyroids
 - (d) Muscle layers showing lesion

70
29

Ultrasound. Not an EM wave at all: (a), (b) a baby, (c) thyroid, (d) muscle layers showing a lesion.

- **Ultrasound.** A probe sends high-frequency sound (1 to 5 MHz) into the body. The sound bounces back at every boundary between tissues. The machine measures the time each echo takes, uses the speed of sound in tissue (about 1540 m/s) to turn that time into a distance, and draws the result as an image. Used for babies, thyroid, muscle lesions, and in manufacturing.
- **Electron microscopy.** A focused beam of electrons scans the surface of a sample (SEM), giving very high magnification.
- **Synthetic images.** Made entirely by a computer, for modelling and visualisation. No camera involved.

WORTH NOTICING

Remote sensing is why LANDSAT satellites carry several separate bands. Band 1 (visible blue) penetrates water best, band 2 (green) measures how healthy plants are, band 4 (near infrared) maps biomass and shorelines, band 6 (thermal infrared) shows soil moisture and heat. The same scene photographed in different bands answers completely different questions — for example a river shows up sharply in bands 4 and 5 and weakly in the visible bands.

Three words about light that get mixed up

Monochromatic light has no colour. Its only attribute is **intensity**, going from black through gray to white — which is exactly why monochromatic images are called gray-scale images.

Chromatic light is coloured light, and is described by three quantities:

Radiance	The <i>total amount of energy</i> that flows from the light source. Measured in watts (W).
Luminance	The <i>amount of that energy an observer actually perceives</i> . Measured in lumens (lm). Infrared light has high radiance but almost zero luminance for a human — you can feel it but not see it.
Brightness	A <i>subjective</i> feeling. It cannot be measured, because it lives in the viewer's head, not in the light.

FOR THE EXAM

"Distinguish between radiance, luminance and brightness" is a standard short question. The one-line answer: radiance is energy sent out (W), luminance is energy perceived (lm), brightness is a subjective impression that cannot be measured.

Check yourself: arrange in order of increasing wavelength — X-ray, radio, visible, gamma.

Gamma, X-ray, visible, radio. Energy per photon runs the opposite way: gamma has the most, radio the least.

Check yourself: why can radar work at night and through clouds, while an ordinary camera cannot?

Because radar supplies its own energy — it sends out a microwave pulse and measures the echo — so it does not depend on sunlight, and microwaves pass through cloud. A camera can only collect the visible light that happens to be there.

CHAPTER 4 HANDOUT 1, LECTURE 4 UNIT 1

How a camera turns a scene into numbers

Every imaging system, from a phone camera to a CT scanner, does the same two things: **illuminate** the scene with some kind of energy, and **collect** the energy that comes back.

The word "illumination" is wider than it sounds. For a photograph it is light from the sun or a lamp. For an X-ray it is the X-ray tube. For ultrasound it is a pulse of sound. For an electron microscope it is a beam of electrons. And "the scene" can be a family at a wedding, a skeleton, an unborn baby, or a molecule.

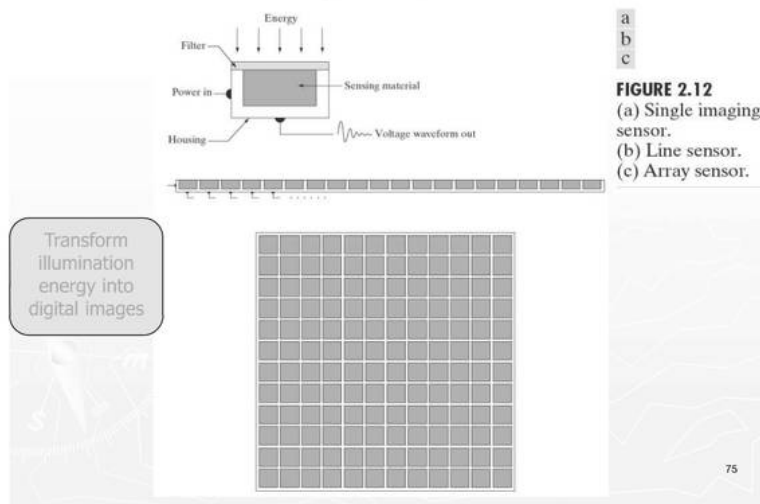
The three sensor arrangements

A single sensor gives one number. To get a whole picture you need to cover the plane, and there are exactly three ways to do it:

ARRANGEMENT	HOW IT COVERS THE IMAGE	WHERE YOU SEE IT
Single sensor	The one sensor must be moved in both directions, line by line, like a person mowing a lawn. Slow, but cheap and very precise.	Flatbed scanners, some microscopes
Line sensor (sensor strip)	A row of sensors covers one direction in one go; motion supplies the other direction.	Photocopiers, satellite imagers, CT scanners
Array sensor	A full 2-D grid of sensors. The whole image is captured in one shot, with no movement at all.	Every digital camera and phone

FROM YOUR LECTURE SLIDES

Image Acquisition



How illumination energy is turned into a digital image, and the three sensor arrangements: (a) a single sensor, (b) a line sensor, (c) an array sensor.

The simple image formation model

This is the one formula from this chapter that gets asked.

$$f(x, y) = i(x, y) \cdot r(x, y)$$

brightness = how much light falls on the point $\times$ what fraction that point sends back

where

- **$i(x, y)$** is the **illumination** — the amount of light from the source falling on the scene. Its range is $0 < i(x, y) < \infty$.
- **$r(x, y)$** is the **reflectance** (or transmissivity) — the fraction the object sends back. Its range is $0 < r(x, y) < 1$.

Reflectance of 0 means the surface absorbs everything (total absorption) and 1 means it reflects everything. It can never be more than 1, because an object cannot send back more light than it received.

Real numbers to quote

REFLECTANCE R	VALUE	ILLUMINATION I	VALUE
black velvet	0.01	clear day, sun on the Earth	$\approx 90,000 \text{ lm/m}^2$
stainless steel	0.65	cloudy day	$< 10,000 \text{ lm/m}^2$
flat-white wall paint	0.80	clear evening, full moon	$\approx 0.1 \text{ lm/m}^2$
silver-plated metal	0.90	typical office	$\approx 1,000 \text{ lm/m}^2$
snow	0.93		

FOR THE EXAM

Notice how wide the illumination range is: moonlight to sunlight is a factor of nearly a million, while reflectance only spans 0.01 to 0.93. So most of the variation in f comes from the lighting, not from the objects. That is exactly why images taken in poor light need enhancement, and it is the reason for the shading-correction trick in Chapter 10.

Sampling and quantization

The sensor output is still an analog signal — a smooth voltage. Two separate steps make it digital:

analog signal → SAMPLING → QUANTIZATION → digital image

Sampling digitises the coordinates. Quantization digitises the amplitude.

- **Sampling** decides *how many points* you measure along x and y . It fixes the size of the image, $M \times N$.
- **Quantization** decides *how many different brightness values* each measured point is allowed to take. It fixes the number of gray levels, L .

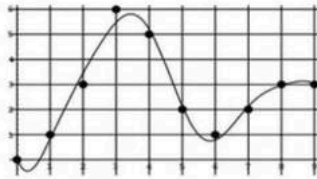
The classic picture in your handout shows a smooth curve taken along a line AB of an image: sampling picks values at ten positions ($x = 0 \dots 9$), then quantization pushes each of those values onto one of seven allowed levels ($y = 0 \dots 6$). Both steps lose information, and neither can be undone afterwards.

FROM YOUR LECTURE SLIDES

129
of
36

Sampling and Quantization

- When an image is sensed and converted into digital form, the image is discretized in two ways: sampling and quantization.
- The concepts of sampling and quantization can be understood by the following example in Fig., in which a one-dimensional signal is quantized on a uniform grid.



The signal is sampled at ten positions ($x = 0, \dots, 9$), and each sampled value is then quantized to one of seven levels ($y = 0, \dots, 6$).

Sampling and quantization on one signal. The dots are the ten sample positions; the horizontal grid lines are the seven levels each sample is forced onto.

IN SIMPLE WORDS

Sampling is *where* you measure. Quantization is *how finely* you measure. Mix them up and you will lose marks — remember: sampling $\rightarrow$ coordinates, quantization $\rightarrow$ amplitude.

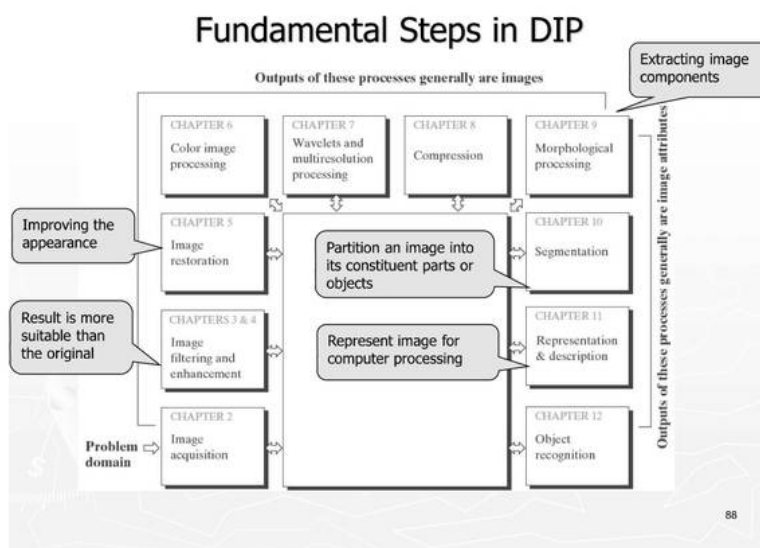
The fundamental steps of digital image processing

Your teacher's chart is the standard one from the textbook. It splits into two halves: the steps whose *output is an image*, and the steps whose *output is information about the image*.

#	STEP	WHAT IT DOES	OUTPUT
1	Image acquisition	Capture the picture, and any simple scaling done at capture.	image
2	Image enhancement	Make it look better for a chosen purpose.	image
3	Image restoration	Undo a known damage using a model of that damage.	image
4	Colour image processing	Work with colour models and colour images.	image
5	Wavelets and multiresolution	Represent the image at several sizes at once.	image
6	Compression	Reduce the storage needed.	image
7	Morphological processing	Extract shape-related components.	attributes
8	Segmentation	Cut the image into its objects and parts.	attributes
9	Representation and description	Describe each region — its boundary, or all its points — in a form a program can use.	attributes
10	Object recognition	Give each object a label.	understanding

Knowledge about the problem sits beside all of it and feeds every step — a program that knows it is looking at car number plates can ignore most of the picture.

FROM YOUR LECTURE SLIDES



The full chart from your handout. The left-hand branch produces images, the right-hand branch produces information about the image.

Enhancement is not restoration

These two get confused constantly, so learn the difference as one sentence each.

Enhancement **Subjective.** It is based on human preference about what looks good. There is no general theory of enhancement, because the viewer is the final judge. A method that is excellent for X-rays may be useless for satellite infrared pictures.

Restoration **Objective.** It is based on a mathematical or probabilistic model of how the image was damaged. You can measure whether it worked, because you are trying to reverse a known process.

FOR THE EXAM

"Differentiate between image enhancement and image restoration" is almost guaranteed somewhere. Answer with the pair of words **subjective** versus **objective**, then give one example each: stretching contrast because it pleases the eye (enhancement) against removing a blur whose cause is known (restoration).

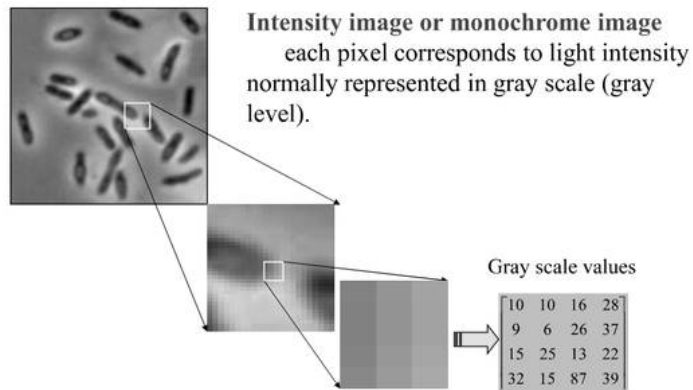
CHAPTER 5 HANDOUT 1, LECTURE 4 UNIT 1

Image types and file formats

The four image types

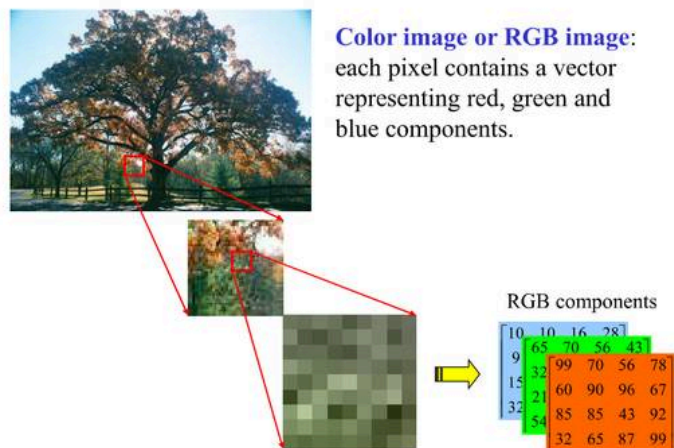
TYPE	WHAT ONE PIXEL HOLDS	ARRAY SHAPE	EXAMPLE VALUES
Intensity (monochrome, gray)	one number = light intensity	rows × cols	0 ... 255
Binary (black and white)	one bit	rows × cols	0 = black, 1 = white
RGB (colour)	a vector of three numbers	rows × cols × 3	(65, 70, 43)
Indexed	an index number pointing into a colour table	rows × cols + a palette	index 4 → (0.5, 0.5, 0.5)

FROM YOUR LECTURE SLIDES

Digital Image Types : Intensity Image

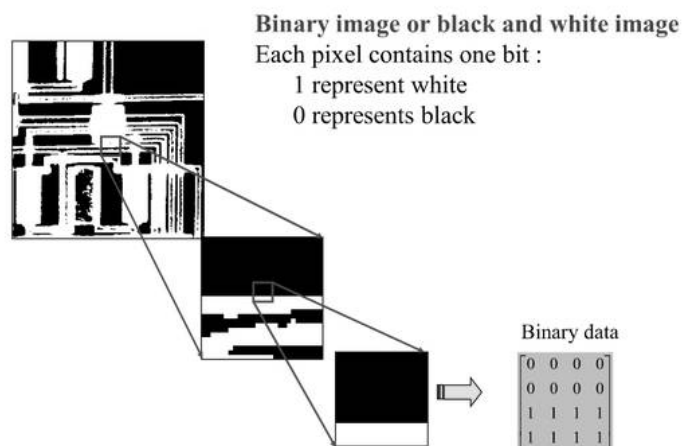
Intensity image. Zoom in and every pixel is a single gray-scale number.

FROM YOUR LECTURE SLIDES

Digital Image Types : RGB Image

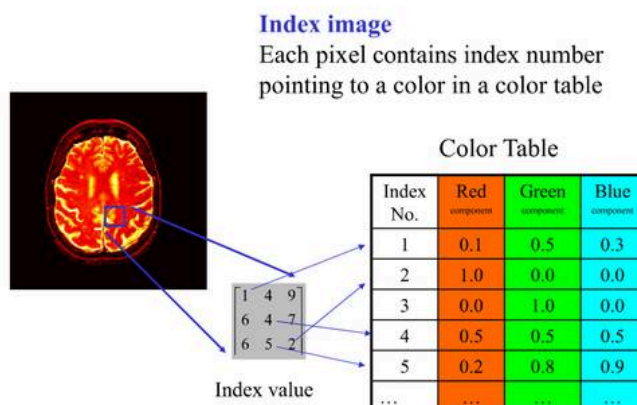
RGB image. Each pixel is a vector of three numbers, one per colour plane.

FROM YOUR LECTURE SLIDES

Image Types : Binary Image

Binary image. One bit per pixel: 1 is white, 0 is black.

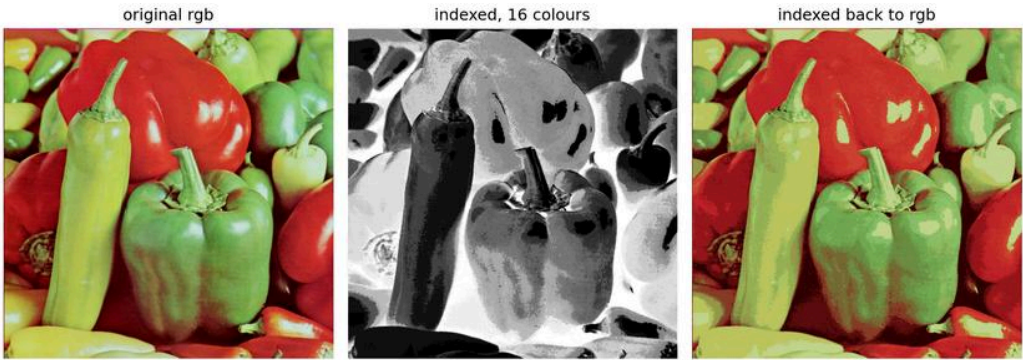
FROM YOUR LECTURE SLIDES

Image Types : Index Image

Indexed image. The pixel stores an index; the colour table holds the actual colours.

The indexed type is the one worth a second look. Instead of storing three colour numbers at every pixel, you store a small **colour table** (also called a palette or colour map) — say 16 or 256 colours — and each pixel stores only the *row number* of the colour it uses. With a 16-colour palette a pixel needs 4 bits instead of 24. That is where the saving comes from, and it is exactly how the GIF format works.

FROM YOUR ASSIGNMENT



Your Assignment 5 result: the original photograph, the indexed image (the middle picture is shown in gray because each pixel is now only a number from 0 to 15), and the picture rebuilt from the palette. The shading has turned into flat patches.

YOU HAVE ALREADY DONE THIS

Assignment 5, parts 6 and 7: you converted the peppers image to a 16-colour indexed image and back to RGB. Going back gave an average error of 9.57 gray levels — proof that the loss happens when the palette is chosen, and cannot be undone later.

Raster against vector

	RASTER (BITMAP)	VECTOR
Stored as	a rectangular array of sampled pixels	mathematical formulas for lines, curves and shapes
Pixel count	fixed	none — there are no pixels until it is drawn
When you zoom	interpolation is needed, and beyond some zoom factor the quality clearly breaks down into blocks and jagged edges	can be scaled to any size at any resolution with no loss at all
Good for	photographs	logos, illustrations, designs that get resized often
Formats	BMP, GIF, PNG, TIFF, JPEG	SVG, EPS

The letter "a" in your handout shows it best: zoomed in vector form the edges of the letter stay perfectly smooth; zoomed in raster form the same letter breaks into visible steps.

FROM YOUR LECTURE SLIDES

112 of 36 Classification of Digital Images

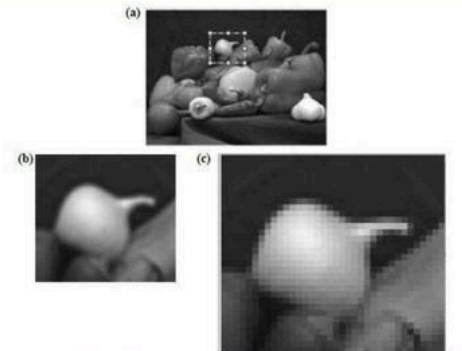


Fig. Zooming of a Bitmap image (a) Original image (b) Part of image zoomed 4 times (c) Same part of the image zoomed 8 times.
Original image reprinted with permission of The MathWorks, Inc.

Zooming a raster image: (a) the original, (b) part of it zoomed 4 times, (c) the same part zoomed 8 times. The quality clearly breaks down.

FROM YOUR LECTURE SLIDES

114 of 36 Classification of Digital Images

The degradation in the quality due to zooming is clearly visible on the boundaries of the character stored in raster format.

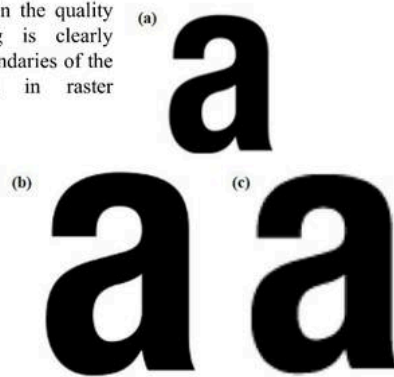


Fig. (a) An image (b) Image in (a) zoomed in vector format (c) Image in (a) zoomed in Raster format.

The same letter (b) in vector form and (c) in raster form, at the same zoom. The vector version has no pixels to break up, so its edges stay sharp.

YOU HAVE ALREADY DONE THIS

This is why your `my_imread()` refuses an SVG file with the message "svg is a vector format, it has no pixel matrix to read". There genuinely is no pixel matrix inside the file until something draws it at a chosen size.

The file formats you must be able to compare

FORMAT	COMPRESSION	COLOURS	USE IT WHEN
JPEG / JPG	lossy, high ratio	full colour	photographs and continuous-tone images with many colours
PNG	lossless — the uncompressed image is identical to the original	full colour	high-quality web graphics, transparency needed, small palettes, small files
GIF	lossless (LZW)	limited to an 8-bit palette, 256 colours	web animation, transparency, graphics with few colours: diagrams, logos, cartoons
TIFF	lossy or lossless; the details are stored inside the file	full colour	high-quality printing; the highest-quality formats available, but the files are large
BMP	none — uncompressed	full colour	simple, Microsoft's own format; large files
RAW	lossless	full sensor data	shooting and editing photos on a camera; needs the manufacturer's software

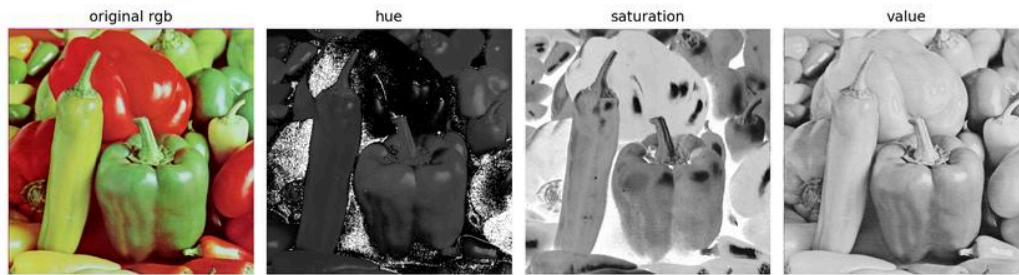
And the honest answer to "why are there so many formats?" from your handout: the reasons are mostly **money and practicality**. Some owners (CompuServe with GIF) wanted control of their format, so others refused and made their own.

FROM YOUR LECTURE SLIDES



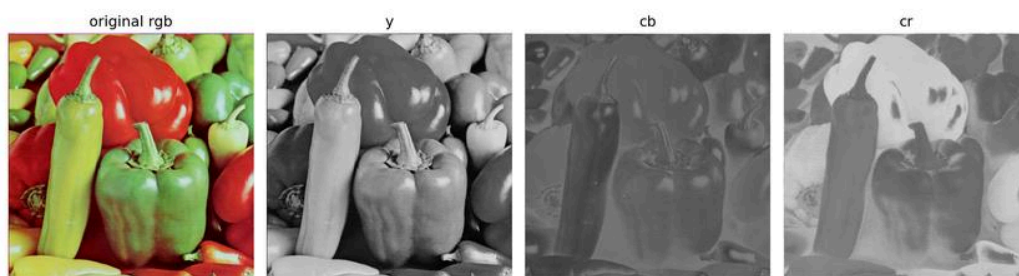
Your Assignment 5. RGB to grayscale, then to binary with two different thresholds. Otsu chose 119 automatically; a fixed 128 changed 9,071 pixels.

FROM YOUR LECTURE SLIDES



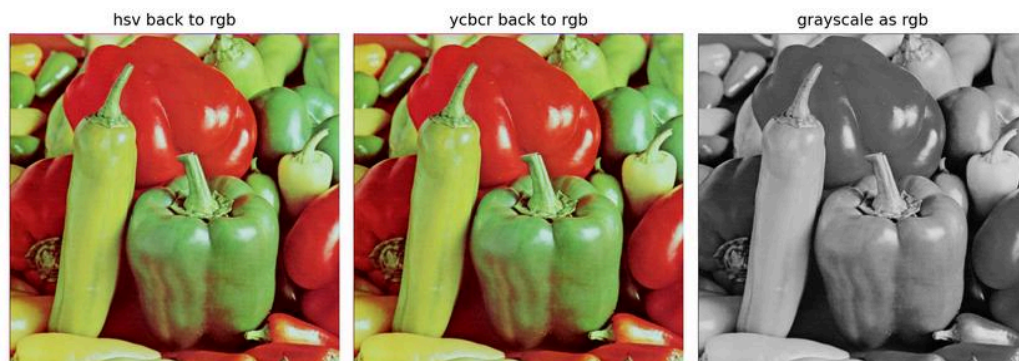
Your Assignment 5. The same picture split into hue, saturation and value. V looks like the gray image; the red peppers speckle in H because hue wraps around at red.

FROM YOUR LECTURE SLIDES



Your Assignment 5. Y, Cb and Cr. All the detail sits in Y, which is why JPEG can store Cb and Cr at half size.

FROM YOUR LECTURE SLIDES



Your Assignment 5. HSV and YCbCr converted back to RGB, and a gray image made three-channel. The last one stays gray, because copying a plane three times cannot invent colour.

Reading an image

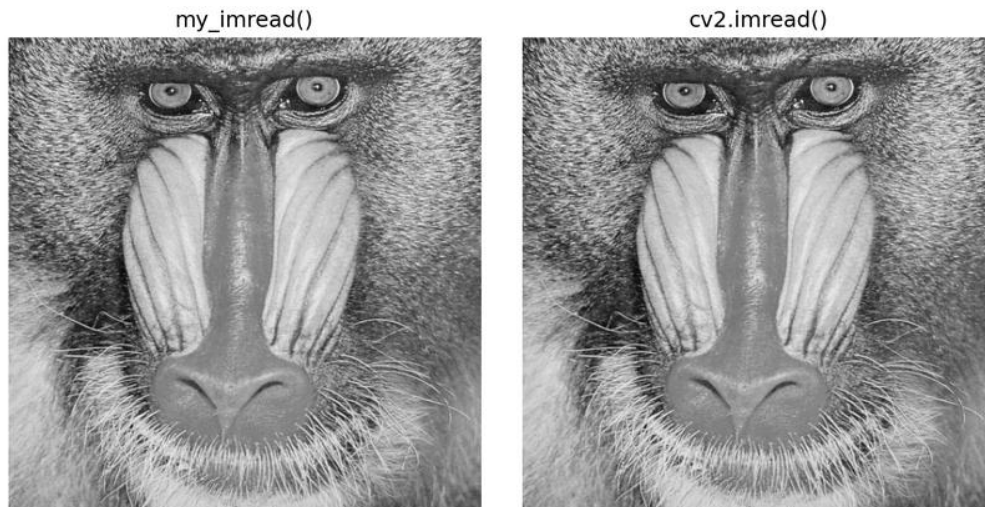
In MATLAB the whole job is one function:

```
>> F = imread('Penguins_grey.jpg'); >> [M, N] = size(F); rows and
columns >> whos F name, size, bytes, class >> imfinfo('Penguins_grey.jpg')
format, width, height, bit depth ...
```

- The filename must include the extension, and if the file is in another folder you must give the path.
- The semicolon at the end stops MATLAB printing the whole array on the screen.
- `>>` is just the MATLAB prompt, not something you type.

For a 768×1024 colour image, `whos` reports size $768 \times 1024 \times 3$, 2,359,296 bytes, class `uint8` — three bytes per pixel, which matches Chapter 1 exactly.

FROM YOUR ASSIGNMENT



Your own reader against the standard one. The left picture came out of `my_imread()`, which parsed the PGM header and pixels itself; the right one came from `cv2.imread()`. Zero differing pixels out of 262,144.

YOU HAVE ALREADY DONE THIS

Assignment 1 was this chapter turned inside out. Instead of calling `imread()`, you opened the file yourself, read the P2/P5 header, checked the magic number, pulled the pixels out and built the matrix — then proved it matched `cv2.imread()` exactly, with 0 differing pixels out of 262,144.

CHAPTER 6 HANDOUT 1, LECTURE 4 UNIT 1

Resolution: how many pixels, how many gray levels

An image has two completely separate kinds of resolution, and mixing them up is the most common mistake in this unit.

Spatial resolution

Spatial resolution is decided by how the sampling was carried out. In plain words it is the *smallest detail you can still make out* in the image. Vision people describe it as pixel size; printers and graphic designers describe the same thing as **dots per inch (DPI)**.

As you cut the number of samples, the picture first gets soft, then the individual pixels start to show as squares, and straight edges turn into staircases. That squared-off look is called the **checkerboard effect**.

Intensity level resolution

Intensity level resolution is the number of gray levels used to represent the image. The more levels, the finer the shading you can see. It is always quoted as a number of bits.

$$L = 2^k$$

L = number of gray levels, k = number of bits per pixel

BITS, K	LEVELS, L	EXAMPLE VALUES
1	2	0, 1
2	4	00, 01, 10, 11
4	16	0000, 0101, 1111
8	256	00110011, 01010101
16	65,536	1010101010101010

The intensity values are therefore whole numbers in the interval $[0, L - 1]$, and the storage needed for the whole image is:

$$b = M \times N \times k$$

b = number of bits, M = rows, N = columns, k = bits per pixel

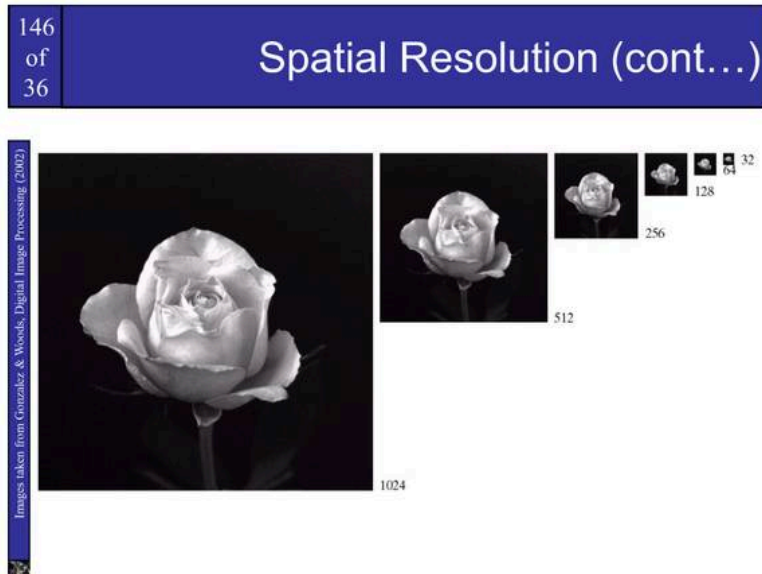
IN SIMPLE WORDS

Spatial resolution = **how many boxes** you cut the picture into. Intensity resolution = **how many shades** each box is allowed to be. One is about position, the other is about brightness.

What each one looks like when you reduce it

Reducing **spatial** resolution: the rose picture in your handout goes $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$, and the flower slowly turns into a mosaic of squares. Detail and texture disappear first, then edges become staircases, then the subject becomes unrecognisable.

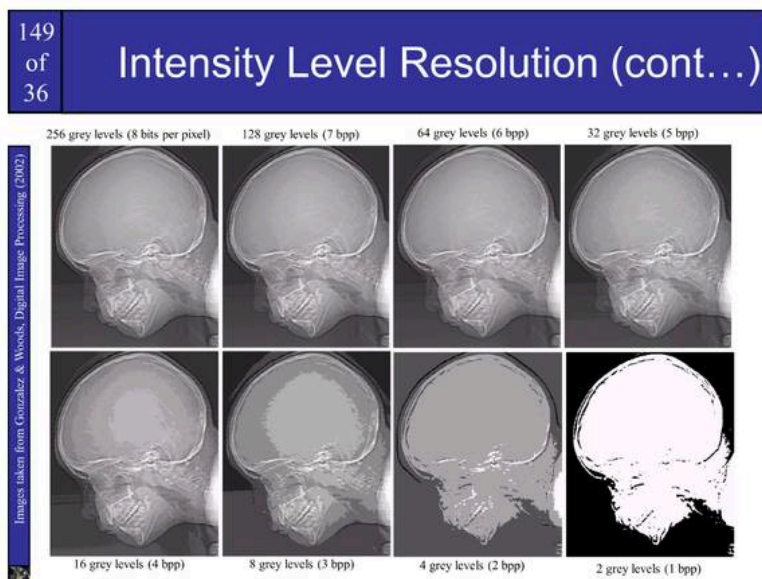
FROM YOUR LECTURE SLIDES



The rose at 1024, 512, 256, 128, 64 and 32 samples per side. Each step throws away three quarters of the remaining pixels.

Reducing **intensity** resolution: the X-ray skull goes $256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$ levels. Down to about 64 levels almost nothing changes. Below that, smooth areas break into flat patches with hard edges between them. Those false edges are not in the real scene at all, and they are called **false contouring**.

FROM YOUR LECTURE SLIDES



The same X-ray at 256 down to 2 gray levels. Watch the smooth forehead area: that is where the false contours appear first.

YOU HAVE ALREADY DONE THIS

Assignment 2 measured all of it. Going from 2024×2024 to 512×512 threw away **93.60 %** of the pixels and was almost invisible. False contouring appeared faintly at 6 bits, was clear at 5 bits and unmistakable at 4 bits. Your measured average errors: 3.15 gray levels for 512×512 against 4.85 for 4-bit — similar numbers, but the 4-bit picture looked far worse, because banding hurts the eye more than blur.

How much resolution is enough?

There is no single answer. It depends entirely on what is in the picture and what you want to do with it. The two questions to ask are: does the picture look acceptable, and *can you see the thing you need to see?*

Your handout's example says it perfectly: of two pictures of a car, the low-resolution one is perfectly fine for **counting cars**, but useless for **reading the number plate**. Same picture, same resolution, different job, different answer.

FROM YOUR LECTURE SLIDES



The picture on the right is fine for counting the number of cars, but not for reading the number plate

Enough resolution for one job is not enough for another.

Detail and gray levels are linked

The last set of pictures in the handout — the woman's face (low detail), the cameraman (medium detail) and the crowd (high detail) — makes a point that is easy to ask in an exam:

- Pictures with **a lot of fine detail** can survive with **fewer gray levels**, because the busy texture hides the banding.
- Pictures with **large smooth areas** (a face, a sky) need **more gray levels**, because there is nothing to hide the false contours.

So the two resolutions are not independent — the right choice for one depends on the other and on the content.

FROM YOUR LECTURE SLIDES

152
of
36

Intensity Level Resolution (cont...)



Low, medium and high detail. The crowd picture on the right can survive with far fewer gray levels than the face on the left, because its texture hides the banding.

FOR THE EXAM

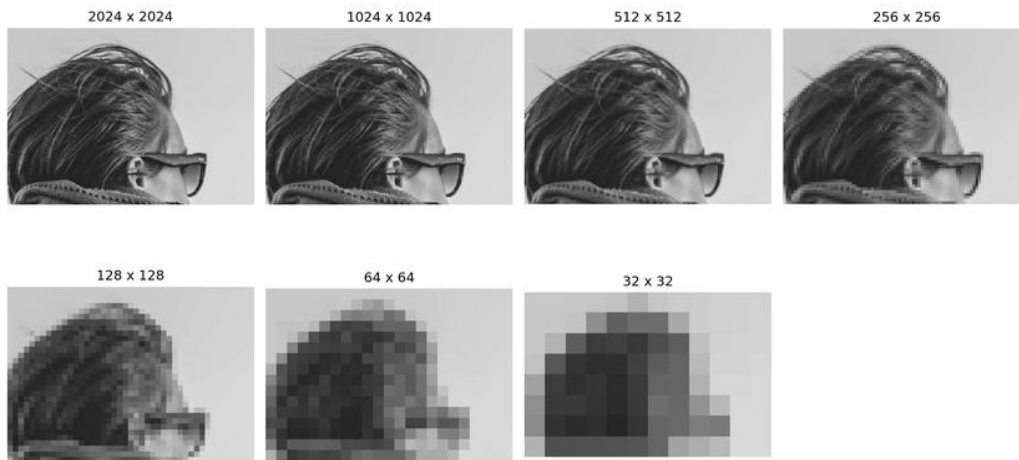
A numerical question in this area is very likely. Practise this one until it is automatic:
"How many bits are needed to store a 1024×1024 image with 256 gray levels?" $\rightarrow k = 8$
 because $256 = 2^8$, so $b = 1024 \times 1024 \times 8 = 8,388,608$ bits = 1,048,576 bytes = 1 MB.
 Always convert: 8 bits = 1 byte, 1024 bytes = 1 KB.

FROM YOUR LECTURE SLIDES



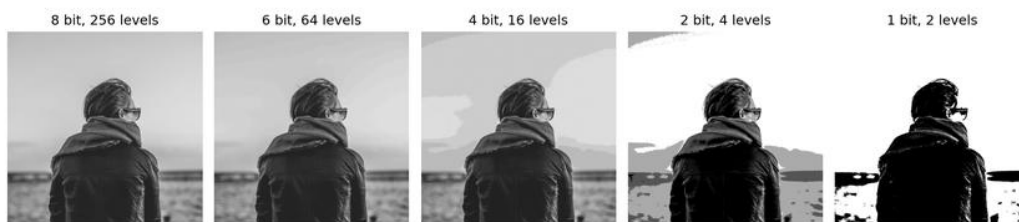
Your Assignment 2. The 2048×2048 photograph at seven spatial resolutions, all printed the same size. The first three look identical here, which is the point.

FROM YOUR LECTURE SLIDES



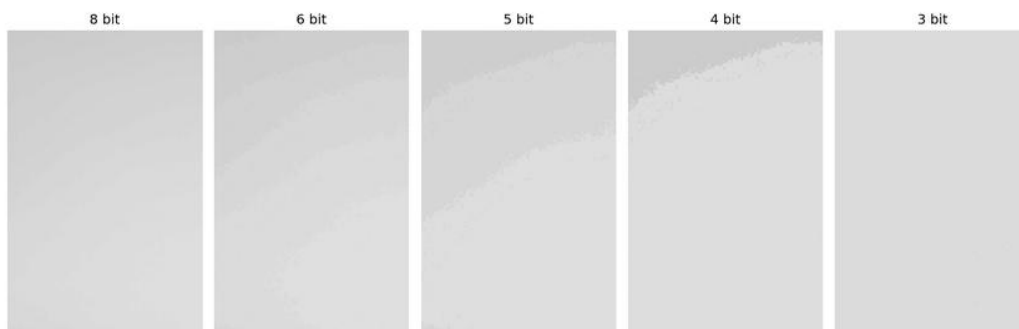
Your Assignment 2. The same head and glasses cut from each of those seven images. Now the loss is obvious: hair strands merge, then the glasses become a smear.

FROM YOUR LECTURE SLIDES



Your Assignment 2. The same picture at 8, 6, 4, 2 and 1 bits per pixel. At 4 bits the sky has broken into hard-edged patches.

FROM YOUR LECTURE SLIDES



Your Assignment 2. A smooth piece of sky at 8, 6, 5, 4 and 3 bits. This is where you measured that banding starts faintly at 6 bits and is clear at 5.

Check yourself: an image is 512×512 with 32 gray levels. How many bits does it need?

$32 = 2^5$, so $k = 5$. $b = 512 \times 512 \times 5 = 1,310,720$ bits, which is 163,840 bytes $\approx$ 160 KB.

Check yourself: name the effect you see when too few gray levels are used, and where you notice it first.

False contouring (also called posterization). You see it first in smooth, slowly changing areas such as sky, skin or a plain wall, because there the quantization steps become visible as bands with hard edges.

Check yourself: going from 1024×1024 to 256×256 , what percentage of pixels is lost?

Pixels go from 1,048,576 to 65,536. Lost = 983,040, and $983,040 / 1,048,576 \times 100 = 93.75\%$. Notice the side is divided by 4 but the count by 16.

CHAPTER 7 HANDOUT 2, LECTURES 5-7 UNIT 1

Neighbours and adjacency

Until now a pixel was a lonely number. This chapter is about how a pixel relates to the pixels around it, and it is the foundation of everything later in the course — segmentation, morphology and object counting all rest on it.

The neighbours of a pixel

Take a pixel p at position (x, y) . Three neighbourhoods are defined.

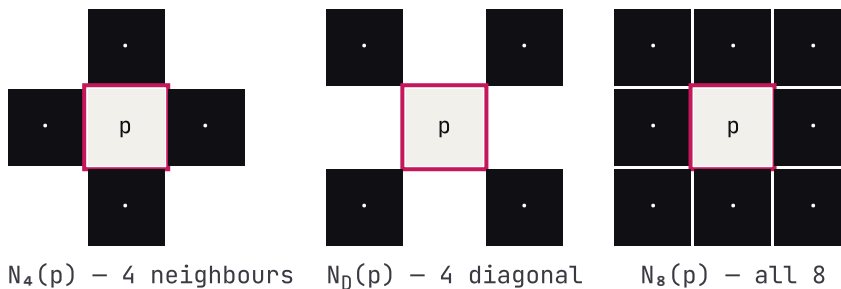
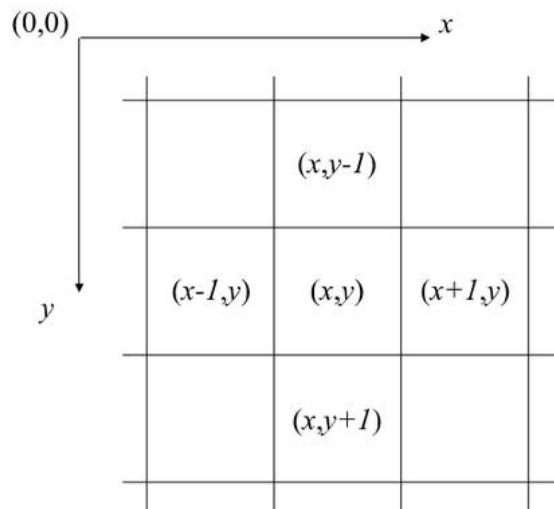


Figure 7.1 — The three neighbourhoods of a pixel p .

FROM YOUR LECTURE SLIDES

Basic Relationship of Pixels

➤ **4-neighbors of p** , denoted by $N_4(p)$:
 $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, and $(x, y+1)$.



Conventional indexing method

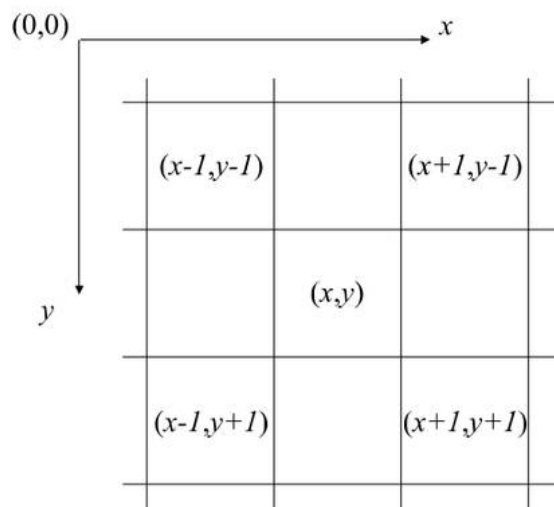


$N_4(p)$. The four pixels that share a side, with the coordinate labels used in the slides.

FROM YOUR LECTURE SLIDES

Basic Relationship of Pixels

➤ **4 diagonal neighbors of p** , denoted by $N_D(p)$:
 $(x-1, y-1)$, $(x+1, y+1)$, $(x+1, y-1)$, and $(x-1, y+1)$.

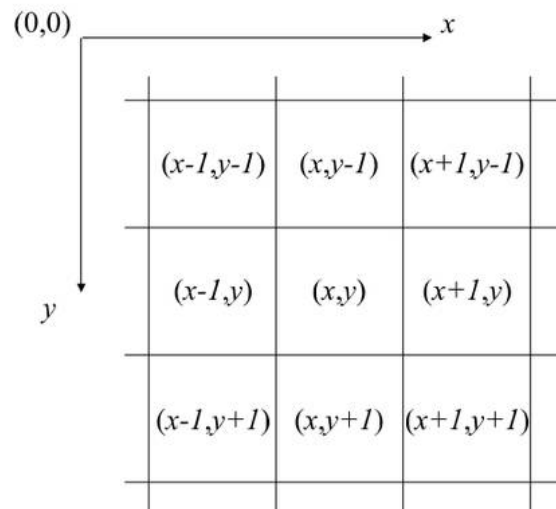


Conventional indexing method



$N_D(p)$. The four pixels that touch only at a corner.

FROM YOUR LECTURE SLIDES

Basic Relationship of Pixels

Conventional indexing method

► **$N_8(p)$.** Both sets together, all eight surrounding pixels.

- $N_4(p)$ The **4-neighbours**: the pixels above, below, left and right. Their coordinates are $(x-1, y)$, $(x+1, y)$, $(x, y-1)$ and $(x, y+1)$. Each is exactly one unit away from p .
- $N_D(p)$ The **4 diagonal neighbours**: $(x-1, y-1)$, $(x+1, y+1)$, $(x+1, y-1)$ and $(x-1, y+1)$. They touch p only at a corner.
- $N_8(p)$ The **8-neighbours**, which is simply the two sets joined: $N_8(p) = N_4(p) \cup N_D(p)$.

FOR THE EXAM

Always add the sentence: "if p is a border pixel, some of these neighbours fall outside the image, so it has fewer neighbours." Examiners look for it, and it is the reason every program needs a bounds check.

The set V — adjacency is not only about position

Here is the idea students most often miss. Two pixels sitting next to each other are **not automatically adjacent**. First we must fix V , the set of intensity values that we are calling "the object".

- In a **binary** image, if we are talking about the adjacency of the pixels with value 1, then $V = \{1\}$.

- In a **gray-scale** image the idea is the same, but V usually contains more elements. With values 0 to 255, V could be any subset of those 256 values — for example $V = \{1, 2\}$, or $V = \{200, \dots, 255\}$ for "bright pixels".

IN SIMPLE WORDS

V is your definition of "belongs to the object". Two pixels are adjacent only if **both of their values are in V** and they are positioned close enough. Change V and the same picture can have completely different objects.

The three kinds of adjacency

Let p and q both have values from V .

NAME	CONDITION	IN WORDS
4-adjacency	$q \in N_4(p)$	they share a full side
8-adjacency	$q \in N_8(p)$	they share a side or just a corner
m-adjacency (mixed adjacency)	(i) $q \in N_4(p)$, or (ii) $q \in N_D(p)$ and the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from V	a side touch always counts; a corner touch counts <i>only if</i> there is no way round through a shared side-neighbour that is also in the object

Why m-adjacency was invented

8-adjacency has an annoying defect: it can connect two pixels in more than one way at the same time, so "the path between p and q " is not a single, definite thing. m-adjacency removes that ambiguity. Work through the standard example, which is in your handout and is asked again and again.

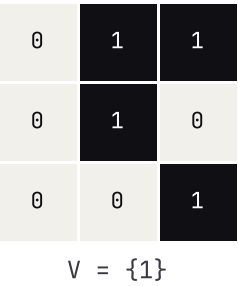


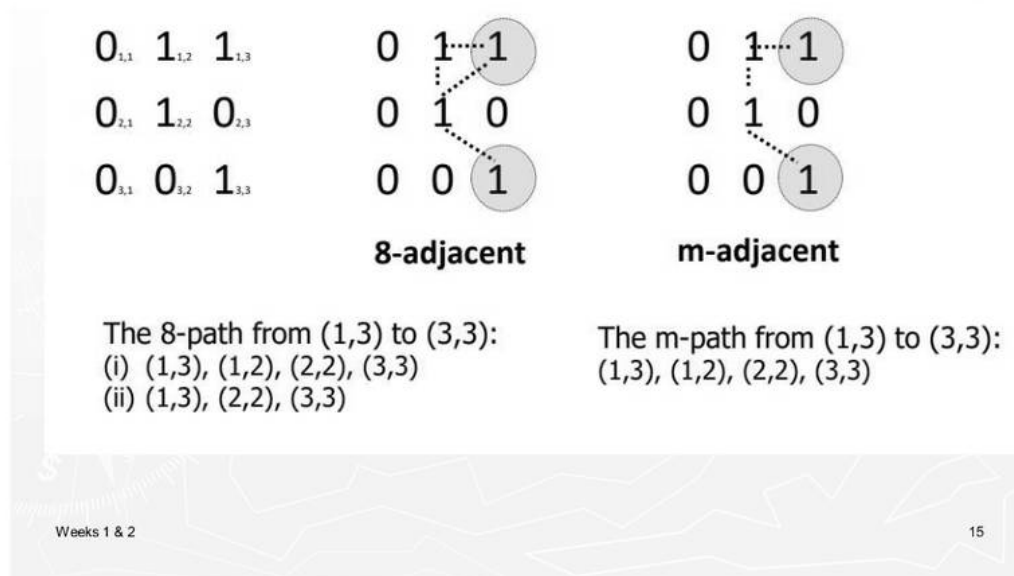
Figure 7.2 — The object pixels sit at $(0,1)$, $(0,2)$, $(1,1)$ and $(2,2)$, writing positions as (row, column) counting from 0.

PAIR	HOW THEY TOUCH	4-ADJ?	8-ADJ?	M-ADJ?
(0,1) – (0,2)	side	yes	yes	yes
(0,1) – (1,1)	side	yes	yes	yes
(0,2) – (1,1)	corner	no	yes	no
(1,1) – (2,2)	corner	no	yes	yes

Look carefully at the two corner pairs, because they behave differently:

- **(0,2) and (1,1) are not m-adjacent.** They are diagonal, so we must test condition (ii). Their common 4-neighbours are (0,1) and (1,2). The value at (0,1) is 1, which *is* in V — so a way round already exists, and the corner step is refused.
- **(1,1) and (2,2) are m-adjacent.** Their common 4-neighbours are (1,2) and (2,1), and both are 0, so nothing is in V — no way round exists, and the corner step is allowed.

FROM YOUR LECTURE SLIDES



The same example in your slides. On the left the two 8-adjacent routes are drawn; on the right m-adjacency has removed one of them, and the paths from (1,3) to (3,3) are listed underneath. (The slides count rows and columns from 1, so their (1,3) is our (0,2).)

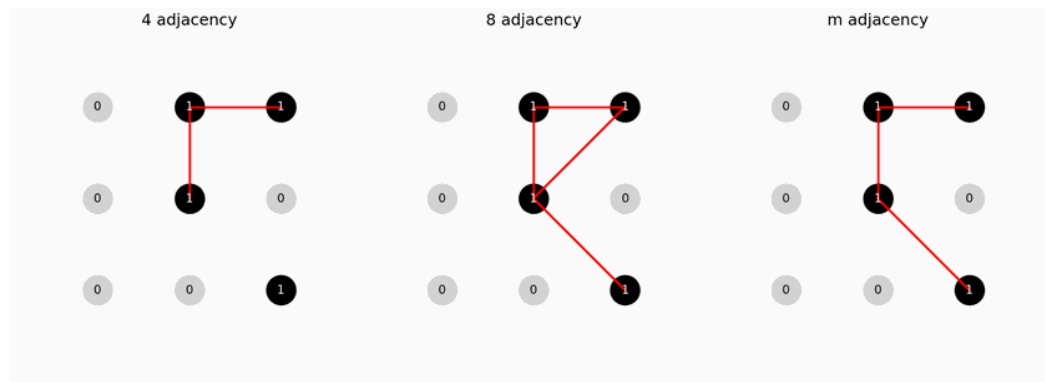
FOR THE EXAM

When you test m-adjacency, always write the intersection set out explicitly: " $N_4(p) \cap N_4(q) = \{(1,2), (2,1)\}$, both are 0, none in V , therefore m-adjacent." Marks are given for that line, not just for the final yes or no.

YOU HAVE ALREADY DONE THIS

Assignment 4 printed exactly this table. Your program found 2 four-adjacent pairs, 4 eight-adjacent pairs and 3 m-adjacent pairs for this same image — the missing one being (0,2)–(1,1).

FROM YOUR ASSIGNMENT



Your own program drew this. Each red line joins a pair the rule calls adjacent. The 8-adjacency picture in the middle contains a closed triangle, which is the ambiguity; the m-adjacency picture on the right is a simple chain.

CHAPTER 8 HANDOUT 2, LECTURES 5–7 UNIT 1

Paths, connectivity, regions and boundaries

Path

A **digital path** (or curve) from pixel p at (x, y) to pixel q at (s, t) is a sequence of distinct pixels

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

where $(x_0, y_0) = (x, y)$, $(x_n, y_n) = (s, t)$, and each pixel is adjacent to the next one, for $1 \leq i \leq n$

Here n is the **length of the path** — the number of steps, not the number of pixels. If the start and the end are the same pixel, the path is a **closed path**. And because there are three kinds of adjacency, there are three kinds of path: **4-paths**, **8-paths** and **m-paths**.

Worked example: the ambiguity, with numbers

Take the same image as Figure 7.2 with $V = \{1\}$, and go from (0,2) to (2,2).

USING	SHORTEST PATH	LENGTH	HOW MANY DIFFERENT PATHS EXIST?
4-adjacency	none exists	—	0
8-adjacency	$(0,2) \rightarrow (1,1) \rightarrow (2,2)$	2	2 — the other one is $(0,2) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,2)$
m-adjacency	$(0,2) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,2)$	3	1

That is the whole point of m-adjacency in one table: with 8-adjacency there are two competing paths, so "the path" is ambiguous; with m-adjacency there is exactly one. The price is that the m-path is longer.

FROM YOUR LECTURE SLIDES

```

0  1  1
0  1  0
0  0  1

```

```

0  1--1
0  1  0
0  0  1

```

8- neighbour of
the center pixel

```

0  1--1
0  1  0
0  0  1

```

m- neighbour
of the center
pixel

- m-connectivity eliminates the multiple path connections that arise in 8-connectivity.



The slide that states it directly: m-connectivity eliminates the multiple path connections that arise in 8-connectivity.

FOR THE EXAM

An m-path can never be *shorter* than the 8-path, because m-adjacency is a subset of 8-adjacency — it only ever removes links, never adds them. If a question gives you both, and your m-path comes out shorter, you have made a mistake.

Connectivity

Connectivity is built directly on adjacency. Let S be a subset of pixels of an image. Two pixels p and q are **connected in S** if there is a path between them consisting *entirely* of pixels in S .

- 4-connected p and q are 4-connected if $q \in N_4(p)$ and both are in the same class of values.
- 8-connected same with $N_8(p)$.
- m -connected $q \in N_4(p)$, or $q \in N_D(p)$ and $N_4(p) \cap N_4(q) = \emptyset$ for pixels from V .

Then three definitions follow, and they are worth learning word for word:

FROM YOUR LECTURE SLIDES

Examples: Adjacency and Path

$$V = \{1, 2\}$$

0 1 1
0 2 0
0 0 1

0 1 1
0 2 0
0 0 1

0 1 1
0 2 0
0 0 1



A second example from the slides, this time with $V = \{1, 2\}$. Changing V changes which pixels count as the object, and so changes every path.

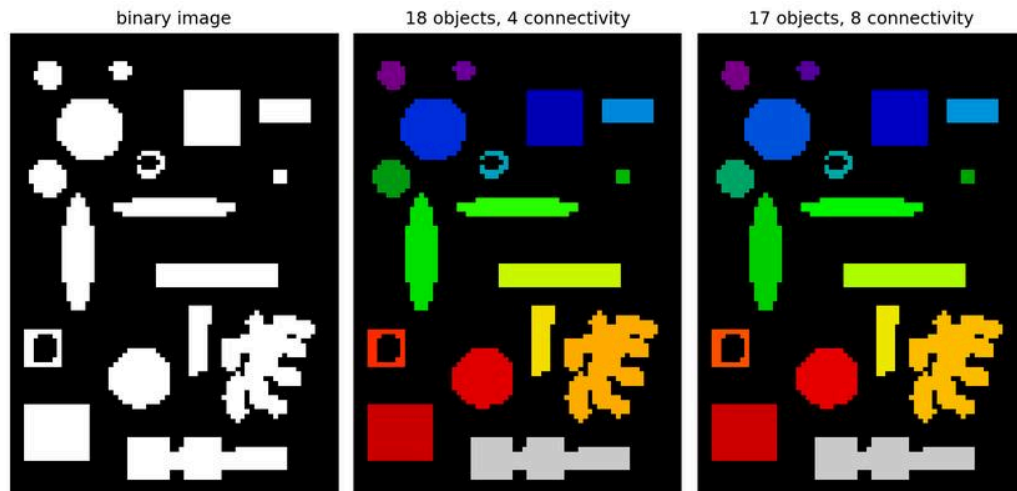
- For any pixel p in S , the set of all pixels connected to p is called a **connected component** of S .
- If S has only one connected component, S is called a **connected set**.
- A subset R of the image is called a **region** of the image if R is a connected set.

Two regions R_i and R_j are **adjacent** if their union forms a connected set. Regions that are not adjacent are called **disjoint**.

YOU HAVE ALREADY DONE THIS

Assignment 4 labelled a real binary picture three times. 4-connectivity found **18** objects, 8-connectivity found **17**, and m -connectivity also found 17. The extra object under 4-connectivity was a single pixel touching its shape only at a corner. m -connectivity always gives the same components as 8-connectivity, because it only removes a corner link when a route round already exists — so it changes the *paths*, never the *objects*.

FROM YOUR ASSIGNMENT



Your own result. Same binary picture, same program, only the connectivity rule changed — and the object count changed with it.

Boundary, foreground and background

- Boundary** The **boundary** (also called border or contour) of a region R is the set of pixels in R that have at least one neighbour *not* in R . If R happens to be the entire image, its boundary is defined as the set of pixels in the first and last rows and columns.
- Foreground** If an image has K disjoint regions R_k , then the union of all K regions is the **foreground**.
- Background** The complement of the foreground, written $(R_u)^c$, is the **background**.

One subtle rule from your handout: you must always say *which* connectivity you mean. As a rule, adjacency between points *inside* a region is defined using 8-connectivity, and the relation between the region and its background is then defined using 4-connectivity — this pairing avoids the contradictions that appear if you use the same rule for both.

The four questions from your handout, answered

FROM YOUR LECTURE SLIDES

Question 1

- In the following arrangement of pixels, are the two regions (of 1s) adjacent? (if 8-adjacency is used)

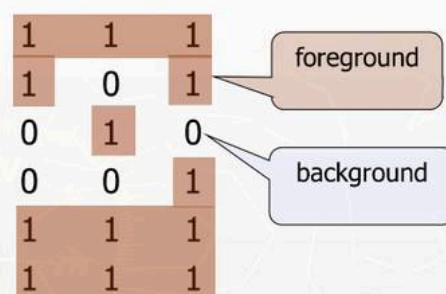


24

Question 1 as it appears in your slides. Two regions of 1s, and the question of whether they are adjacent under 8-adjacency.

FROM YOUR LECTURE SLIDES

- In the following arrangement of pixels, the two regions (of 1s) are disjoint (if 4-adjacency is used)



27

Foreground and background. The same arrangement, with the regions of 1s marked as foreground and everything else as background.

Q1. In this arrangement, are the two regions of 1s adjacent, if 8-adjacency is used?

1 1 1 / 1 0 1 / 0 1 0 / 0 0 1 / 1 1 1 / 1 1 1

Yes. With 8-adjacency a corner touch counts, so the lower edge of Region 1 and the upper edge of Region 2 join through diagonal contact, and their union is a connected set. Being a connected set is exactly the definition of adjacent regions.

Q2. The same arrangement — are the two parts of 1s adjacent if 4-adjacency is used?

No. Under 4-adjacency only full side contact counts. The two parts touch only at corners, so no path of 4-adjacent object pixels crosses from one to the other. Their union is not connected, so they are two disjoint regions, not one.

Q3. Is the circled pixel part of the boundary of the 1-valued pixels, if 8-adjacency is used?

Test the definition: does the pixel have at least one *neighbour* that is not in the region? With 8-adjacency we look at all 8 surrounding pixels. If all eight are 1s, the pixel is interior, so it is **not** on the boundary. It is on the boundary only if at least one of the eight is a 0.

Q4. The same circled pixel, if 4-adjacency is used?

Now only the 4 side neighbours are examined. A pixel whose four side neighbours are all 1s is interior under 4-adjacency, even if one of its diagonal neighbours is 0. This is why the answer to Q3 and Q4 can differ for the same pixel — **the boundary depends on the connectivity you choose**, which is the lesson of both questions.

CHAPTER 9 HANDOUT 2, LECTURE 7 UNIT 1

Distance between pixels

How far apart are two pixels? There is more than one sensible answer, and the course uses three.

First, what makes a function a distance at all

For pixels p, q and z with coordinates (x, y) , (s, t) and (u, v) , a function D is a **distance function** (a metric) only if it satisfies all three conditions:

- (a) $D(p, q) \geq 0$, and $D(p, q) = 0$ only if $p = q$ (b) $D(p, q) = D(q, p)$ it does not matter which way round you measure (c) $D(p, z) \leq D(p, q) + D(q, z)$ a detour is never shorter than going direct

The three distances

$$\text{Euclidean: } D_e(p, q) = [(x - s)^2 + (y - t)^2]^{1/2} \quad \text{City-block: } D_4(p, q) = |x - s| + |y - t| \quad \text{Chessboard: } D_8(p, q) = \max(|x - s|, |y - t|)$$

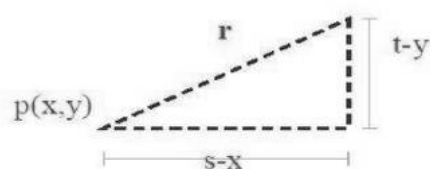
- Euclidean** The straight-line distance, the one from school geometry. The pixels at a distance $\leq r$ from (x, y) form a **disk of radius r** centred on it.
- D_4 , city-block** The smallest number of elementary steps if **only horizontal and vertical moves are allowed**. It is called city-block because it is like walking in a city laid out in a rectangular grid of streets and closed blocks of houses — you cannot cut through a building. Equal-distance pixels form a **diamond**. The pixels with $D_4 = 1$ are exactly the 4-neighbours of (x, y) .
- D_8 , chessboard** Diagonal moves are now allowed and cost the same as straight ones, so it equals the number of moves a **king** needs on a chessboard to get from one square to another. Equal-distance pixels form a **square**. The pixels with $D_8 = 1$ are exactly the 8-neighbours of (x, y) .

FROM YOUR LECTURE SLIDES

Euclidean distance between p and q

$$D_e(p, q) = [(x - s)^2 + (y - t)^2]^{1/2}$$

$q(s, t)$



radius (r) centered at (x, y)

For this distance measure, the pixels having a distance less than or equal to some value r from (x, y) are the points contained in a disk of radius r centered at (x, y) .



Euclidean distance. The straight line between p and q , and the disk of radius r it defines.

FROM YOUR LECTURE SLIDES

City-block distance: D_4 distance

In this case, the pixels having a D_4 distance from (x, y) less than or equal to some value r form a diamond centered at (x, y) . For example, the pixels with D_4 distance ≤ 2 from (x, y) (the center point) form the following contours of constant distance:

$$D_4(p, q) = |x - s| + |y - t|$$

		2		
	2	1	2	
2	1	0	1	2
	2	1	2	
		2		

- diamond centered at (x, y)
- $D_4 = 1$ are 4-neighbors of (x, y)

City-block distance D_4 . The contours of constant distance form a diamond.

FROM YOUR LECTURE SLIDES

Chessboard distance: D_8 distance

$$D_8(p, q) = \max(|x - s|, |y - t|)$$

In this case, the pixels with D_8 distance from (x, y) less than or equal to some value r form a square centered at (x, y) . For example, the pixels with D_8 distance ≤ 2 from (x, y) (the center point) form the following contours of constant distance:

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

square centered at (x, y)

Chessboard distance D_8 . The contours form a square.

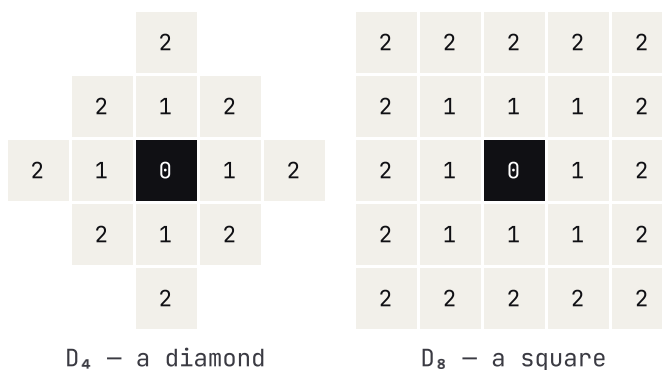


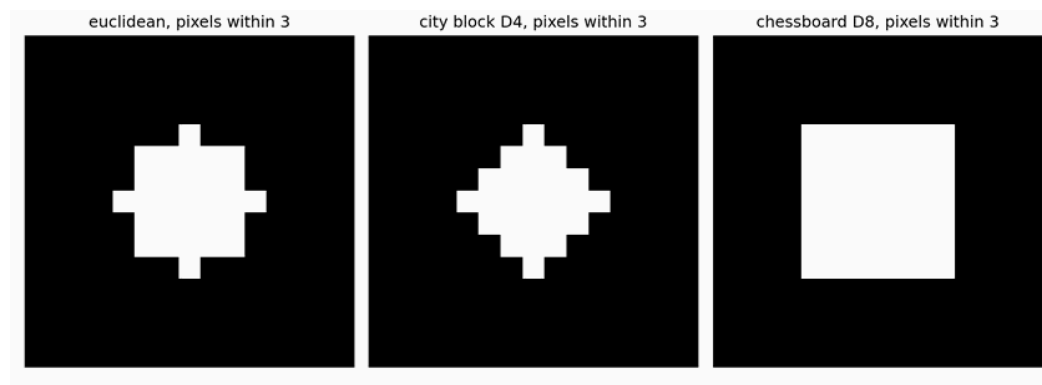
Figure 9.1 — Contours of constant distance from the centre pixel. Draw these two grids from memory in the exam; they answer half the questions on this topic by themselves.

For any pair of pixels the three always come in the same order:

$$D_8(p, q) \leq D_e(p, q) \leq D_4(p, q)$$

The reason is simple: a diagonal step costs 1 under D_8 , about 1.414 under D_e , and a full 2 under D_4 .

FROM YOUR ASSIGNMENT



Your own figure: the pixels lying within a distance of 3 from the centre. A disc for Euclidean, a diamond for D_4 , a square for D_8 — exactly the shapes the contour tables predict.

YOU HAVE ALREADY DONE THIS

Assignment 4(b) worked out $p = (2,1)$ and $q = (6,4)$: row difference 4, column difference 3, so $D_e = \sqrt{(16+9)} = 5.0$, $D_4 = 4+3 = 7$, $D_8 = \max(4,3) = 4$. Your program also confirmed $D_8 \leq D_e \leq D_4$ at every pixel of a 7×7 window.

The catch that examiners love

D_4 and D_8 between p and q are **independent of any path** that might exist between them — they involve only the coordinates. But D_m , the distance measured along the shortest m -path, **does** depend on the values of the pixels along the path and of their neighbours.

The handout's example, fully worked

Consider this arrangement, where p and p_4 have value 1, and p_1, p_2, p_3 can be 0 or 1. Adjacency is for $V = \{1\}$.

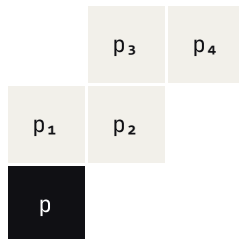


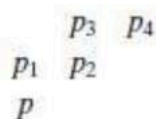
Figure 9.2 — p and p_4 are 1; the values of p_1 , p_2 and p_3 decide the answer.

CASE	SHORTEST M-PATH FROM P TO P_4	D_M
$p_1 = 0$ and $p_3 = 0$	$p \rightarrow p_2 \rightarrow p_4$	2
$p_1 = 1$, $p_3 = 0$	p_2 is no longer m -adjacent to p , so go $p \rightarrow p_1 \rightarrow p_2 \rightarrow p_4$	3
$p_3 = 1$, $p_1 = 0$	the same by symmetry: $p \rightarrow p_2 \rightarrow p_3 \rightarrow p_4$	3
$p_1 = 1$ and $p_3 = 1$	both corner shortcuts are blocked: $p \rightarrow p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow p_4$	4

Four different answers for the same two pixels, purely because the pixels in between changed. That never happens with D_4 or D_8 .

FROM YOUR LECTURE SLIDES

Note that the D_4 and D_8 distances between p and q are independent of any paths that might exist between the points because these distances involve only the coordinates of the points. If we elect to consider m -adjacency, however, the D_m distance between two points is defined as the shortest m -path between the points. In this case, the distance between two pixels will depend on the values of the pixels along the path, as well as the values of their neighbors. For instance, consider the following arrangement of pixels and assume that p , p_2 , and p_4 have value 1 and that p_1 and p_3 can have a value of 0 or 1:



Suppose that we consider adjacency of pixels valued 1 (i.e., $V = \{1\}$). If p_1 and p_3 are 0, the length of the shortest m -path (the D_m distance) between p and p_4 is 2. If p_1 is 1, then p_2 and p will no longer be m -adjacent (see the definition of m -adjacency) and the length of the shortest m -path becomes 3 (the path goes through the points $pp_1p_2p_4$). Similar comments apply if p_3 is 1 (and p_1 is 0); in this case, the length of the shortest m -path also is 3. Finally, if both p_1 and p_3 are 1, the length of the shortest m -path between p and p_4 is 4. In this case, the path goes through the sequence of points $pp_1p_2p_3p_4$.



The full note from your slides, working through the same four cases.

FOR THE EXAM

Two traps in one topic. First: D_4 and D_8 are formulas on coordinates, D_m is a path length in the picture. Second: the formula value and the real path length are not the same thing — in a binary image the path must stay on pixels in V , so the true path can be longer than D_4 says, or may not exist at all. Assignment 4 showed a case where $D_4 = 8$ by formula while no 4-path existed at all.

Check yourself: $p = (0,0)$, $q = (3,4)$. Find D_e , D_4 and D_8 .

Differences are 3 and 4. $D_e = \sqrt{(9+16)} = \sqrt{25} = 5$. $D_4 = 3 + 4 = 7$. $D_8 = \max(3,4) = 4$. Check the order: $4 \leq 5 \leq 7$. ✓

Check yourself: when are all three distances equal?

When the two pixels lie in the same row or the same column, so one of the differences is 0. Example $(0,0)$ and $(0,5)$: $D_e = D_4 = D_8 = 5$.

CHAPTER 10 HANDOUT 2, LECTURE 7 UNIT 1

Basic operations on images

Array operations are not matrix operations

Both are written with two 2×2 arrays in your handout, and the difference matters in every program you write.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

OPERATION	SYMBOL	RESULT	MEANING
Array product	$A .* B$	$a_{11}b_{11} \quad a_{12}b_{12}$ $a_{21}b_{21} \quad a_{22}b_{22}$	every pixel multiplied by the pixel in the same place
Matrix product	$A * B$	$a_{11}b_{11} + a_{12}b_{21}$ $a_{11}b_{12} + a_{12}b_{22}$ $a_{21}b_{11} + a_{22}b_{21}$ $a_{21}b_{12} + a_{22}b_{22}$	rows times columns, the usual linear-algebra product

In image processing we almost always want the **array** product, because we want to work pixel by pixel. In MATLAB that is why the dot matters: $A .* B$ and $A * B$ give completely

different pictures.

FROM YOUR LECTURE SLIDES

► **Array vs. Matrix Operation**

Array product operator

$$A \circ * B = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix}$$

Array product

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

$$B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

Matrix product operator

$$A * B = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

Matrix product

The two products written out side by side in your slides.

Linear and nonlinear operators

Let H be an operator that takes an image and returns an image. H is **linear** if, for any two images and any two constants,

$$H[a_i f_i(x, y) + a_j f_j(x, y)] = a_i H[f_i(x, y)] + a_j H[f_j(x, y)] = a_i g_i(x, y) + a_j g_j(x, y)$$

That single equation contains two separate properties:

- **Additivity** — the operator applied to a sum equals the sum of the operator applied to each part.
- **Homogeneity** — multiplying the input by a constant multiplies the output by the same constant.

If an operator fails either one, it is **nonlinear**. Summing images is linear. Taking the maximum of a neighbourhood is not — the maximum of a sum is not the sum of the maximums.

FROM YOUR LECTURE SLIDES

► **Linear vs. Nonlinear Operation**

$$H[f(x, y)] = g(x, y)$$

$$H[a_i f_i(x, y) + a_j f_j(x, y)]$$

$$= H[a_i f_i(x, y)] + H[a_j f_j(x, y)]$$

Additivity

$$= a_i H[f_i(x, y)] + a_j H[f_j(x, y)]$$

Homogeneity

$$= a_i g_i(x, y) + a_j g_j(x, y)$$

H is said to be a **linear operator**;

H is said to be a **nonlinear operator** if it does not meet the above qualification.

40

The definition as it appears in your slides, with the two halves labelled: additivity and homogeneity.

FOR THE EXAM

To *prove* an operator is nonlinear you only need **one** counter-example with actual numbers. Take $H = \max$. With $f_1 = [0 \ 2]$ and $f_2 = [3 \ 0]$: $\max(f_1 + f_2) = \max([3 \ 2]) = 3$, but $\max(f_1) + \max(f_2) = 2 + 3 = 5$. $3 \neq 5$, so $\max$ is nonlinear. Done.

The four arithmetic operations

Arithmetic between two images is always an *array* operation, done pixel by pixel:

$$\begin{aligned} s(x, y) &= f(x, y) + g(x, y) & d(x, y) &= f(x, y) - g(x, y) & p(x, y) &= f(x, y) \times g(x, y) \\ v(x, y) &= f(x, y) \div g(x, y) \end{aligned}$$

Where each one is actually used**Addition**

Averaging to kill noise. Add many noisy pictures of the same still scene and divide by their number. The signal is the same in every frame and adds up; the random noise partly cancels. Astronomers do this every night.

Subtraction

Mask mode radiography, the technique behind digital subtraction angiography (DSA). Take $h(x, y)$, an X-ray of a region of the patient's body — this is the **mask**. Then inject a contrast medium and take live images $f(x, y)$ at TV rates. The enhanced detail is simply

$$g(x, y) = f(x, y) - h(x, y)$$

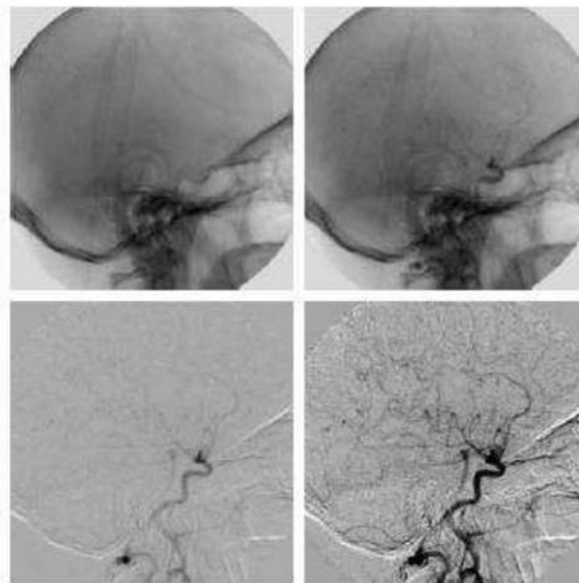
Everything that did not change cancels to zero, so only the contrast medium moving through the arteries remains. Your handout's

pictures show a 50 % and an 80 % blockage found exactly this way.

Multiplication **Shading correction.** An electron microscope image is often brighter in the middle than at the edges because the illumination is uneven. If you can capture that shading pattern on its own, multiplying the image by the *reciprocal* of the pattern flattens it out. Multiplication is also how a mask is applied — multiply by 1 inside the region of interest and by 0 outside it.

Division The same shading correction seen the other way round: divide the shaded image by the shading pattern.

FROM YOUR LECTURE SLIDES



a b
c d

FIGURE 2.28

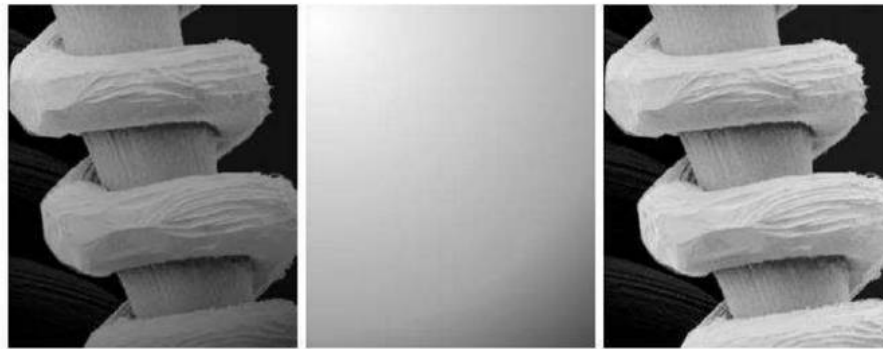
Digital subtraction angiography. (a) Mask image. (b) A live image. (c) Difference between (a) and (b). (d) Enhanced difference image. (Figures (a) and (b) courtesy of The Image Sciences Institute, University Medical Center, Utrecht, The Netherlands.)

43

Subtraction. (a) the mask, (b) a live image, (c) the difference, (d) the enhanced result. Only the contrast medium in the arteries survives the subtraction.

FROM YOUR LECTURE SLIDES

An Example of Image Multiplication



a b c

FIGURE 2.29 Shading correction. (a) Shaded SEM image of a tungsten filament and support, magnified approximately 130 times. (b) The shading pattern. (c) Product of (a) by the reciprocal of (b). (Original image courtesy of Michael Shaffer, Department of Geological Sciences, University of Oregon, Eugene.)

A scanning electron microscope (SEM) is a type of microscope which uses a focused beam of electrons to scan a surface of a sample to create a high resolution image. SEM produces **images that can show information on a material's surface composition and topography.**

SEM produces magnified detailed images of an object by scanning a focused beam of electrons.

44

Multiplication. (a) a shaded electron-microscope image, (b) the shading pattern, (c) the product of (a) with the reciprocal of (b), which is flat and even.

Set and logical operations

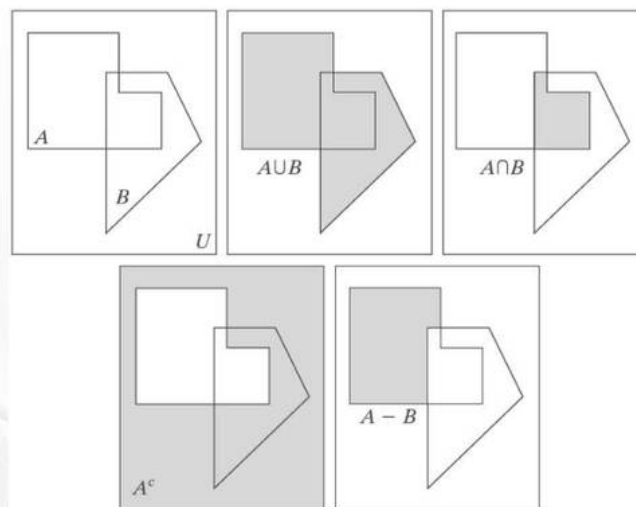
Because a binary image is really just a set of pixel coordinates, set theory applies directly. For two sets A and B in the same image:

SET OPERATION	SYMBOL	LOGIC NAME	KEEPS THE PIXELS THAT ARE...
Union	$A \cup B$	OR	in A, or in B, or in both
Intersection	$A \cap B$	AND	in both A and B
Complement	A^c	NOT	everything in the image that is not in A
Difference	$A - B$	AND-NOT	in A but not in B; formally $A - B = \{w \mid w \in A, w \notin B\} = A \cap B^c$

These four are the entire foundation of Unit 5, morphological image processing, which is why they are introduced here.

FROM YOUR LECTURE SLIDES

Set and Logical Operations



a	b	c
d	e	

FIGURE 2.31

(a) Two sets of coordinates, A and B , in 2-D space. (b) The union of A and B . (c) The intersection of A and B . (d) The complement of A . (e) The difference between A and B . In (b)–(e) the shaded areas represent the member of the set operation indicated.

The difference of two sets A and B , denoted $A - B$, is defined as

$$A - B = \{w | w \in A, w \notin B\} = A \cap B^c$$

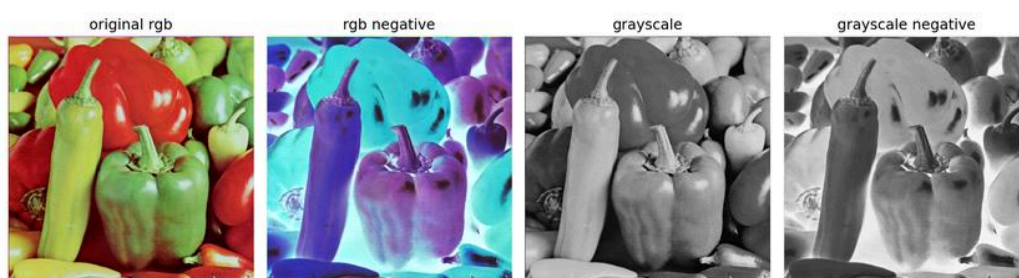
45

The four operations drawn as sets: the union, the intersection, the complement of A , and the difference $A - B$.

YOU HAVE ALREADY DONE THIS

The complement is the negative you wrote in Assignment 5, parts 11 and 12: $s = 255 - r$. On a binary image, where values are only 0 and 1, that same operation is exactly the set complement A^c .

FROM YOUR ASSIGNMENT



Your own result. In the colour negative every colour is replaced by its opposite, so the red peppers turn cyan; in the gray negative the dark and bright areas simply swap places.

Three kinds of operation, by how much of the image they look at

Single-pixel The new value of a pixel depends only on its own old value.
Negatives, log and power-law transforms, thresholding.

Neighbourhood	The new value depends on a small window of pixels around it. Smoothing, sharpening, median filtering — and this is convolution, which is Part III.
Geometric	The pixel <i>values</i> stay the same but their <i>positions</i> change: scaling, rotation, translation. These need interpolation, because the new positions rarely land exactly on whole-number coordinates.

CHAPTER 11 HANDOUT 3, LECTURES 7-9 GROUNDWORK FOR UNIT 2

2D signals and impulses

Part III changes the language slightly. Instead of writing an image as $f(x, y)$ we write a 2-D sequence as $\mathbf{x}(n_1, n_2)$, where n_1 and n_2 are whole numbers. It is the same object — a table of numbers — but now we treat it as a signal, so that the tools of signals and systems apply to it.

The 2D unit impulse

$$\delta(n_1, n_2) = 1 \text{ if } n_1 = 0 \text{ and } n_2 = 0 \quad \delta(n_1, n_2) = 0 \text{ otherwise}$$

A single 1 at the origin and 0 everywhere else. It is the 2-D version of the "one sharp spike" you already know from 1-D signals, and it matters because *any* sequence can be written as a sum of shifted, scaled impulses. That fact is what makes convolution work.

Four impulses worth knowing

NAME	WHERE THE 1S SIT	WRITTEN AS
2D impulse	a single point, the origin	$\delta(n_1, n_2)$
Vertical line impulse	a whole line along the n_2 axis — value 1 for every n_2 when $n_1 = 0$	$\delta(n_1)$
Horizontal line impulse	a whole line along the n_1 axis — value 1 for every n_1 when $n_2 = 0$	$\delta(n_2)$
Diagonal impulse	1s along the diagonal, where $n_1 = n_2$	$\delta(n_1 - n_2)$

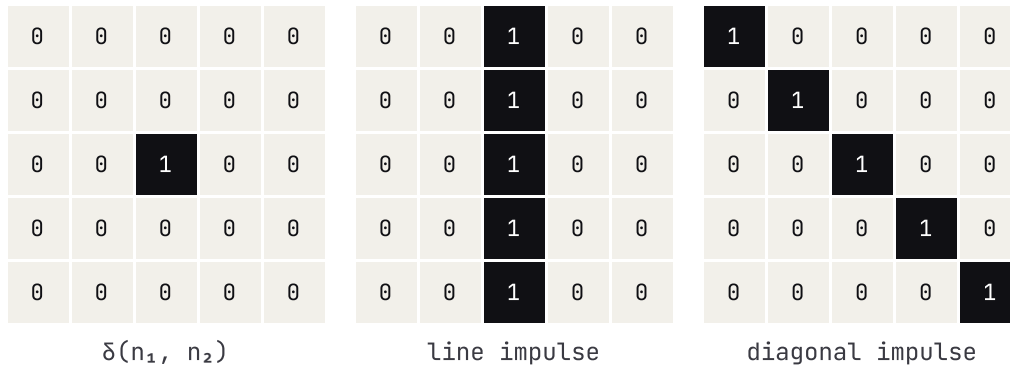


Figure 11.1 — Three of the four impulse sequences.

FROM YOUR LECTURE SLIDES

The 2D impulse sequence is given by $x(n_1, n_2) = \delta(n_1, n_2) = \begin{cases} 1, & n_1 = n_2 = 0 \\ 0, & \text{else} \end{cases}$. The 2D impulse sequence is depicted in Fig. 2.1.

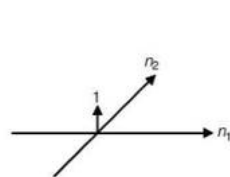


Fig. 2.1 2D impulse sequence

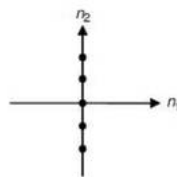


Fig. 2.2 Vertical line impulse

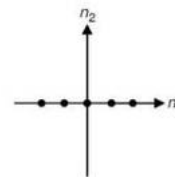


Fig. 2.3 Horizontal line impulse

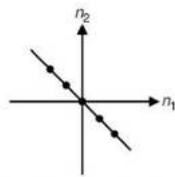


Fig. 2.4 Diagonal impulse function

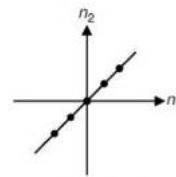


Fig. 2.5 Diagonal function representation

The same sequences as drawn in your slides: the 2D impulse, the vertical and horizontal line impulses, and the diagonal impulse function.

CHAPTER 12 HANDOUT 3, LECTURES 7-9 GROUNDWORK FOR UNIT 2

2D convolution

Convolution is how every filter in this course works. In one sentence: **slide a small kernel over the image, and at each position multiply the overlapping numbers and add them up.**

$$y(n_1, n_2) = \sum_{k_1} \sum_{k_2} x(k_1, k_2) \cdot h(n_1 - k_1, n_2 - k_2)$$

written $y = x ** h$, where $**$ means 2D convolution

The size of the answer — learn this first

If x is $M_1 \times M_2$ and h is $N_1 \times N_2$, then the result is

$$(M_1 + N_1 - 1) \times (M_2 + N_2 - 1)$$

You should compute this *before* starting any calculation, because it tells you how many numbers you are looking for, and half the marks in a numerical question come from getting the size right.

FOR THE EXAM

Add the sizes and subtract one, in each direction separately. 2×3 convolved with 3×1 gives $(2+3-1) \times (3+1-1) = 4 \times 3$. Never multiply the sizes.

Method 1 — the graphical (tabular) method

This is the method your handout works through first, and the fastest one by hand. Here is the complete example from the lecture.

$$x(m, n) = \begin{vmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{vmatrix} \quad h(m, n) = \begin{vmatrix} 1 \\ 1 \\ 1 \end{vmatrix}$$

Step 1 — **the size**. x is 2×3 and h is 3×1 , so y is $(2+3-1) \times (3+1-1) = 4 \times 3$.

FROM YOUR LECTURE SLIDES

EXAMPLE:
PERFORM THE LINEAR CONVOLUTION BETWEEN THESE TWO MATRICES USING GRAPHICAL METHOD

$$x(m, n) = \begin{pmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad h(m, n) = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Dimension of the resultant Matrix:



The problem exactly as your teacher set it.

Step 2 — **write down the indices.** Every value gets its (m, n) address:

$$x: (0,0)=4 \ (0,1)=5 \ (0,2)=6 \quad h: (0,0)=1 \ (1,0)=7 \ (1,1)=8 \ (1,2)=9 \\ (1,0)=1 \ (2,0)=1$$

Step 3 — **every product lands at the sum of the two addresses.** A value of x at (a, b) multiplied by a value of h at (c, d) contributes to y at **(a+c, b+d)**. That single rule is the whole method.

Step 4 — **collect the contributions.** For example the position (1, 0) receives $x(0,0) \cdot h(1,0) = 4$ and $x(1,0) \cdot h(0,0) = 7$, giving $4 + 7 = 11$.

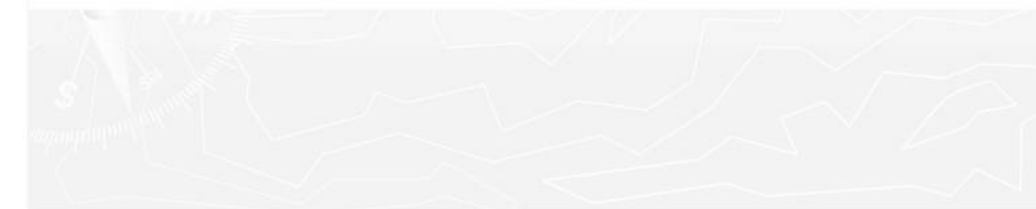
$$y(m, n) = \begin{vmatrix} 4 & 5 & 6 \\ 11 & 13 & 15 \\ 11 & 13 & 15 \\ 7 & 8 & 9 \end{vmatrix} \quad \boxed{4 \times 3}$$

Read the answer and it makes sense: h is a column of three 1s, so this convolution adds up every group of three vertically neighbouring pixels. The first and last rows of the answer are the original rows on their own (the kernel hanging half outside), and the middle rows are sums such as $4 + 7 = 11$ and $5 + 8 = 13$.

FROM YOUR LECTURE SLIDES

Substituting the above values in the corresponding positions in the matrix and graphic form, the resultant matrix is obtained as

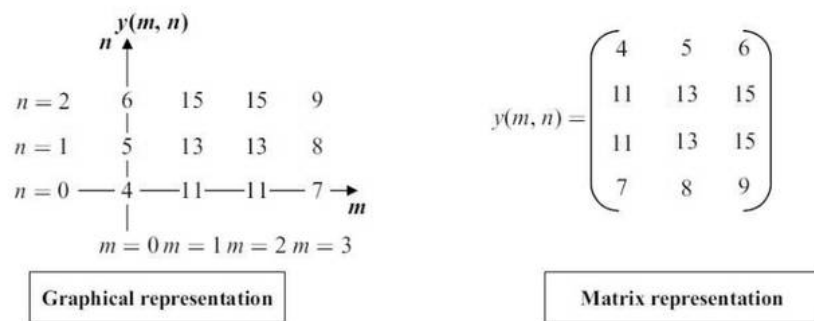
$$y(m, n) = \begin{matrix} & \begin{matrix} n \\ \uparrow \end{matrix} & & & \\ \begin{matrix} n=2 \\ n=1 \\ n=0 \end{matrix} & \begin{matrix} X & X & X & X \\ X & X & X & X \\ X & X & X & X \end{matrix} & & & \\ & \begin{matrix} \downarrow \\ m=0 \end{matrix} & \begin{matrix} m=1 \end{matrix} & \begin{matrix} m=2 \end{matrix} & \begin{matrix} m=3 \end{matrix} & \begin{matrix} \rightarrow \\ m \end{matrix} \end{matrix} \quad y(m, n) = \begin{pmatrix} y(0,0) & y(0,1) & y(0,2) \\ y(1,0) & y(1,1) & y(1,2) \\ y(2,0) & y(2,1) & y(2,2) \\ y(3,0) & y(3,1) & y(3,2) \end{pmatrix}$$



Setting out the empty answer grid first, so every contribution has a box to land in.

FROM YOUR LECTURE SLIDES

The final result in graphical and matrix representations is given below.



The finished answer in both the graphical form and the matrix form. Note how the two are related: m runs across the graph but down the matrix.

IN SIMPLE WORDS

Multiply each number of x by each number of h . Put the product in the box whose address is the *sum of the two addresses*. When every product is placed, add up whatever landed in each box. That is 2D convolution.

The assignment from your handout, solved

Your teacher then set this one. x is 3×3 , h is 3×2 , so y is $(3+3-1) \times (3+2-1) = 5 \times 4$.

$$\begin{vmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{vmatrix} ** \begin{vmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{vmatrix} = \begin{vmatrix} 1 & 3 & 5 & 3 \\ 5 & 12 & 16 & 9 \\ 12 & 27 & 33 & 18 \\ 11 & 24 & 28 & 15 \\ 7 & 15 & 17 & 9 \end{vmatrix}$$

Spot-check two of them so you trust the method:

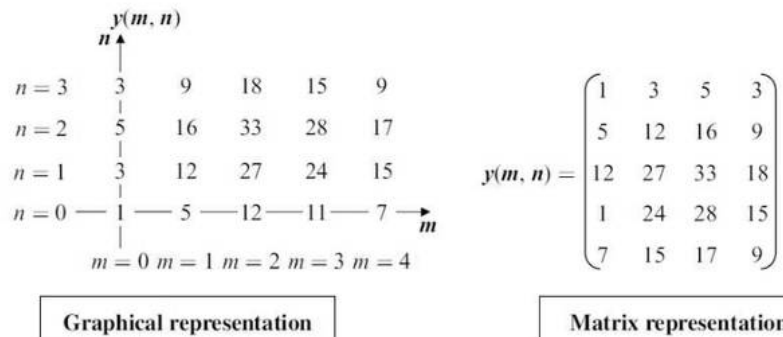
- $y(0,0) = x(0,0) \cdot h(0,0) = 1 \times 1 = 1$ — only one product can land in the corner.
- $y(1,1) = x(0,0)h(1,1) + x(0,1)h(1,0) + x(1,0)h(0,1) + x(1,1)h(0,0) = 1 + 2 + 4 + 5 = 12$. ✓

FROM YOUR LECTURE SLIDES

ASSIGNMENT:

$$x(m, n) = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad h(m, n) = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}$$

The final result in the graphical and matrix forms are given below:



The assignment and its answer, as given in your slides.

FOR THE EXAM

Two free checks on any convolution answer. **Corners:** $y(0,0)$ is always just $x(0,0) \cdot h(0,0)$, and the opposite corner is the product of the two last elements. **Total sum:** the sum of all numbers in y always equals $(\text{sum of } x) \times (\text{sum of } h)$. Here $45 \times 6 = 270$, and adding up the 5×4 answer also gives 270. If those two agree, your answer is almost certainly right.

CHAPTER 13 HANDOUT 3, LECTURES 7-9 GROUNDWORK FOR UNIT 2

The Z-transform method

The 2D Z-transform

$$X(z_1, z_2) = \sum_{n_1} \sum_{n_2} x(n_1, n_2) \cdot z_1^{-n_1} z_2^{-n_2}$$

Read it slowly, because it is simpler than it looks. Each sample of the sequence becomes one *term*. The **value** of the sample becomes the coefficient, and its **position** becomes the powers of z_1 and z_2 . A sample of value 5 sitting at position (1, 2) becomes the term $5 z_1^{-1} z_2^{-2}$.

IN SIMPLE WORDS

The Z-transform is a filing system. Instead of drawing a table, you write one long sum, and the powers of z_1 and z_2 act as the address labels that remember where each number came from. Since our sequences are finite, $X(z_1, z_2)$ is just an ordinary polynomial in $1/z_1$ and $1/z_2$.

Worked example

For $x = \begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ the transform is

$$X(z_1, z_2) = 4 + 5z_2^{-1} + 6z_2^{-2} + 7z_1^{-1} + 8z_1^{-1}z_2^{-1} + 9z_1^{-1}z_2^{-2}$$

and for $h = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ it is simply $H(z_1, z_2) = 1 + z_1^{-1} + z_1^{-2}$.

FROM YOUR LECTURE SLIDES

CONVOLUTION THROUGH Z-TRANSFORM

$$x(m, n) = \begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}; \quad h(m, n) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

The indices of the given input matrices are shown below.

$$x(m, n) = \begin{bmatrix} \begin{smallmatrix} (0,0) \\ 4 \end{smallmatrix} & \begin{smallmatrix} (0,1) \\ 5 \end{smallmatrix} & \begin{smallmatrix} (0,2) \\ 6 \end{smallmatrix} \\ \begin{smallmatrix} (1,0) \\ 7 \end{smallmatrix} & \begin{smallmatrix} (1,1) \\ 8 \end{smallmatrix} & \begin{smallmatrix} (1,2) \\ 9 \end{smallmatrix} \end{bmatrix} \quad h(m, n) = \begin{bmatrix} \begin{smallmatrix} (0,0) \\ 1 \end{smallmatrix} \\ \begin{smallmatrix} (1,0) \\ 1 \end{smallmatrix} \\ \begin{smallmatrix} (2,0) \\ 1 \end{smallmatrix} \end{bmatrix}$$

15

Writing the index of every sample above it is the safest way to start. Those indices become the powers of z_1 and z_2 .

Why we bother: convolution becomes multiplication

This is the key property, and the reason the Z-transform appears in an image processing course at all.

if $y(n_1, n_2) = x(n_1, n_2) ** h(n_1, n_2)$ then $Y(z_1, z_2) = X(z_1, z_2) \times H(z_1, z_2)$

Convolution in one domain is multiplication in the other domain.

The four-step method

1. **Take the Z-transform** of both sequences, writing each sample as coefficient $\times z_1^{-n_1} z_2^{-n_2}$.
2. **Multiply** the two polynomials.
3. **Expand** — write out every product term.
4. **Group** terms with the same powers of z_1 and z_2 . The coefficient of $z_1^{-n_1} z_2^{-n_2}$ is the sample $y(n_1, n_2)$. Place them in the matrix.

For the example above, step 3 gives eighteen terms:

$$\begin{aligned}
 &4(z_1^0 z_2^0) + 4(z_1^{-1} z_2^0) + 4(z_1^{-2} z_2^0) + 5(z_1^0 z_2^{-1}) + 5(z_1^{-1} z_2^{-1}) + \\
 &5(z_1^{-2} z_2^{-1}) + 6(z_1^0 z_2^{-2}) + 6(z_1^{-1} z_2^{-2}) + 6(z_1^{-2} z_2^{-2}) + 7(z_1^{-1} z_2^0) + \\
 &7(z_1^{-2} z_2^0) + 7(z_1^{-3} z_2^0) + 8(z_1^{-1} z_2^{-1}) + 8(z_1^{-2} z_2^{-1}) + 8(z_1^{-3} z_2^{-1}) + \\
 &9(z_1^{-1} z_2^{-2}) + 9(z_1^{-2} z_2^{-2}) + 9(z_1^{-3} z_2^{-2})
 \end{aligned}$$

and step 4 collects them. For instance $z_1^{-1} z_2^0$ appears twice, with coefficients 4 and 7, so $y(1,0) = 11$ — the same 11 the graphical method gave. Grouping everything:

$$y(m, n) = \begin{vmatrix} 4 & 5 & 6 \\ 11 & 13 & 15 \\ 11 & 13 & 15 \\ 7 & 8 & 9 \end{vmatrix} \quad \text{identical to the graphical answer}$$

FROM YOUR LECTURE SLIDES

$$\begin{aligned}
&= 4(Z_1^{-0}, Z_2^{-0}) + 4(Z_1^{-1}, Z_2^{-0}) + 4(Z_1^{-2}, Z_2^{-0}) + 5(Z_1^{-0}, Z_2^{-1}) + 5(Z_1^{-1}, Z_2^{-1}) + 5(Z_1^{-2}, Z_2^{-1}) \\
&\quad + 6(Z_1^{-0}, Z_2^{-2}) + 6(Z_1^{-1}, Z_2^{-2}) + 6(Z_1^{-2}, Z_2^{-2}) + 7(Z_1^{-1}, Z_2^{-0}) + 7(Z_1^{-2}, Z_2^{-0}) + 7(Z_1^{-3}, Z_2^{-0}) \\
&\quad + 8(Z_1^{-1}, Z_2^{-1}) + 8(Z_1^{-2}, Z_2^{-1}) + 8(Z_1^{-3}, Z_2^{-1}) + 9(Z_1^{-1}, Z_2^{-2}) + 9(Z_1^{-2}, Z_2^{-2}) + 9(Z_1^{-3}, Z_2^{-2})
\end{aligned}$$

Step 4 Grouping of the terms

The result obtained in Step 3 is grouped together to get the final result as

$$y(m, n) = \begin{pmatrix} 4 & 5 & 6 \\ 11 & 13 & 15 \\ 11 & 13 & 15 \\ 7 & 8 & 9 \end{pmatrix}$$

16

The expansion and the grouping step from your slides, ending in the same 4×3 answer.

YOU HAVE ALREADY DONE THIS

Assignment 3 did both routes in code and compared them. Your

`conv_ztransform()` built the two polynomials in SymPy, multiplied and expanded them, then read the coefficient of $z_1^{-n_1} z_2^{-n_2}$ back out. It matched the Toeplitz answer with a largest difference of 0.0.

Frequency response

Put $z_1 = e^{j\omega_1}$ and $z_2 = e^{j\omega_2}$ in the Z-transform and you get the **frequency response** — how the system treats each spatial frequency.

Your handout's example is a sequence with four 1s, one step away from the origin in each of the four directions. Its transform is

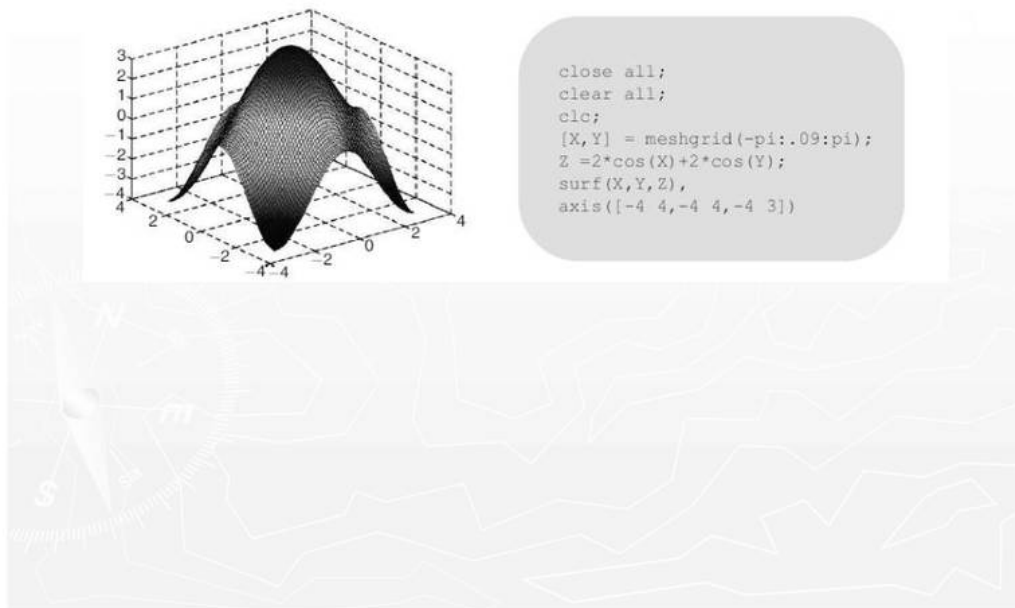
$$\begin{aligned}
X(z_1, z_2) &= z_1^{-1} + z_1^{+1} + z_2^{-1} + z_2^{+1} \text{ substituting } z = e^{j\omega}: X(\omega_1, \omega_2) = \\
&= e^{-j\omega_1} + e^{+j\omega_1} + e^{-j\omega_2} + e^{+j\omega_2} = 2\cos \omega_1 + 2\cos \omega_2 \\
&\text{using } e^{j\theta} + e^{-j\theta} = 2\cos \theta
\end{aligned}$$

That is exactly the surface plotted by the MATLAB lines in your slide:

```
[X,Y] = meshgrid(-pi:.09:pi); Z = 2*cos(X) + 2*cos(Y); surf(X,Y,Z),
axis([-4 4 -4 4 -4 3])
```

The surface has its peak of 4 at $\omega_1 = \omega_2 = 0$, which tells you this kernel passes low frequencies (smooth areas) strongly and cancels high frequencies — in other words, it is a **smoothing filter**. This is how you tell what a kernel does without applying it to a single image.

FROM YOUR LECTURE SLIDES



The frequency response $2\cos \omega_1 + 2\cos \omega_2$ plotted by the MATLAB lines above. The peak in the middle is the low-frequency response.

FOR THE EXAM

The pair $e^{j\theta} + e^{-j\theta} = 2\cos \theta$ is the trick that turns almost every one of these questions into a short answer. A symmetric pair of impulses at ± 1 always collapses to a cosine.

CHAPTER 14 HANDOUT 3, LECTURES 7-9 GROUNDWORK FOR UNIT 2

The Toeplitz matrix method

The third route to the same answer. Convolution is a *linear* operation, so it must be possible to write it as one matrix multiplication — and it is.

A **Toeplitz matrix** is one in which every diagonal carries the same value; in other words each column is the column before it, shifted down by one place. That shifting is exactly what sliding a kernel does, which is why this particular shape appears. Doing it in two dimensions needs a matrix of Toeplitz matrices, called a **block Toeplitz matrix**.

The procedure (your handout's flow chart)

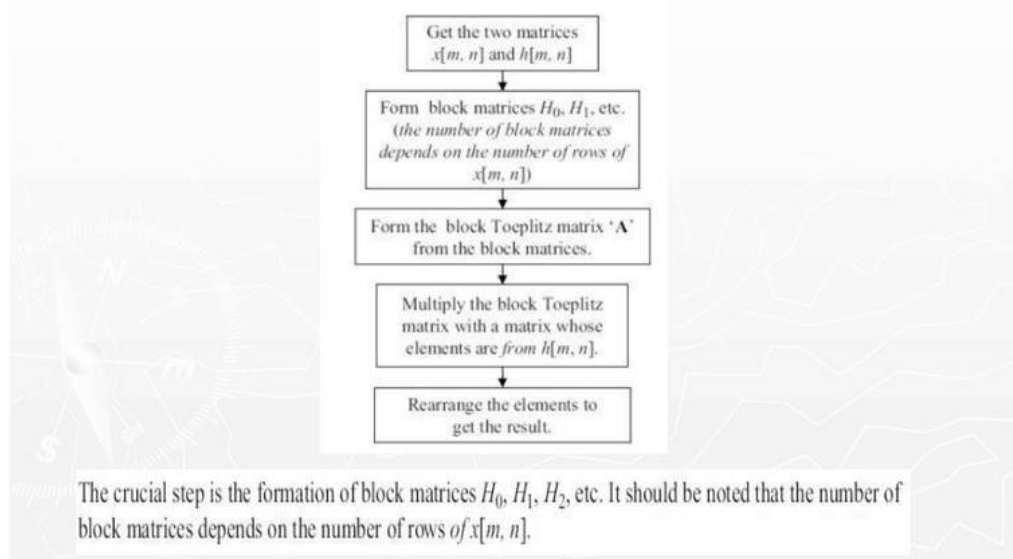
1. Get the two matrices $x[m, n]$ and $h[m, n]$.
2. Form the **block matrices** $H_0, H_1, H_2 \dots$ — one from each row of x . *The number of block matrices depends on the number of rows of $x[m, n]$.*
3. Form the **block Toeplitz matrix A** from those blocks.
4. Multiply A by the other matrix, $h[m, n]$, written in **column form**.
5. Rearrange the resulting column into a matrix to get the answer.

Your handout says it plainly: *the crucial step is the formation of the block matrices*. Get those right and the rest is arithmetic.

FROM YOUR LECTURE SLIDES

2D CONVOLUTION THROUGH MATRIX ANALYSIS

2D convolution can be performed through matrix multiplication. To do this, the block Toeplitz matrix has to be formed from one of the matrices. The other matrix is written in column form. Then the multiplication between the block Toeplitz matrix and the matrix written in column form will yield a result similar to 2D convolution. The flow chart to perform 2D convolution through matrix multiplication is given below.



The flow chart from your handout, in five steps.

The worked example, step by step

$$x[m, n] = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad h[m, n] = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Step 1 — the size. Both are 2×2 , so y is 3×3 , which means the final column will have 9 numbers.

Step 2 — one block per row of x . Each block is built by writing that row down the first column and shifting it one place to the right each time. Each block has 3 rows (the output height) and 2 columns (the height of h).

| |

$$\text{from row [1 2]} \rightarrow H_0 = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 2 \end{bmatrix} \quad \text{from row [3 4]} \rightarrow H_1 = \begin{bmatrix} 3 & 0 \\ 4 & 3 \\ 0 & 4 \end{bmatrix}$$

Step 3 — arrange the blocks in Toeplitz order too. The blocks now play the part that individual numbers played inside a block: first column H_0 , H_1 , then shifted down by one.

$$A = \begin{bmatrix} H_0 & 0 \\ H_1 & H_0 \\ 0 & H_1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 3 & 0 & 1 & 0 \\ 4 & 3 & 2 & 1 \\ 0 & 4 & 0 & 2 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 4 & 3 \\ 0 & 0 & 0 & 4 \end{bmatrix} \quad 9 \times 4$$

Check the shape before going on: 9 rows because the answer has $3 \times 3 = 9$ numbers, and 4 columns because h has $2 \times 2 = 4$ numbers.

Step 4 — multiply by h in column form. $h = [[5, 6], [7, 8]]$ read row by row becomes the column $[5, 6, 7, 8]^T$.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 3 & 0 & 1 & 0 \\ 4 & 3 & 2 & 1 \\ 0 & 4 & 0 & 2 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 4 & 3 \\ 0 & 0 & 0 & 4 \end{bmatrix} \times \begin{bmatrix} 5 \\ 6 \\ 7 \\ 8 \end{bmatrix} = \begin{bmatrix} 5 \\ 16 \\ 12 \\ 22 \\ 60 \\ 40 \\ 21 \\ 52 \\ 32 \end{bmatrix}$$

Row by row, that is $1 \times 5 = 5$; $2 \times 5 + 1 \times 6 = 16$; $2 \times 6 = 12$; $3 \times 5 + 1 \times 7 = 22$; $4 \times 5 + 3 \times 6 + 2 \times 7 + 1 \times 8 = 60$; and so on.

Step 5 — rearrange into a matrix, three numbers per row:

$$y[m, n] = \begin{bmatrix} 5 & 16 & 12 \\ 22 & 60 & 40 \\ 21 & 52 & 32 \end{bmatrix} \quad 3 \times 3$$

Verify it with the free checks from Chapter 12: the corner $y(0,0) = 1 \times 5 = 5 \checkmark$, the far corner $= 4 \times 8 = 32 \checkmark$, and the total $5+16+12+22+60+40+21+52+32 = 260$, while $(\text{sum of } x)(\text{sum of } h) = 10 \times 26 = 260 \checkmark$.

FROM YOUR LECTURE SLIDES

Then, the convolution result is obtained by multiplying the block Toeplitz matrix 'A' with the matrix $h[m, n]$. The matrix $h[m, n]$ is written in columnar form.

$$y[m, n] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 3 & 0 & 1 & 0 \\ 4 & 3 & 2 & 1 \\ 0 & 4 & 0 & 2 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 4 & 3 \\ 0 & 0 & 0 & 4 \end{bmatrix} \times \begin{bmatrix} 5 \\ 6 \\ 7 \\ 8 \end{bmatrix} = \begin{bmatrix} 5 \\ 16 \\ 12 \\ 22 \\ 60 \\ 40 \\ 21 \\ 52 \\ 32 \end{bmatrix}$$

The result is given by

$$y[m, n] = \begin{bmatrix} 5 & 16 & 12 \\ 22 & 60 & 40 \\ 21 & 52 & 32 \end{bmatrix} \text{ which is a } 3 \times 3 \text{ matrix.}$$

Weeks 1 & 2

19

The same multiplication as printed in your slides.

FOR THE EXAM

The single most common mistake is mixing up the order when flattening. In this handout's convention the blocks are built from x and it is h that is written as a column, read *row by row*. Reading it column by column gives a 3×3 answer of the right shape with completely wrong numbers, and nothing warns you. Because convolution is commutative you may swap the roles of x and h , but you must then swap them everywhere.

Why use this method at all?

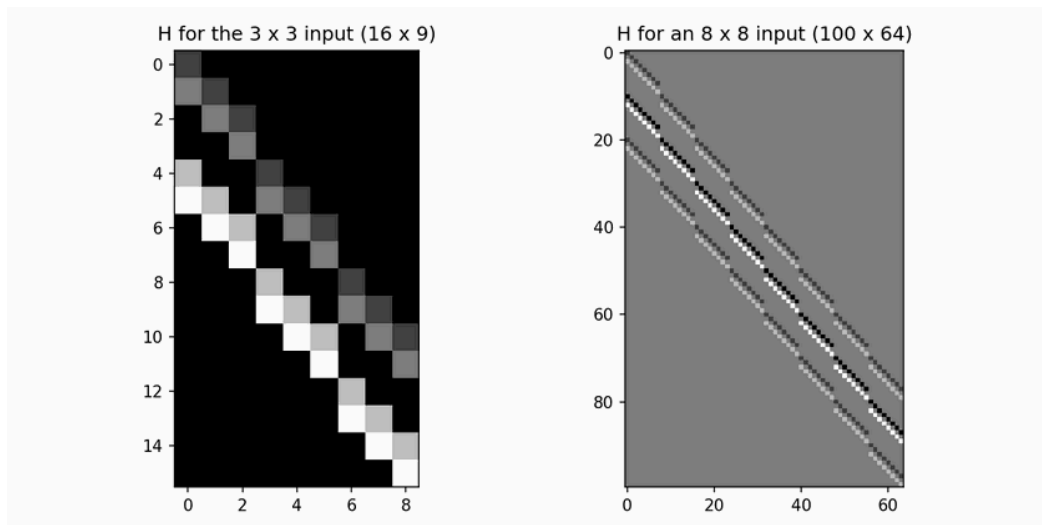
By hand it is more work than the graphical method. Its value is conceptual: it shows that convolution is just a matrix multiplication, $y = A \cdot h$. Once an operation is a matrix, everything from linear algebra applies to it — most importantly, *undoing* a blur becomes the problem of inverting or least-squares solving a matrix, which is exactly how image restoration (your Unit 3) is set up.

The cost is memory. The matrix has (output size) rows $\times$ (kernel size) columns, and it grows roughly as the fourth power of the image side.

YOU HAVE ALREADY DONE THIS

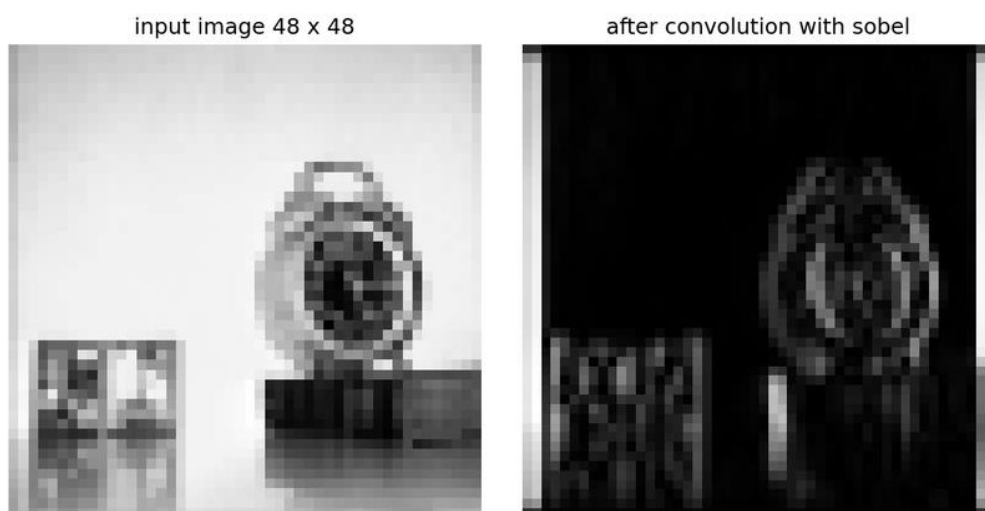
Assignment 3 measured it. For a 48×48 image with a 3×3 kernel the matrix was already $2500 \times 2304 = 5.76$ million numbers, and for an ordinary 512×512 image it would be $264196 \times 262144 \approx 554$ GB. That is why real programs slide the kernel or use the FFT instead, and never build this matrix.

FROM YOUR LECTURE SLIDES



Your Assignment 3. The block Toeplitz matrix drawn as a picture. The kernel values repeat along the diagonals inside each block, and the blocks repeat along the block diagonals. Almost all of it is zero.

FROM YOUR LECTURE SLIDES



Your Assignment 3. A real 48×48 image convolved with the Sobel kernel using this exact matrix. The bright strips at the sides are the boundary effect of full convolution.

The three methods side by side

METHOD	BEST FOR	WATCH OUT FOR
Graphical / tabular	doing it by hand in an exam — fastest and hardest to get wrong	keep the index bookkeeping tidy
Z-transform	proving things, and understanding filters through frequency response	many terms to expand; easy to drop one
Block Toeplitz	showing convolution is a linear operator; the starting point for restoration	flattening order, and enormous memory for real images

All three give the same numbers, always. If two of them disagree, one of them has an arithmetic slip — that is exactly why your assignment compared them.

CHAPTER 15 LECTURE 10 UNIT 2

What image enhancement means

Everything up to here was about *understanding* an image. From now on we start *changing* it.

THE DEFINITION TO LEARN

Enhancement is the process of manipulating an image so that the result is **more suitable than the original for a specific application**.

The word **specific** is the important one, and examiners like it. It says at the very start that enhancement is **problem-oriented**: a method that works beautifully on X-ray images may be the wrong choice for infrared satellite images. There is **no general theory of image enhancement**. When an image is processed for a human to look at, the viewer is the final judge of how well a method worked.

That leads to a neat pair of statements about what "a good image" even means:

For human vision	Judging image quality is a highly subjective process. It is hard even to standardise the definition of a good image, so a certain amount of trial and error is always needed before a method is chosen.
For machine vision	The job is easier, because a good image is simply the one that gives the best recognition results . You can measure that.

The two domains

DOMAIN	WHAT IS MANIPULATED	EXAMPLES
Spatial domain (the image plane)	the pixels themselves, directly	negatives, log and power-law transforms, histogram equalization, smoothing and sharpening masks
Frequency domain	the Fourier transform of the image	low-pass smoothing, high-pass sharpening, band-reject, homomorphic filtering

Some techniques mix the two. Your course has covered the spatial domain so far; the frequency domain comes later in Unit 2.

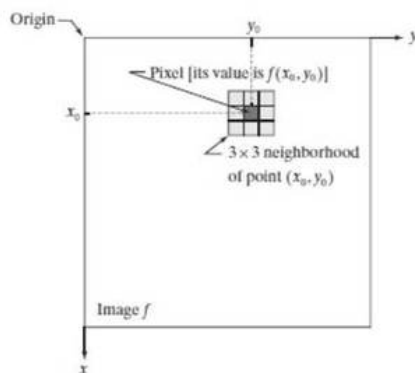
The master equation of the spatial domain

$$g(x, y) = T[f(x, y)]$$

f = input image · g = processed image · T = an operator on f , defined over some neighbourhood of (x, y)

FROM YOUR LECTURE SLIDES

Spatial Domain



- Procedures that operate directly on pixels.

$$g(x,y) = T[f(x,y)]$$

where

- $f(x,y)$ is the input image
- $g(x,y)$ is the processed image
- T is an operator on f defined over some neighborhood of (x,y)

The spatial-domain definition, with the neighbourhood drawn around the point (x, y) .

The **size of that neighbourhood** splits the whole of image enhancement into two families, and this single idea organises the next several chapters.

NEIGHBOURHOOD	NAME	WHAT THE NEW PIXEL DEPENDS ON	CHAPTERS
1×1	point processing	only the old value of that same pixel	16 to 22
bigger than 1×1	mask processing, also called filtering	a small window of neighbours around it	not taught yet

Point processing

When the neighbourhood is a single pixel, T becomes a plain **gray level transformation function**, and the whole operation shrinks to one line:

$$s = T(r)$$

r = the gray level of $f(x, y)$, the input · s = the gray level of $g(x, y)$, the output

FROM YOUR LECTURE SLIDES



Point Processing

- Neighborhood = 1×1 pixel
- g depends on only the value of f at (x, y)
- T = gray level (or intensity or mapping) transformation function, operates on one pixel
 $s = T(r)$
- Where,
 - r = gray level of $f(x, y)$
 - s = gray level of $g(x, y)$

Point processing: a 1×1 neighbourhood, so the output depends only on the value of f at that same point.

IN SIMPLE WORDS

Point processing is a **lookup table**. Feed in an old gray level, read out a new one. The same input always gives the same output no matter where in the picture the pixel sits, so you can apply it to all 262,144 pixels without ever looking at a neighbour.

The three kinds of point operation

Your handout lists exactly three, and the rest of Unit 2 fills them in:

1. **Brightness modification** — add or subtract a constant from every pixel.

$$g(x, y) = f(x, y) + c \text{ brighter} \quad g(x, y) = f(x, y) - c \text{ darker}$$

2. **Contrast manipulation** — scale every pixel by a constant.

$$g(x, y) = f(x, y) \times c$$

3. **Histogram manipulation** — reshape the whole distribution of gray levels. Chapters 19 to 22.

FOR THE EXAM

Brightness and contrast are easy to mix up. **Adding** a constant slides the whole histogram sideways and changes brightness. **Multiplying** stretches or squeezes the histogram about zero and changes contrast. Adding never changes the *difference* between two pixels; multiplying always does.

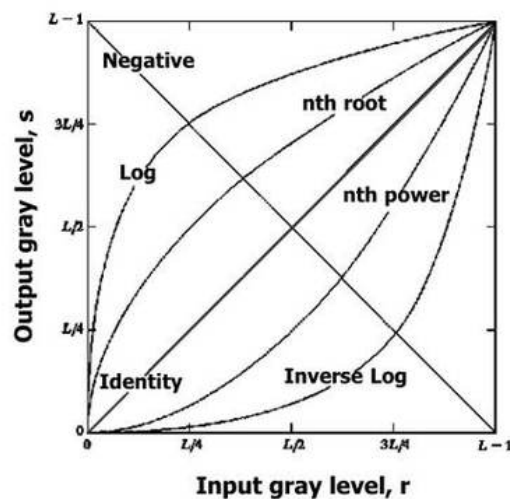
CHAPTER 16 LECTURE 11 UNIT 2

The three basic intensity transformations

Three shapes of $T(r)$ cover almost every point operation in the syllabus: **linear** (identity and negative), **logarithmic** (log and inverse log) and **power-law** (n-th power and n-th root). One picture holds all of them.

FROM YOUR LECTURE SLIDES

Identity function



- Output intensities are identical to input intensities.
- Is included in the graph only for completeness.

11

Every basic transformation on one pair of axes: input gray level r across, output gray level s up. The straight diagonal is the identity, which changes nothing and is drawn only for comparison.

HOW TO READ THIS GRAPH

Any curve that bulges **above** the diagonal makes the picture **brighter** (it maps a given input to a higher output). Any curve that sags **below** it makes the picture **darker**. The steeper the curve is over a range of inputs, the more contrast that range gets.

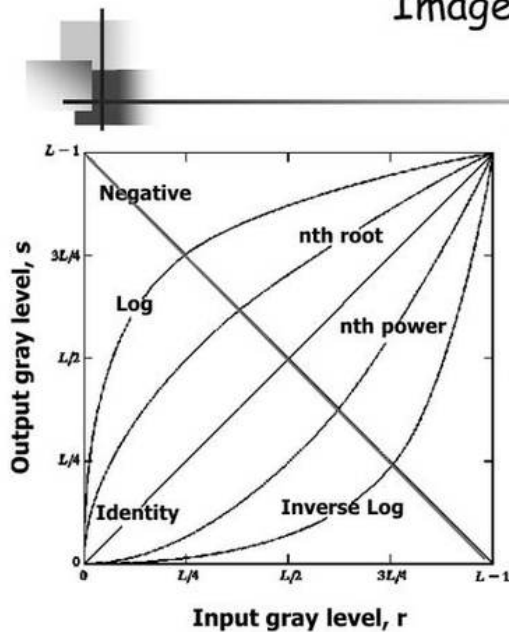
Image negative

$$s = L - 1 - r \text{ for an 8-bit image, } s = 255 - r$$

The intensity levels are simply reversed. It is worth knowing exactly when this is useful, because that is the usual question: the negative is **suited to enhancing white or gray detail embedded in dark regions of an image, especially when the black areas are dominant in size**. The classic case is a medical X-ray, where the interesting tissue is bright but drowned in a large black surround.

FROM YOUR LECTURE SLIDES

Linear Grey Level Transformation Image Negatives



- An image with gray level in the range $[0, L-1]$ where $L = 2^n$; $n = 1, 2, \dots$
- Negative transformation :
 $s = L - 1 - r \Rightarrow s = 255 - r$
- Reversing the intensity levels of an image.
- Suitable for enhancing white or gray detail embedded in dark regions of an image, especially when the black area dominant in size.

12

The negative is the straight line falling from left to right, the mirror of the identity.

FROM YOUR LECTURE SLIDES

Example of Negative Image



Input image (X-ray image)



Output image (negative)

An X-ray and its negative. The small bright detail is far easier to study in the reversed picture.

Log transformation

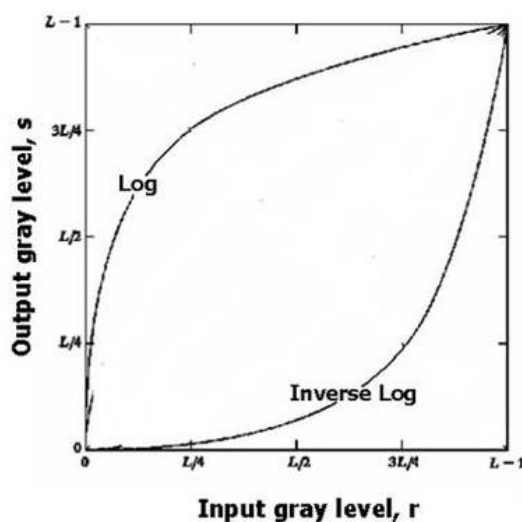
$$s = c \log(1 + r) \quad c \text{ is a constant, and } r \geq 0$$

The 1 inside the bracket is there so that $r = 0$ gives $s = 0$ rather than minus infinity. The shape of the log curve maps a **narrow range of low gray levels** in the input into a **much wider range** of output levels, and does the opposite at the top end. In one sentence: it expands dark pixels and compresses bright ones.

Its real job is **compressing dynamic range**. The standard example in your slides is the Fourier spectrum of an image, whose values can run from 0 to 10^6 or more. On a display that only has 256 levels, everything except the brightest spike appears black. Applying the log with $c = 1$ squeezes a range of 0 to 1.5×10^6 down to 0 to 6.2, and the detail appears.

FROM YOUR LECTURE SLIDES

Non-Linear Grey Level Transformation Log Transformations



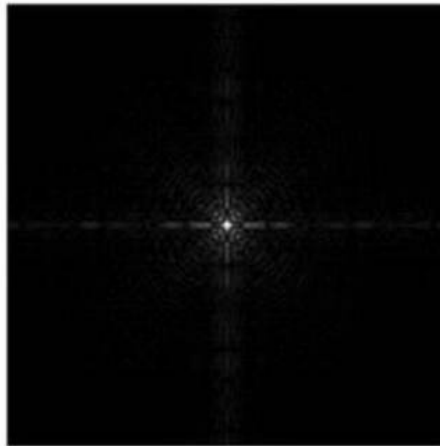
$$s = c \log (1+r)$$

- c is a constant and $r \geq 0$
- Log curve maps a narrow range of low gray-level values in the input image into a wider range of output levels.
- Used to expand the values of dark pixels in an image while compressing the higher-level values.

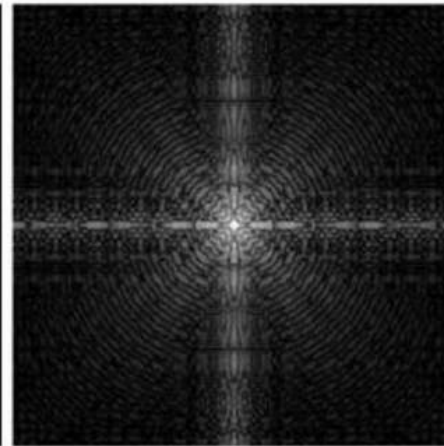
15

The log and inverse-log curves. The log sits above the diagonal, so it brightens.

FROM YOUR LECTURE SLIDES



**Fourier Spectrum with
range = 0 to 1.5×10^6**



**Result after apply the log
transformation with $c = 1$,
range = 0 to 6.2**

17

A Fourier spectrum before and after the log transformation. Same data, but now you can see it.

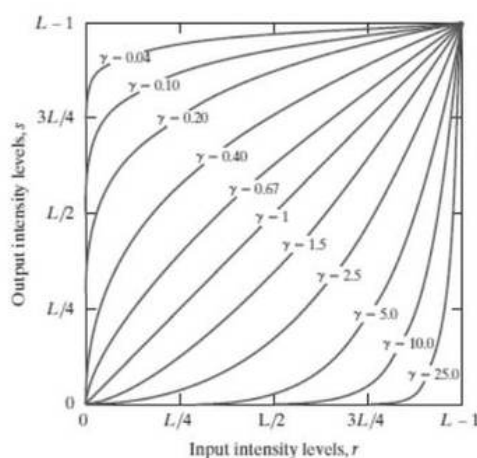
The **inverse log** does the opposite: it expands the high values and compresses the dark ones.

Power-law transformation and gamma

$$s = c \cdot r^\gamma \quad c \text{ and } \gamma \text{ are positive constants}$$

This is the most useful family of the three, because one parameter gives a whole set of curves.

FROM YOUR LECTURE SLIDES



Input gray level, r

**Plots of $s = cr^\gamma$ for various values of γ
($c = 1$ in all cases)**

$$s = cr^\gamma$$

- c and γ are positive constants
- Used in devices where power-law characteristics to be followed
- For example
 - display devices
 - printer devices
 - CRT devices

20

$s = c r^\gamma$ for several values of γ , with $c = 1$ throughout. Fractional γ bulges above the diagonal, γ greater than 1 sags below it.

VALUE OF Γ	CURVE	EFFECT	USE IT WHEN THE IMAGE IS...
$\gamma < 1$	above the diagonal	expands dark values, image becomes brighter	too dark
$\gamma = 1, c = 1$	the diagonal itself	identity, no change	—
$\gamma > 1$	below the diagonal	compresses dark values, image becomes darker with more contrast at the top	washed out

Gamma correction, the application that matters

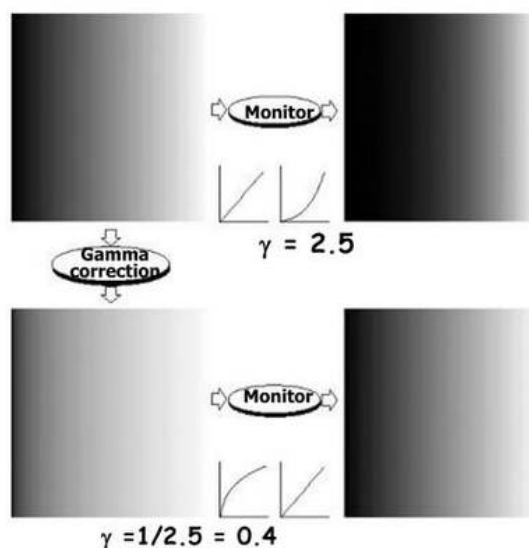
A physical display is not linear. The intensity produced at the surface of a CRT is roughly the applied voltage raised to a power of about 2.5, and that exponent is what the word **gamma** originally meant. Real devices have γ between **1.8 and 2.5**. Left alone, the picture comes out too dark.

The cure is to pre-distort the image by the *inverse* power before sending it to the monitor:

$$s = c \cdot r^{1/\gamma} \text{ for a monitor with } \gamma = 2.5, \text{ correct with } 1/2.5 = 0.4$$

The two non-linearities then cancel and what you see matches what was captured. The same idea is used in cameras, printers, medical imaging and computer graphics.

FROM YOUR LECTURE SLIDES



- Cathode ray tube (CRT) devices have an intensity-to-voltage response that is a power function, with γ varying from 1.8 to 2.5
- The picture will become darker.
- Gamma correction is done by preprocessing the image before inputting it to the monitor with $s = cr^{1/\gamma}$

26

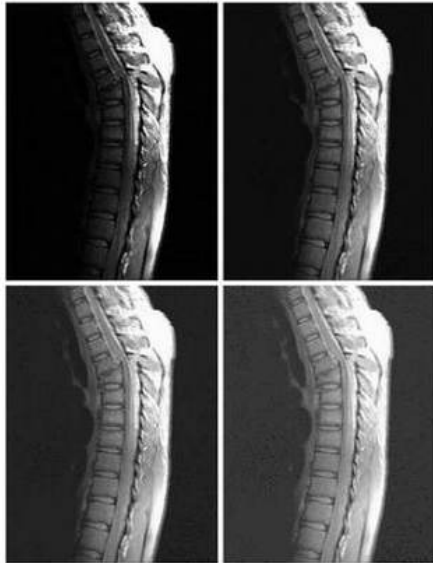
Top: the ramp sent straight to a $\gamma = 2.5$ monitor comes out too dark. Bottom: pre-corrected with $\gamma = 1/2.5 = 0.4$, it comes out right.

Choosing γ in practice

FROM YOUR LECTURE SLIDES

Another example : MRI

a	b
c	d



(a) a magnetic resonance image of an upper thoracic human spine with a fracture dislocation and spinal cord impingement

- The picture is predominately dark
- An expansion of gray levels are desirable $\Rightarrow$ needs $\gamma < 1$

(b) result after power-law transformation with $\gamma = 0.6$, $c=1$

(c) transformation with $\gamma = 0.4$
(best result)

(d) transformation with $\gamma = 0.3$
(under acceptable level)

28

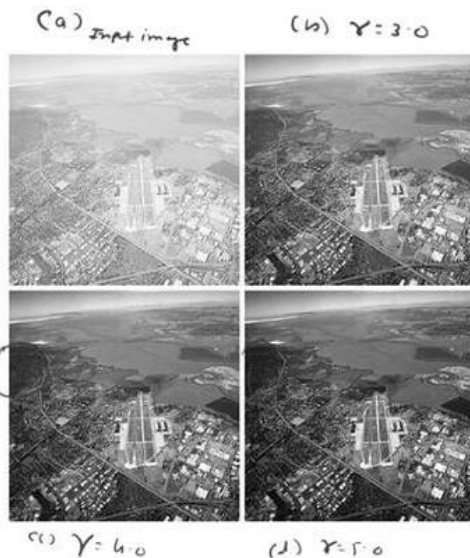
A dark MRI of a fractured spine. It needs the gray levels *expanded*, so $\gamma < 1$: (b) $\gamma = 0.6$, (c) $\gamma = 0.4$ is the best result, (d) $\gamma = 0.3$ goes too far and the background begins to look washed out.

FROM YOUR LECTURE SLIDES



Another example

a	b
c	d



- (a) image has a washed-out appearance, it needs a compression of gray levels $\Rightarrow$ needs $\gamma > 1$ ✓
- (b) result after power-law transformation with $\gamma = 3.0$ (suitable)
- (c) transformation with $\gamma = 4.0$ (suitable)
- (d) transformation with $\gamma = 5.0$ (high contrast, the image has areas that are too dark, some detail is lost)

The opposite case: a washed-out aerial image needs the gray levels *compressed*, so $\gamma > 1$. (b) 3.0 and (c) 4.0 are suitable, (d) 5.0 is too much and dark detail is lost.

FOR THE EXAM

Learn the direction by heart, because it is a one-mark question that is easy to get backwards: **dark image** $\rightarrow$ γ **less than 1** (expand the dark end), **washed-out image** $\rightarrow$ γ **greater than 1** (compress it). And remember that reducing γ too far does not keep improving things — contrast falls until the picture takes on a "wash-out" look, especially in the background.

YOU HAVE ALREADY DONE THIS

The negative in Assignment 5, parts 11 and 12, was exactly $s = 255 - r$, the simplest member of this family.

CHAPTER 17 LECTURE 12 UNIT 2

Piecewise linear transformations

The curves of Chapter 16 are fixed shapes controlled by one constant. A **piecewise linear** function is built from straight segments whose corners you place yourself, so it can take an arbitrary shape. The price is that it needs more input from the user.

Contrast stretching

Low-contrast images come from three causes, and all three are worth quoting:

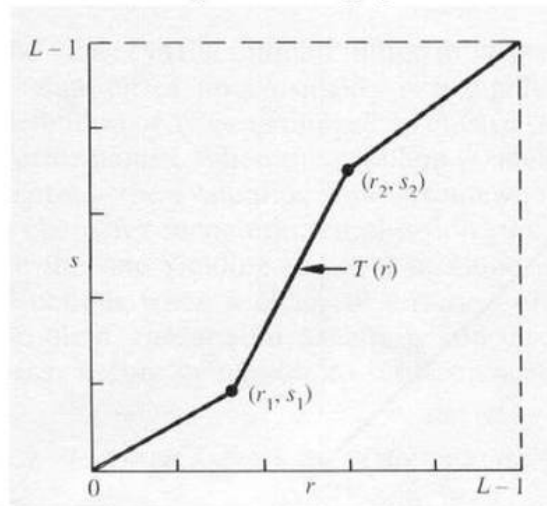
- poor illumination,
- lack of dynamic range in the imaging sensor,
- the wrong setting of a lens aperture during acquisition.

The goal is to **increase the dynamic range of the gray levels** so the image spans the full range the display can show. The transformation is drawn with two control points, (r_1, s_1) and (r_2, s_2) , and their positions decide the whole shape.

FROM YOUR LECTURE SLIDES

Contrast Stretching

- To increase the dynamic range of the gray levels in the image being processed.



The contrast-stretching function. The middle segment is steep, so the gray levels between r_1 and r_2 are spread out; the two end segments are shallow, so the extremes are squeezed.

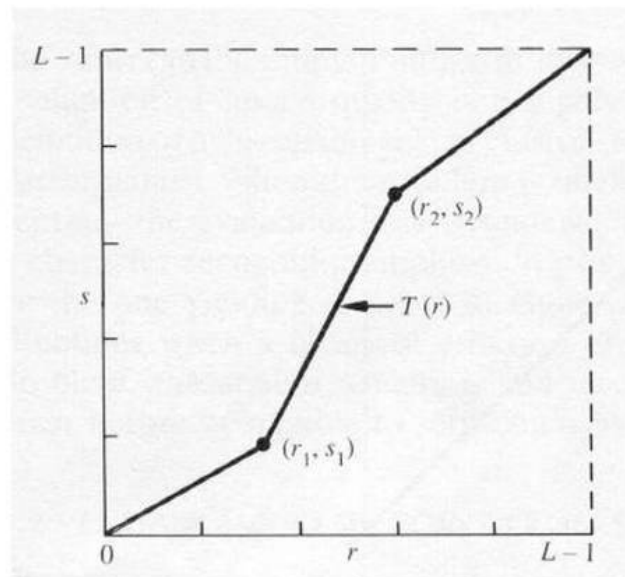
IF THE CONTROL POINTS ARE... THE FUNCTION BECOMES...

$r_1 = s_1$ and $r_2 = s_2$	a straight line — a linear function that produces no change at all
$r_1 = r_2, s_1 = 0, s_2 = L - 1$	a step — a thresholding function that creates a binary image
anything in between	various degrees of spread in the output gray levels, so various degrees of contrast

In general $r_1 \leq r_2$ and $s_1 \leq s_2$ are assumed, which keeps the function single-valued and monotonically increasing — the same two conditions that will come back in Chapter 20.

FROM YOUR LECTURE SLIDES

Contrast Stretching



The same transformation applied to a real low-contrast image.

Thresholding

The extreme case of contrast stretching. Everything below the threshold becomes 0 and everything above becomes $L - 1$, producing a **two-level (binary) image**. You have already used it.

YOU HAVE ALREADY DONE THIS

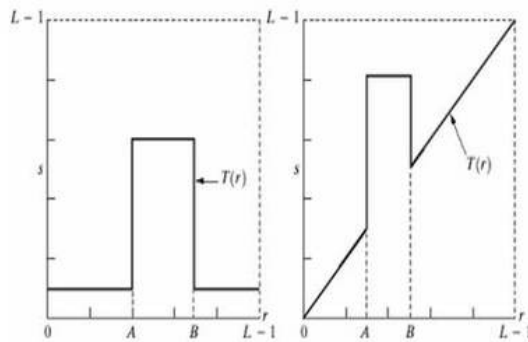
Assignment 5, parts 2 and 5: Otsu's method picked the threshold 119 automatically, while a fixed 128 changed 9,071 pixels. Same operation, different corner position.

Gray-level slicing

Here the aim is to **highlight one specific range of gray levels** $[A, B]$ — water in a satellite image, or a particular tissue in a scan. There are two versions, and the difference between them is a standard question.

FROM YOUR LECTURE SLIDES

Gray-level slicing



- Highlighting a specific range of gray levels in an image
 - Display a high value of all gray levels in the range of interest and a low value for all other gray levels
- (a) transformation highlights range $[A, B]$ of gray level and reduces all others to a constant level, Gray-level slicing without background
- (b) transformation highlights range $[A, B]$ but preserves all other levels, Gray-level slicing with Background

12

(a) Slicing *without* background: the range $[A, B]$ is given a high value and everything else is pushed down to a single constant low level. (b) Slicing *with* background: the range $[A, B]$ is raised but all other levels are preserved as they were.

FOR THE EXAM

Say it in one line each. **Without background** — the range of interest is brightened and the rest is flattened to one constant, so you lose the context. **With background** — the range of interest is brightened and the rest keeps its original values, so you can still see where the highlighted region sits in the picture.

CHAPTER 18 LECTURES 13 & 14 UNIT 2

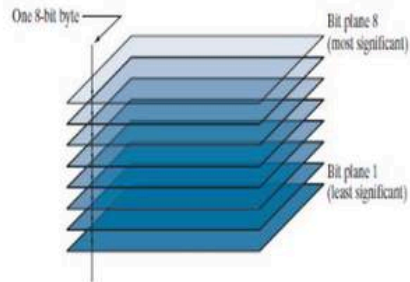
Bit-plane slicing

Instead of slicing by gray *level*, this technique slices by **bit position**. An 8-bit image can be imagined as a stack of eight 1-bit images laid on top of each other.

plane 1 = the lowest-order bit (LSB) of every pixel ... plane 8 = the highest-order bit (MSB) of every pixel

FROM YOUR LECTURE SLIDES

Bit-plane slicing



- Highlighting the contribution made to total image appearance by specific bits
- Suppose each pixel is represented by 8 bits
- Higher-order bits contain the majority of the visually significant data
- Useful for analyzing the relative importance played by each bit of the image

4

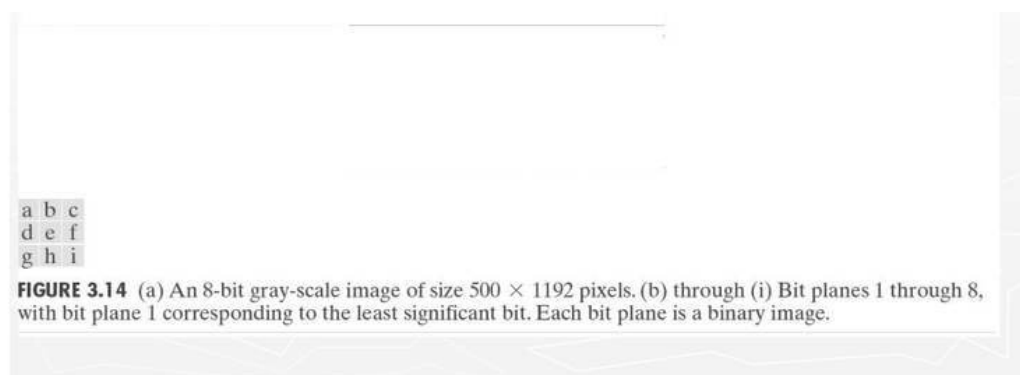
The eight planes of an 8-bit image. Each plane on its own is a binary image.

The reason it is useful: **the higher-order bits contain the majority of the visually significant data**, while the lower planes contribute the fine, almost random-looking detail. So bit-plane slicing tells you how much each bit is really worth.

Your handout gives two main goals:

1. convert a gray-level image into a binary image,
2. represent the image with fewer bits, that is, compress it.

FROM YOUR LECTURE SLIDES



An 8-bit image and its eight bit planes. Planes 8 and 7 still look like the picture; plane 1 looks like noise.

The worked example from your slides

Take this 3×3 patch of the standard `coins.png` image:



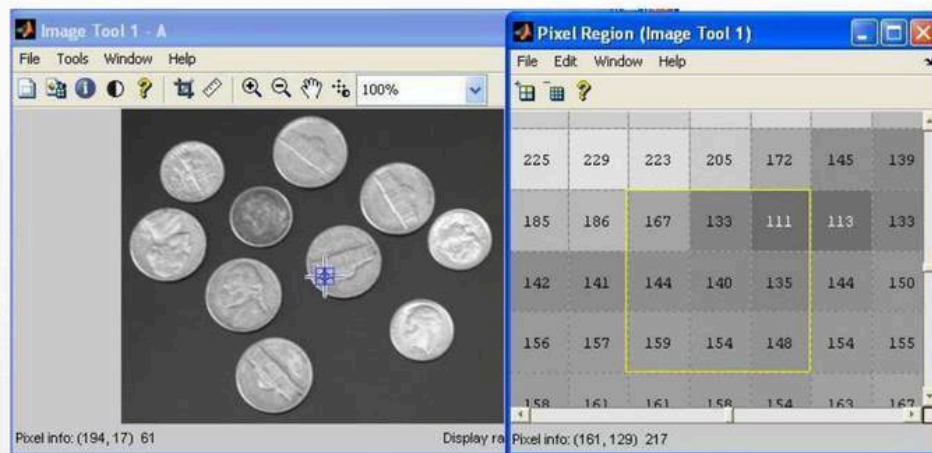
$$A = \begin{bmatrix} 167 & 133 & 111 \\ 144 & 140 & 135 \\ 159 & 154 & 148 \end{bmatrix}$$

In binary, 167 is 10100111 and 144 is 10010000. Reading one bit position out of every pixel gives one plane. In MATLAB that is `bitget` :

```
B = bitget(A,1); % lowest-order bit of all pixels
B = bitget(A,8); % highest-order bit
1 1 1 1 1 0 0 0 1 1 1 1 1 0 0 1 1 1
```

FROM YOUR LECTURE SLIDES

Digitally, an image is represented in terms of pixels. These pixels can be expressed further in terms of bits. Consider the image 'coins.png' and the pixel representation of the image.



Where those numbers come from: the pixel region tool showing the actual gray values inside the yellow box.

Rebuilding the image from a few planes

Each plane carries a weight, because it is a binary place value:

plane n contributes 2^{n-1}
 plane 8 $\rightarrow$ 128, plane 7 $\rightarrow$ 64, plane 6 $\rightarrow$ 32, ... plane 1 $\rightarrow$ 1

So if you keep only planes 8 and 7 of the pixel 10100111 and throw the rest away:

$$(1 \times 128) + (0 \times 64) + 0 + 0 + 0 + 0 + 0 + 0 = 128$$

Doing that for all nine pixels turns the patch into

128	128	64
128	128	128
128	128	128

only two planes kept, so only a few distinct values remain

In MATLAB the rebuild uses `bitset` to put the bits back:

```
A = imread('coins.png'); B = zeros(size(A)); B = bitset(B, 8,
bitget(A,8)); B = bitset(B, 7, bitget(A,7)); B = uint8(B); figure,
imshow(B);
```

Adding planes 6 and 5 as well gives a picture that is already close to the original — which is exactly the point about where the visual information lives.

FOR THE EXAM

A very likely numerical: *"Find the 8th and 1st bit planes of the given matrix", or "reconstruct using planes 8 and 7".* Method: write each number in 8-bit binary, read down the column you want, and for a rebuild multiply each kept plane by its place value and add. Check yourself with one pixel: keeping only plane 8 can give only 0 or 128.

CHAPTER 19 LECTURES 13 & 14 UNIT 2

The histogram

The histogram is the single most useful summary of an image, and the next three chapters are built on it.

$$h(r_k) = n_k$$

r_k = the k-th gray level · n_k = how many pixels in the image have that gray level

It is a plot of the **number of occurrences of each gray level against the gray level value**. Nothing more. For an 8-bit image it is a list of 256 counts.

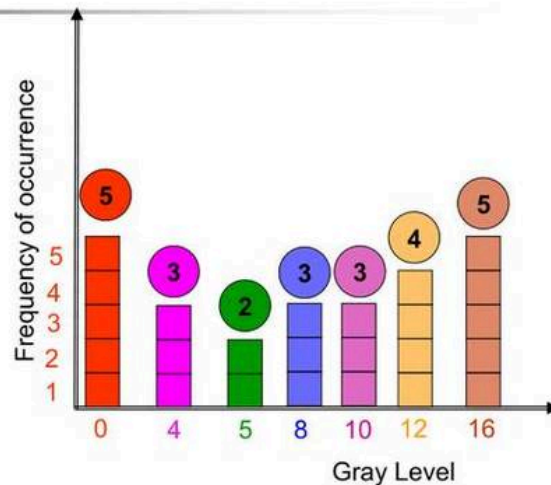
FROM YOUR LECTURE SLIDES

Image Histogram



0	4	8	10	12
12	16	5	0	16
4	16	8	5	10
10	0	4	12	16
12	0	16	0	8

A plot of number of occurrences of grey levels in the image against the grey level values



18

A small image and its histogram, counted level by level. Gray level 16 occurs 5 times, level 12 occurs 3 times, and so on.

The normalized histogram

It is common practice to divide each count by the total number of pixels:

$$p(r_k) = n_k / n \text{ for } k = 0, 1, \dots, L-1$$

$n = MN$, the total number of pixels

Two facts follow, and both get asked:

- $p(r_k)$ is an **estimate of the probability** of gray level r_k occurring in the image.
- The sum of all components of a normalized histogram is **exactly 1**.

That is what lets us treat the gray levels as a random variable and use probability theory on them, which is the whole trick behind histogram equalization.

Reading an image from its histogram

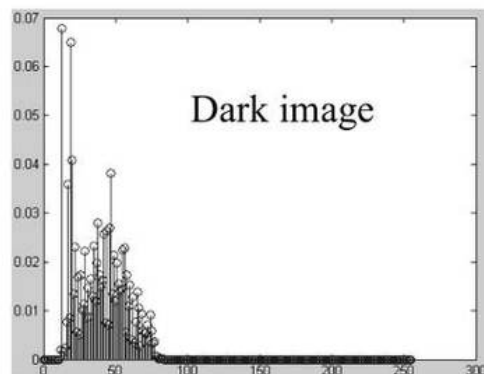
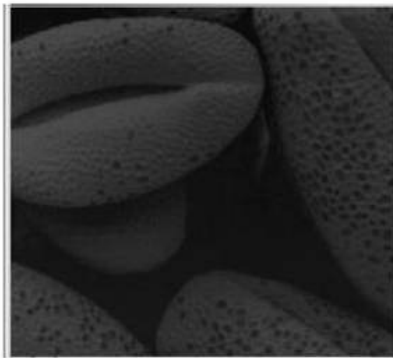
The *shape* of the histogram tells you what is wrong with the picture, and therefore what to do about it.

HISTOGRAM LOOKS LIKE	THE IMAGE IS	WHY
bunched on the low side	dark	most pixels have small values
bunched on the high side	bright	most pixels have large values
narrow and centred in the middle	low contrast	every pixel is a similar shade of mid-gray
broad and fairly uniform , covering the whole scale	high contrast	pixels use the full range, with no single level dominating

FROM YOUR LECTURE SLIDES

Some Typical Histograms

- The shape of a histogram provides useful information for contrast enhancement.

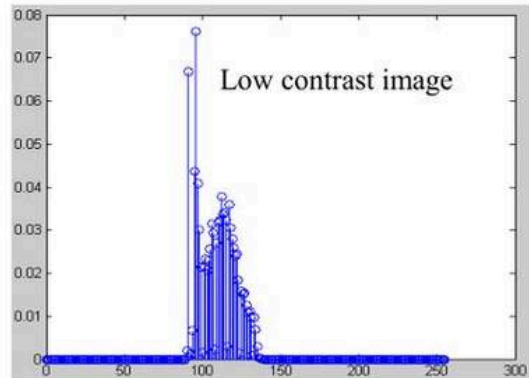
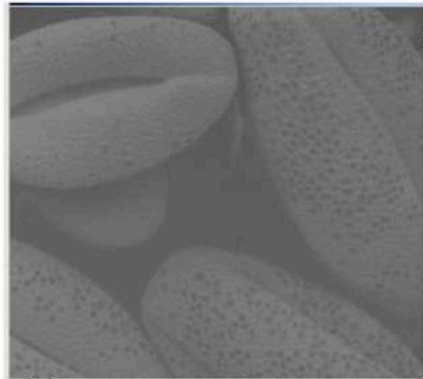
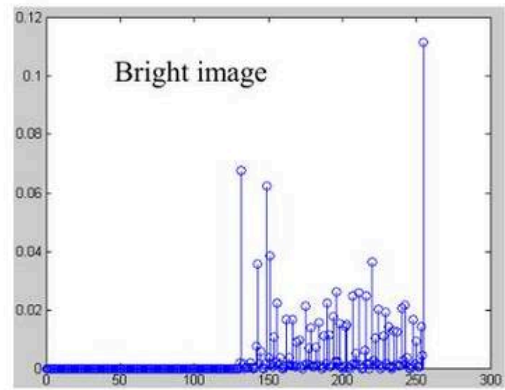


```
>> clear;
[ix,map]=imread('Fig3_15a.jpg');
imshow(ix)
figure;
ix=double(ix);
h=histogram(ix);
figure
stem(0:255,h);
.. |
```

19

A dark image: everything is pushed against the left edge.

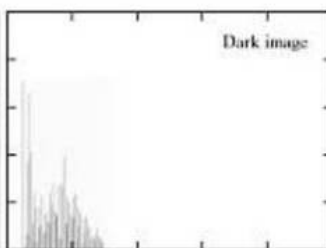
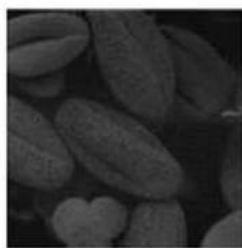
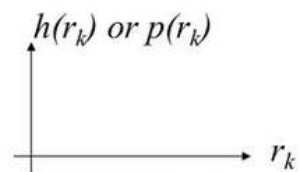
FROM YOUR LECTURE SLIDES



A bright image, and a low-contrast one whose histogram is narrow.

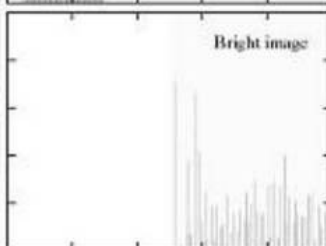
FROM YOUR LECTURE SLIDES

Example



Dark image

The components of the histogram are concentrated on the low side of the grayscale.



Bright image

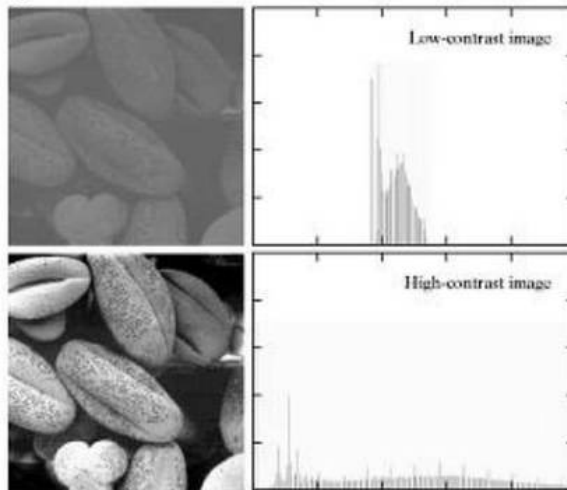
The components of the histogram are concentrated on the high side of the grayscale.

25

The same scene captured dark and bright, with the histograms below each.

FROM YOUR LECTURE SLIDES

Example



Low-contrast image

histogram is narrow and centered toward the middle of the gray scale

High-contrast image

histogram covers a broad range of the grayscale and the distribution of pixels is not too far from uniform, with very few vertical lines being much higher than the others

26

Low contrast against high contrast. The good picture is the one whose histogram covers a broad range and is not too far from uniform.

FOR THE EXAM

The last row of that table is the target, and it is the reason histogram equalization exists: *a good-looking image tends to have a histogram that spreads across the whole scale and is roughly uniform*. So "make the histogram uniform" is a sensible automatic recipe for improving contrast.

WHAT THE HISTOGRAM CANNOT TELL YOU

It counts pixels but forgets **where** they were. Two completely different pictures — a photograph and the same pixels shuffled at random — have exactly the same histogram. So the histogram describes the *distribution* of brightness, never the arrangement.

CHAPTER 20 LECTURES 14, 15 & 16 UNIT 2

Histogram equalization

The idea in one line: a low-contrast image has a narrow histogram, so **spread that histogram over the full range** and the contrast improves. We do it by finding a transformation that makes the probabilities spread out evenly.

The conditions $T(r)$ must satisfy

Before any formula, the transformation $s = T(r)$ for $0 \leq r \leq L-1$ has to obey two conditions, and you should be able to state both *with their reasons*.

- (a) $T(r)$ is **single-valued** and **monotonically increasing** on $0 \leq r \leq L-1$.
- Monotonically increasing guarantees that output values never go below the corresponding input values, which **prevents artifacts caused by reversals of intensity**.
 - Single-valued (a one-to-one relationship) guarantees that **the inverse transformation will exist**.
 - Monotonicity also **preserves the increasing order from black to white**, so the output is not accidentally a negative image.
- (b) $0 \leq T(r) \leq L-1$ for $0 \leq r \leq L-1$. This guarantees that **the output range is the same as the input range**.
- (a') If we also need the inverse $r = T^{-1}(s)$, condition (a) is tightened to **strictly monotonically increasing**, which makes the mapping from s back to r one-to-one and **prevents ambiguities**.

The continuous case

Treat the gray levels as a random variable with probability density function $p_r(r)$. The transformation we choose is the **cumulative distribution function (CDF)**:

$$s = T(r) = (L - 1) \int_0^r p_r(w) dw$$

w is a dummy variable of integration

Why the CDF? Because it satisfies condition (a) automatically: it is the integral of a function that is always positive, so it is an area that can only grow, which makes it single-valued and monotonically increasing.

Proving that the result is uniform

This derivation appears in your slides and is a favourite theory question. Differentiate the transformation using Leibniz's rule — the derivative of a definite integral with respect to its upper limit is just the integrand at that limit:

$$ds/dr = (L - 1) p_r(r)$$

Now use the rule for transforming a density, $p_s(s) = p_r(r) |dr/ds|$:

$$p_s(s) = p_r(r) \cdot 1 / [(L-1) p_r(r)] = 1 / (L - 1)$$

for $0 \leq s \leq L - 1$

The $p_r(r)$ cancels completely. The output density is a constant — a **uniform probability density function** — and, remarkably, it is **uniform no matter what the input histogram looked like**. That is the whole theorem.

The discrete case, which is what you actually compute

Probabilities become counts and the integral becomes a sum:

$$p_r(r_k) = n_k / MN \quad s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) = ((L - 1) / MN) \sum_{j=0}^k n_j$$

for $k = 0, 1, \dots, L - 1$, and the answer is then rounded to the nearest integer

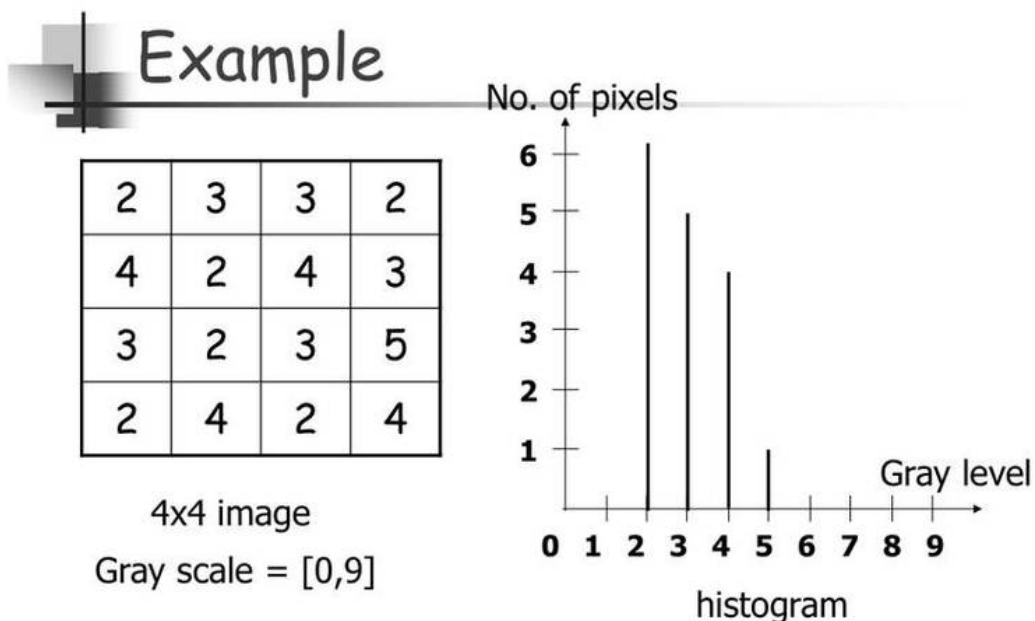
THE RECIPE, FIVE STEPS

1. Count n_k for every level. 2. Divide by MN to get $p_r(r_k)$. 3. Add them up as you go to get the running (cumulative) total. 4. Multiply each running total by $L-1$. 5. Round to the nearest whole number — that is s_k , the new value for every pixel that used to be r_k .

Worked example 1 — the 4×4 image

A 4×4 image on a gray scale of $[0, 9]$:

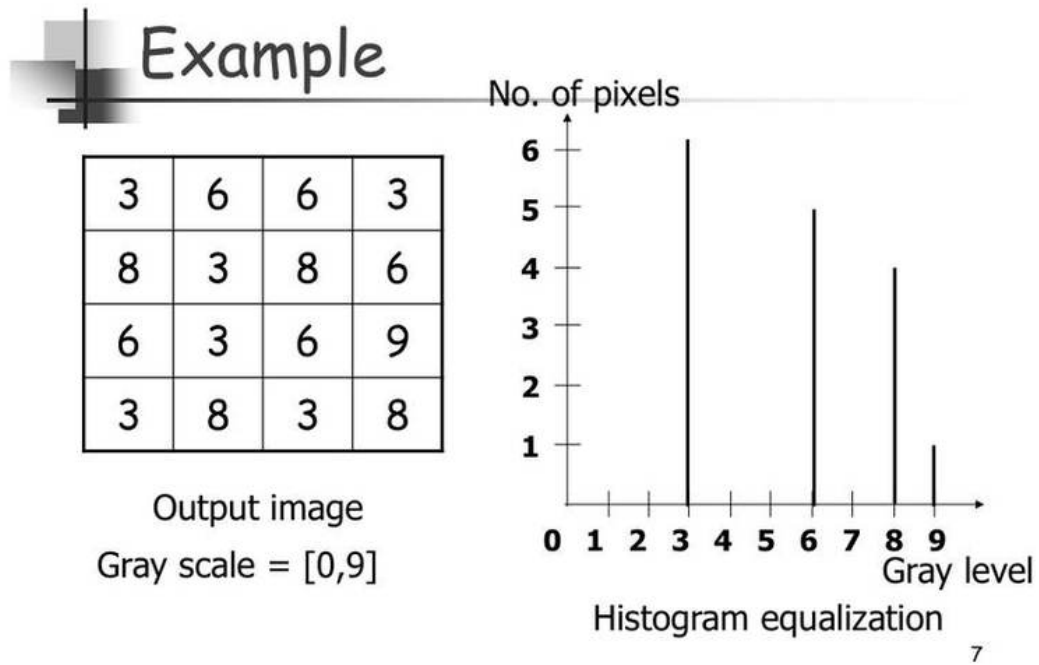
FROM YOUR LECTURE SLIDES



6

The input. Only the levels 2, 3, 4 and 5 are used, all bunched in the middle — a low-contrast image.

FROM YOUR LECTURE SLIDES



After equalization. The same four groups of pixels now sit at 3, 6, 8 and 9, spread across the scale.

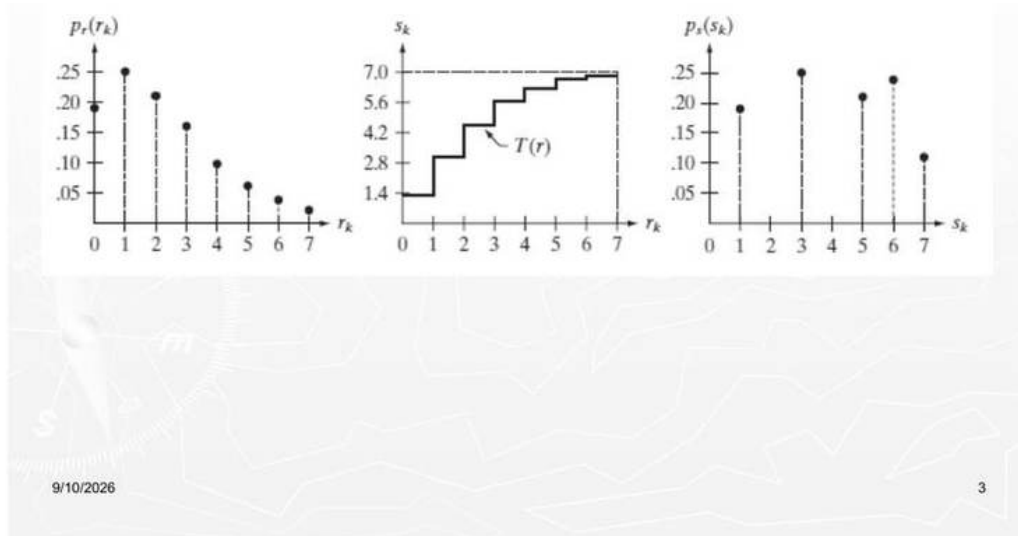
Notice what did and did not happen. The *number of distinct gray levels did not increase* — there are still four of them. They were simply **pushed apart**, and the level that had most pixels got the biggest jump.

Worked example 2 — the standard 3-bit table

A 3-bit image ($L = 8$) of size 64×64 , so $MN = 4096$. This is the example your teacher worked through in full, and very likely the one that appears in the exam.

R_K	N_K	$P_R(R_K) = N_K/MN$	CUMULATIVE	$S_K = 7 \times \text{CUMULATIVE}$	ROUNDED S_K
0	790	0.19	0.19	1.33	1
1	1023	0.25	0.44	3.08	3
2	850	0.21	0.65	4.55	5
3	656	0.16	0.81	5.67	6
4	329	0.08	0.89	6.23	6
5	245	0.06	0.95	6.65	7
6	122	0.03	0.98	6.86	7
7	81	0.02	1.00	7.00	7

FROM YOUR LECTURE SLIDES



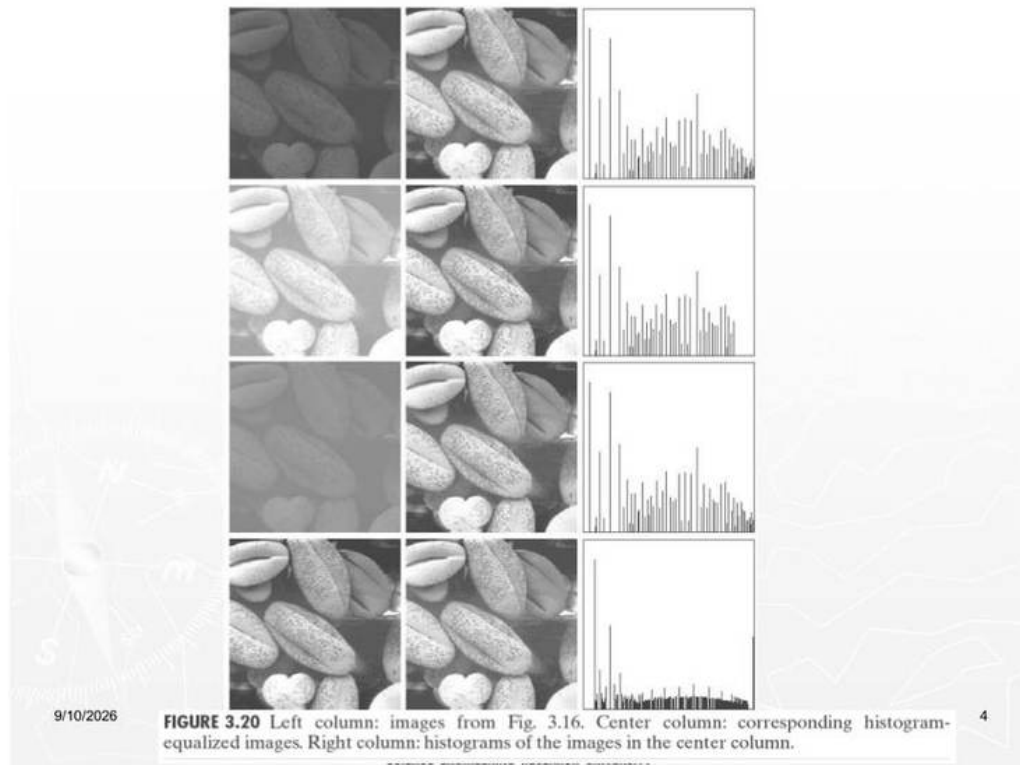
The same table as it appears in your slides, with the intensity distribution on the left.

Now read the result off the table. Eight input levels have been mapped onto only **five distinct output levels** — 1, 3, 5, 6 and 7 — because levels 3 and 4 both became 6, and levels 5, 6 and 7 all became 7. The output histogram is therefore:

$$p_s(1) = 0.19 \quad p_s(3) = 0.25 \quad p_s(5) = 0.21 \quad p_s(6) = 0.16 + 0.08 = 0.24$$

$$p_s(7) = 0.06 + 0.03 + 0.02 = 0.11$$

FROM YOUR LECTURE SLIDES



The four plots the question asks for: n_k against r_k , $p_r(r_k)$ against r_k , the transformation s_k against r_k , and the output $p_s(s_k)$ against s_k .

What to say about the result

- The output histogram is **not perfectly uniform**. In the discrete case it cannot be proved that the transformation produces a uniform histogram, because rounding forces several input levels onto the same output level. The continuous proof does not survive rounding.
- Equalization always **reaches the maximum gray level**, because the cumulative distribution equals 1 at the top, and $1 \times (L-1) = L-1$.
- Where cumulative counts are close together, the levels are mapped to the same or nearly the same output — so **the discrete transformation cannot guarantee a one-to-one mapping**.
- Because the transformation is built from the image's own histogram, it is **automatic**: no parameters to choose. That is its great strength as an adaptive, autonomous contrast-enhancement tool.

FROM YOUR LECTURE SLIDES

- The first three results from top to bottom show significant improvement.

Four versions of the same picture — dark, light, low contrast and high contrast — each with its histogram before and after. The first three improve a great deal; the fourth barely changes, because its intensities already spanned almost the full scale.

FOR THE EXAM

Two marks are regularly lost here. First, **always round** s_k to the nearest integer and show the unrounded value beside it. Second, when asked for the output histogram, **add the probabilities of every input level that landed on the same output level** — that merging is the whole point of the answer.

IS HISTOGRAM EQUALIZATION ALWAYS GOOD?

No. It is automatic, which also means it is blind. It can stretch a noisy dark region until the noise becomes very visible, and it gives you only one possible result — a roughly uniform histogram — whether that suits the picture or not. The fix for the second problem is the next chapter.

CHAPTER 21 LECTURES 16 & 17 UNIT 2

Histogram matching (specification)

Histogram equalization has one disadvantage: **it can generate only one type of output image**, the one with a roughly uniform histogram. With histogram matching, also called histogram specification, *you* choose the shape the output histogram should have. It does not have to be uniform, and that lets you enhance a specific portion of the gray scale.

The continuous theory

Let $p_r(r)$ be the density of the input image and $p_z(z)$ the density you want the output to have. Define two equalizations:

$$s = T(r) = (L - 1) \int_0^r p_r(w) dw \quad \text{equalize the input} \quad G(z) = (L - 1) \int_0^z p_z(t) dt = s \quad \text{equalize the target}$$

Both flatten their own density to the same uniform result, so we can join them at s . Then

$$z = G^{-1}(s) = G^{-1}[T(r)]$$

IN SIMPLE WORDS

Uniform is the meeting point. Push the input *forward* to uniform with T , then push *backward* from uniform to the shape you asked for with G^{-1} . The combination takes you from the histogram you have to the histogram you want.

The procedure, as your handout states it

1. Obtain $T(r)$ by calculating the histogram equalization of the input image.
2. Obtain $G(z)$ by calculating the histogram equalization of the desired density function.
3. Obtain the inverse transformation G^{-1} .
4. Equalize the input image to get the s values, then for each pixel apply the inverse mapping $z = G^{-1}(s)$ to get the output pixel. When every pixel has been processed, the PDF of the output image will equal the specified PDF.

The continuous example from your slides

Input density and specified density:

$$p_r(r) = 2r / (L-1)^2 \quad p_z(z) = 3z^2 / (L-1)^3$$

both zero outside $0 \leq \text{value} \leq L-1$

$$\text{Step 1: } s = T(r) = (L-1) \int_0^r 2w / (L-1)^2 dw = r^2 / (L-1)$$

$$\text{Step 2: } G(z) = (L-1) \int_0^z 3t^2 / (L-1)^3 dt = z^3 / (L-1)^2$$

$$\text{Step 3: set } G(z) = s \text{ and solve for } z: z^3 = (L-1)^2 s, \text{ so } z = [(L-1)^2 s]^{1/3}$$

$$\text{Step 4: substitute } s = r^2 / (L-1):$$

$$z = [(L-1)^2 \cdot r^2 / (L-1)]^{1/3} = [(L-1) r^2]^{1/3}$$

One formula now takes any input pixel r straight to its output pixel z .

The discrete case

Analytical expressions for $T(r)$ and G^{-1} are hard to find in practice. With integers the job becomes much easier, at the price of only approximating the desired histogram.

$$s_k = T(r_k) = (L-1) \sum_{j=0}^k p_r(r_j) \text{ round to } [0, L-1] \quad G(z_q) = (L-1) \sum_{i=0}^q p_z(z_i)$$

$$p_z(z_i) = s_k \text{ round to } [0, L-1] \quad z_q = G^{-1}(s_k)$$

THE TRICK THAT AVOIDS INVERTING ANYTHING

You never actually compute G^{-1} . Instead compute **all** the values of $G(z_q)$ for $q = 0, 1, \dots, L-1$, round them, and store them in a table. Then, given a value s_k , simply **look down the table for the closest match**. If the closest entry is the 64th, then $q = 63$ and that s_k maps to z_{63} .

The worked discrete example

Same 3-bit, 64×64 image as Chapter 20, with this specified histogram:

z_q	0	1	2	3	4	5	6	7
specified $p_z(z_q)$	0.00	0.00	0.00	0.15	0.20	0.30	0.20	0.15
cumulative	0.00	0.00	0.00	0.15	0.35	0.65	0.85	1.00
$G(z_q) = 7 \times \text{cumulative}$	0	0	0	1.05	2.45	4.55	5.95	7.00
rounded $G(z_q)$	0	0	0	1	2	5	6	7

FROM YOUR LECTURE SLIDES

Example: Histogram Matching

Suppose that a 3-bit image ($L=8$) of size 64×64 pixels ($MN = 4096$) has the intensity distribution shown in the following table (on the left). Get the histogram transformation function and make the output image with the specified histogram, listed in the table on the right.

r_k	n_k	$p_r(r_k) = n_k/MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

z_q	Specified $p_z(z_q)$
$z_0 = 0$	0.00
$z_1 = 1$	0.00
$z_2 = 2$	0.00
$z_3 = 3$	0.15
$z_4 = 4$	0.20
$z_5 = 5$	0.30
$z_6 = 6$	0.20
$z_7 = 7$	0.15

9/10/2026

8

The question as set: the input distribution on the left, the specified histogram on the right.

The s_k values were already computed in Chapter 20: **1, 3, 5, 6, 6, 7, 7, 7**. Now match each one to the nearest value in the G table:

R_K	S_K	CLOSEST $G(Z_Q)$	SO $Z_Q =$
0	1	1 (at $q = 3$)	3
1	3	2 (at $q = 4$)	4
2	5	5 (at $q = 5$)	5
3	6	6 (at $q = 6$)	6
4	6	6 (at $q = 6$)	6
5	7	7 (at $q = 7$)	7
6	7	7 (at $q = 7$)	7
7	7	7 (at $q = 7$)	7

FROM YOUR LECTURE SLIDES

Example: Histogram Matching

$$r_k \rightarrow z_q$$

$$0 \rightarrow 3$$

$$1 \rightarrow 4$$

$$2 \rightarrow 5$$

$$3 \rightarrow 6$$

$$4 \rightarrow 6$$

$$5 \rightarrow 7$$

$$6 \rightarrow 7$$

$$7 \rightarrow 7$$

The finished mapping, exactly as it appears in your slides: $0 \rightarrow 3$, $1 \rightarrow 4$, $2 \rightarrow 5$, $3 \rightarrow 6$, $4 \rightarrow 6$, $5 \rightarrow 7$, $6 \rightarrow 7$, $7 \rightarrow 7$.

Counting the pixels that arrive at each output level gives the achieved histogram: z_3 receives 790 pixels (0.19), z_4 receives 1023 (0.25), z_5 receives 850 (0.21), z_6 receives $656 + 329 = 985$ (0.24) and z_7 receives $245 + 122 + 81 = 448$ (0.11). Compare that with the specified 0.15, 0.20, 0.30, 0.20, 0.15 — close, but not exact, which is the expected price of working with integers.

FROM YOUR LECTURE SLIDES

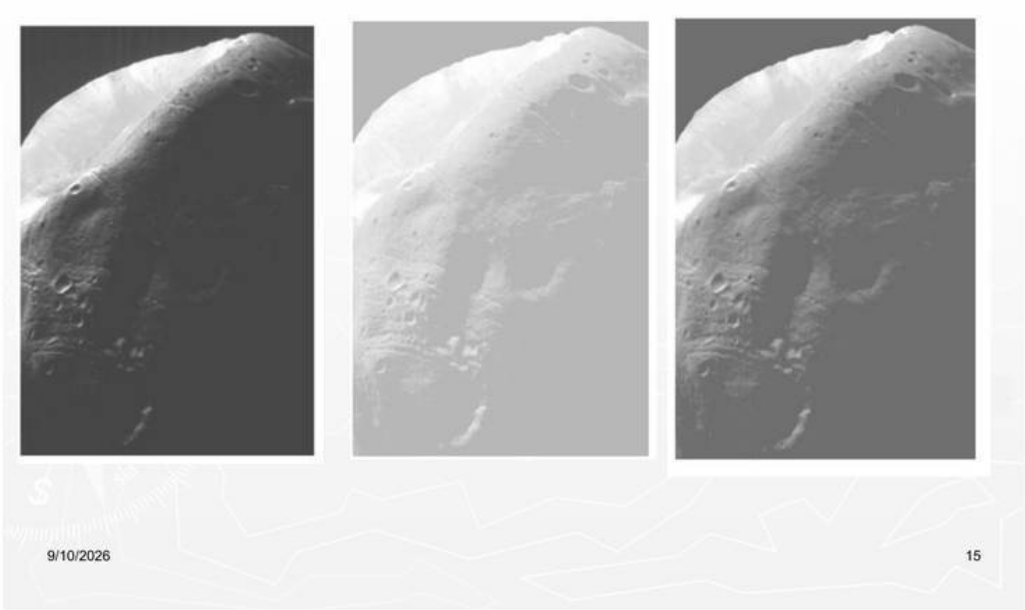
Figure 3.25

(a) Histogram equalization transformation obtained using the histogram in Fig. 3.24(b). (b) Histogram equalized image. (c) Histogram of equalized image.

The transformation functions and histograms for the full-size example: $G(z_q)$ labelled (1) and $G^{-1}(s_k)$ labelled (2).

FROM YOUR LECTURE SLIDES

Example: Histogram Matching



The result on a real image, where plain equalization had washed the picture out and specification recovered it.

FOR THE EXAM

Write the two transformations side by side in a single table — r_k, n_k, p_r , cumulative, s_k in the first block and z_q, p_z , cumulative, $G(z_q)$ in the second — then do the matching in a third small table. Marks are given for the table structure as much as for the final numbers. And remember the honest closing line: **histogram specification is a trial-and-error process. There are no rules for specifying histograms**, so each enhancement task must be analysed case by case.

Local enhancement and histogram statistics

Why global processing is not enough

Everything in the last three chapters was **global**: one transformation function, built from the gray-level content of the *entire* image, applied to every pixel. That fails when the detail you care about lives in a small area — a few hundred pixels cannot move a histogram built from a quarter of a million.

Local histogram processing

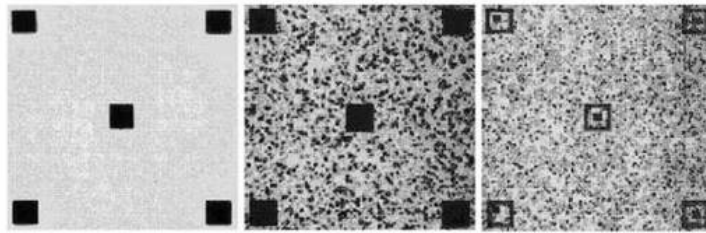
The fix is to do the same work in a small window. The procedure is short enough to memorise:

1. Define a neighbourhood — a square or rectangular window is usual.
2. Move its **centre** from pixel to pixel.
3. At each location, compute the histogram of the points inside that neighbourhood and obtain its histogram equalization transformation function.
4. Use it to map **only the intensity of the pixel at the centre**.
5. Move to the next location and repeat.

A cheaper variant uses **non-overlapping** regions, but it usually produces an undesirable "**blocky**" effect, because neighbouring blocks get unrelated transformations.

FROM YOUR LECTURE SLIDES

Local Enhancement



- a) Original image (slightly blurred to reduce noise)
- b) global histogram equalization (enhance noise & slightly increase contrast but the construction is not changed)
- c) local histogram equalization using 7x7 neighborhood (reveals the small squares inside larger ones of the original image).

(a)

(b)

(c)

- define a square or rectangular neighborhood and move the center of this area from pixel to pixel.
- at each location, the histogram of the points in the neighborhood is computed and histogram equalization transformation function is obtained.
- another approach used to reduce computation is to utilize nonoverlapping regions, but it usually produces an undesirable “**blocky**” effect. ³

(a) The original, slightly blurred to reduce noise. (b) Global histogram equalization: it enhances the noise and slightly increases contrast, but the structure is unchanged. (c) Local histogram equalization with a 7×7 neighbourhood, which reveals the small squares hidden inside the larger ones.

Your handout's explanation of why (c) works is worth learning as written: the original image contains many small squares inside the larger dark ones, but those small squares were **too close in gray level** to the larger ones and **too small in size** to influence global equalization significantly. The local technique reveals them. Note also that the finer noise texture in (c) is a side effect of using relatively small neighbourhoods.

Histogram statistics: mean and variance

Instead of using the whole histogram, you can summarise it with two numbers.

$$\begin{aligned} \text{mean } m &= \sum_{i=0}^{L-1} r_i p(r_i) = (1/MN) \sum_x \sum_y f(x, y) \quad n\text{-th moment } u_n(r) = \\ &= \sum_{i=0}^{L-1} (r_i - m)^n p(r_i) \quad \text{variance } \sigma^2 = u_2(r) = \sum (r_i - m)^2 p(r_i) = \\ &= (1/MN) \sum_x \sum_y [f(x, y) - m]^2 \end{aligned}$$

FROM YOUR LECTURE SLIDES

Using Histogram Statistics for Image Enhancement

For an image with intensity levels in the range $[0, L-1]$, the n^{th} moment of r about its mean, m , is defined as

$$u_n(r) = \sum_{i=0}^{L-1} (r_i - m)^n p(r_i)$$

Average Intensity (Mean)

$$m = \sum_{i=0}^{L-1} r_i p(r_i) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)$$

Variance

$$\sigma^2 = u_2(r) = \sum_{i=0}^{L-1} (r_i - m)^2 p(r_i) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x, y) - m]^2$$

9/10/2026

8

The formulas as given in your slides, in both the histogram form and the direct pixel-sum form.

STATISTIC	WHAT IT MEASURES	HIGH VALUE MEANS
Mean	average intensity — the overall brightness	a brighter image; a low mean means a darker image
Variance (or its square root, the standard deviation)	how spread out the intensities are — the contrast	greater differences between pixels, so higher contrast; it can also mean a grainier, noisier picture

Variance earns its place because it is used everywhere:

- **Local contrast adaptive enhancement** — adjusting pixels using the local mean and variance avoids over-amplifying noise in uniform regions while still sharpening low-contrast ones.
- **Segmentation and thresholding** — Otsu's method chooses its threshold by comparing intra-class and inter-class variance.
- **Noise estimation** — uniform regions have low variance, so high variance means edges, detail, or additive noise.
- **Texture analysis** — smooth regions have near-zero variance; sand or foliage has high variance.

The same two quantities computed over a neighbourhood S_{xy} instead of the whole image are the **local mean** and **local variance**:

$$m_{s_{xy}} = \sum r_i p_{s_{xy}}(r_i) \quad \sigma_{s_{xy}}^2 = \sum (r_i - m_{s_{xy}})^2 p_{s_{xy}}(r_i)$$

Adaptive local enhancement

Now the two statistics are put to work. The rule scans the image pixel by pixel with a small window (3×3 or 5×5) and **brightens only the areas that are both dark and low in contrast**, leaving everything else untouched:

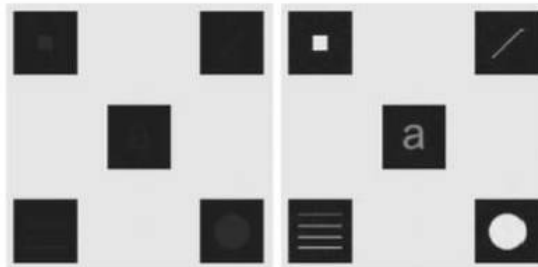
$$g(x, y) = C \cdot f(x, y) \text{ if } m_{s_{xy}} \leq k_0 m_G \text{ AND } k_1 \sigma_G \leq \sigma_{s_{xy}} \leq k_2 \sigma_G \\ = f(x, y) \text{ otherwise}$$

FROM YOUR LECTURE SLIDES

$$g(x, y) = \begin{cases} C f(x, y) & \text{if } k_0 m_G \leq m_{s_{xy}} \leq k_1 m_G \text{ AND } k_2 \sigma_G \leq \sigma_{s_{xy}} \leq k_3 \sigma_G \\ f(x, y) & \text{otherwise} \end{cases}$$

a b

FIGURE 3.27
(a) Original image. (b) Result of local enhancement based on local histogram statistics. Compare (b) with Fig. 3.26(c).



for $x = 0, 1, 2, \dots, M-1$ and $y = 0, 1, 2, \dots, N-1$, where, as indicated above, C , k_0 , k_1 , k_2 , and k_3 are specified constants, m_G is the global mean of the input image, and σ_G is its standard deviation. Parameters $m_{s_{xy}}$ and $\sigma_{s_{xy}}$ are the local mean and standard deviation, respectively, which change for every location (x, y) . As usual, M and N are the number of rows and columns in the input image.

9/10/2026

10

The rule exactly as written in your slides.

SYMBOL	NAME	MEANING
$f(x, y)$	original pixel	the input intensity at (x, y)
$g(x, y)$	enhanced pixel	the output after the rule is applied
C	gain factor	a multiplier, $C > 1$, that boosts the intensity of the pixels that qualify
m_G	global mean	average intensity of the whole image
σ_G	global standard deviation	contrast spread of the whole image
m_{Sxy}	local mean	average inside the window centred at (x, y)
σ_{Sxy}	local standard deviation	spread inside that window
k_0, k_1, k_2	threshold parameters	positive constants chosen by the user, typically $k_0 < 1$, $k_1 > k_0$ and $k_2 < 1$

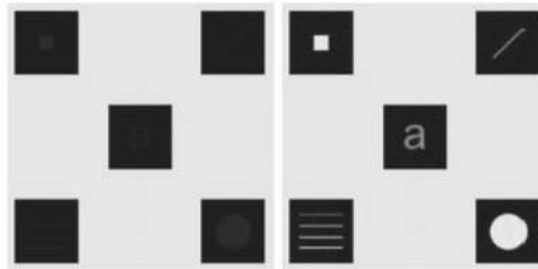
READING THE TWO CONDITIONS IN PLAIN WORDS

First condition, $m_{Sxy} \leq k_0 m_G$: "this patch is darker than the picture as a whole", so it is a candidate for brightening. **Second condition**, $k_1 \sigma_G \leq \sigma_{Sxy} \leq k_2 \sigma_G$: "this patch has some variation but not much". The lower bound throws away perfectly flat patches, which contain nothing to reveal, and the upper bound throws away patches that already have plenty of contrast. Only what is left — dark *and* faint — gets multiplied by C .

FROM YOUR LECTURE SLIDES

a b

FIGURE 3.27
(a) Original image. (b) Result of local enhancement based on local histogram statistics. Compare (b) with Fig. 3.26(c).

**Image Example Interpretation**

•**Left Image (Input):** Contains five small dark squares against a bright gray backdrop. Hidden inside four corner boxes and the center box are very faint, low-contrast shapes (a square, line, 'a', horizontal bars, circle).

•**Right Image (Output):** The algorithm evaluated each pixel's local mean and standard deviation. It detected the faint shapes inside the dark boxes, satisfied both logical conditions, multiplied their intensity values by C, and revealed the hidden shapes brightly without ruining the surrounding background.



3.26(C)

9/10/2026

15

Left, the input: five dark squares on a bright backdrop, with very faint low-contrast shapes hidden inside them — a square, a line, the letter "a", horizontal bars and a circle. Right, the output: the algorithm found the patches that were both dark and faint, multiplied them by C, and revealed the hidden shapes without touching the background.

FOR THE EXAM

If asked what role C plays, the answer is that it is the **gain**: it decides how strongly the qualifying pixels are boosted. Too small and nothing is revealed; too large and the enhanced patches saturate to white. The k values decide *which* pixels qualify, and C decides *how much* they change.

WHAT COMES NEXT IN YOUR COURSE

The last slide of Lecture 19 says it: **mask processing**. That is the other half of Chapter 15's table — neighbourhoods bigger than 1×1 , which means smoothing and sharpening filters, and then the whole frequency-domain half of Unit 2. Everything there is built on the convolution you already learned in Part III.

CHAPTER 23 LECTURE 21 UNIT 2

Mask processing: filtering in the spatial domain

Everything from Chapter 16 to Chapter 22 was **point processing** — the new value of a pixel depended only on its own old value. From here on the new value depends on a *small window*

of pixels around it. That is neighbourhood processing, and the window is called a **mask**.

23.1 The vocabulary

Your slide gives four names for the same object: **filter**, **mask**, **kernel**, **template**, **window**. Use whichever your examiner used. Two more rules from the same slide:

- The numbers inside a mask are called **coefficients**, not pixels. A pixel is data; a coefficient is a weight you chose.
- Masks are almost always of **odd size** — 3×3 , 5×5 , 7×7 — because an odd size has a single centre pixel to sit on. A 4×4 mask has no centre, so there is no pixel to write the answer into.

FROM YOUR LECTURE SLIDES

Spatial Filtering: Equation Form

Diagram illustrating spatial filtering. A 3x3 mask is applied to a 3x3 grid of pixels. The mask coefficients are shown as $w(-1,-1)$ to $w(1,1)$. The pixels are labeled $f(x-1,y-1)$ to $f(x+1,y+1)$. The result is a single pixel value $f(x,y)$.

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Filtering can be given in equation form as shown above

Notations are based on the image shown to the left

Images taken from Gonzalez & Woods, Digital Image Processing (2002)

The whole idea in one picture. The mask sits on the image, its centre over the pixel being processed, and the answer goes into that same position of a *new* image.

23.2 The process, in four sentences

1. Put the centre of the mask on a pixel (x, y) .
2. Multiply every coefficient by the pixel underneath it.
3. Add all those products together. That single number is the **response** R.
4. Write R into position (x, y) of the output image, then slide the mask to the next pixel and repeat.

$$R = w_1 z_1 + w_2 z_2 + \dots + w_{mn} z_{mn} = \sum_{i=1}^{mn} w_i z_i$$

w = the coefficients of the mask · z = the pixels underneath them

FROM YOUR LECTURE SLIDES

- simply move the filter mask from point to point in an image.
- at each point (x,y) , the response of the filter at that point is calculated using a predefined relationship.

$$R = w_1 z_1 + w_2 z_2 + \dots + w_{mn} z_{mn}$$

$$= \sum_{i=1}^{mn} w_i z_i$$

3

The response written out as a plain sum. Nothing in mask processing is harder than this line.

CAREFUL – WRITE IT INTO A NEW IMAGE

You must never write the answer back into the image you are still reading from. If you do, the next window will use numbers you have already changed and the result is nonsense. Always keep an input image and a separate output image.

23.3 The general equation

The same four sentences, written as one line. This is the form to quote in an exam.

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot f(x+s, y+t)$$

$a = (m-1)/2$ · $b = (n-1)/2$ · mask is $m \times n$, both odd

To get a complete filtered image this equation has to be applied for $x = 0, 1, 2, \dots, M-1$ and $y = 0, 1, 2, \dots, N-1$ — that is, once per pixel.

IN SIMPLE WORDS – WHAT A AND B ARE

They are just "half the mask, rounded down". For a 3×3 mask, $m = n = 3$, so $a = b = 1$ and the sums run from -1 to $+1$ — the nine pixels around the centre. For a 5×5 mask $a = b = 2$ and the sums run from -2 to $+2$. That is all the letters mean.

23.4 What happens at the edges

When the mask centre sits on the first row, part of the mask hangs outside the image. There is nothing to multiply. The lecture uses the simplest fix: **zero padding** — surround the image with a border of zeros wide enough for the mask to sit on.

FROM YOUR LECTURE SLIDES

Illustration of Spatial filtering

7	9	11
10	50	8
9	5	6

Original Image

	1	1	1
1/9	1	1	1
	1	1	1

3 x 3 Averaging Mask

0	0	0	0	0
0	7	9	11	0
0	10	50	8	0
0	9	5	6	0
0	0	0	0	0

Input Image after zero padding



The 3×3 image on the left becomes a 5×5 array once a one-pixel border of zeros is added. A 3×3 mask can now be centred on every original pixel.

The lecture's worked example

Take the 3×3 image and the 3×3 box mask (every coefficient $1/9$):

$$\begin{vmatrix} 7 & 9 & 11 \\ 10 & 50 & 8 \\ 9 & 5 & 6 \end{vmatrix} \text{ filtered by } \begin{vmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{vmatrix} \times \frac{1}{9}$$

The corner pixel, value 7 at position (0,0). After zero padding, the nine values under the mask are 0, 0, 0, 0, 7, 9, 0, 10, 50.

$$g(0,0) = (0+0+0+0+7+9+0+10+50) / 9 = 76 / 9 = 8.4$$

The next pixel along, value 9 at position (0,1). Only the row above is missing now, so three zeros come in from the padding: 0, 0, 0, 7, 9, 11, 10, 50, 8.

$$g(0,1) = (0+0+0+7+9+11+10+50+8) / 9 = 95 / 9 = 10.56$$

Those two are the numbers printed on the slide. For completeness, the true centre pixel needs no padding at all — all nine real pixels are available:

$$g(1,1) = (7+9+11+10+50+8+9+5+6) / 9 = 115 / 9 = 12.78$$

FROM YOUR LECTURE SLIDES

7	9	11
10	50	8
9	5	6

Original Image

8.4	10.55	

Image after Spatial Averaging

The slide's own answer sheet. Note how the 50 — a single bright spike — has been dragged down towards its neighbours. That is exactly what averaging is for.

FOR THE EXAM

When the question says "using zero padding", the safest method is to *write out the padded array* before you start. Most lost marks in this topic are not arithmetic — they are pixels taken from the wrong place because the padding was done in the head.

Chapter 23 at a glance

- A mask is a small array of **coefficients**; masks are odd-sized so they have a centre.
- Response $R = \sum w_i z_i$ — multiply, add, write into a *new* image.
- General form $g(x, y) = \sum \sum w(s, t) f(x+s, y+t)$, with $a = (m-1)/2$ and $b = (n-1)/2$.
- Edges are handled by zero padding.

CHAPTER 24 LECTURE 21 UNIT 2

Smoothing (low-pass) filters

The first family of masks. Your slide gives the purpose in one line: **blurring and noise reduction**.

24.1 Why you would deliberately blur an image

- **Removing small detail** before pulling out objects — if you want the big shapes, the texture is in the way.
- **Bridging small gaps** in lines or curves, so a broken outline becomes a continuous one.

- **Reducing noise**, which is the one you will be asked about.

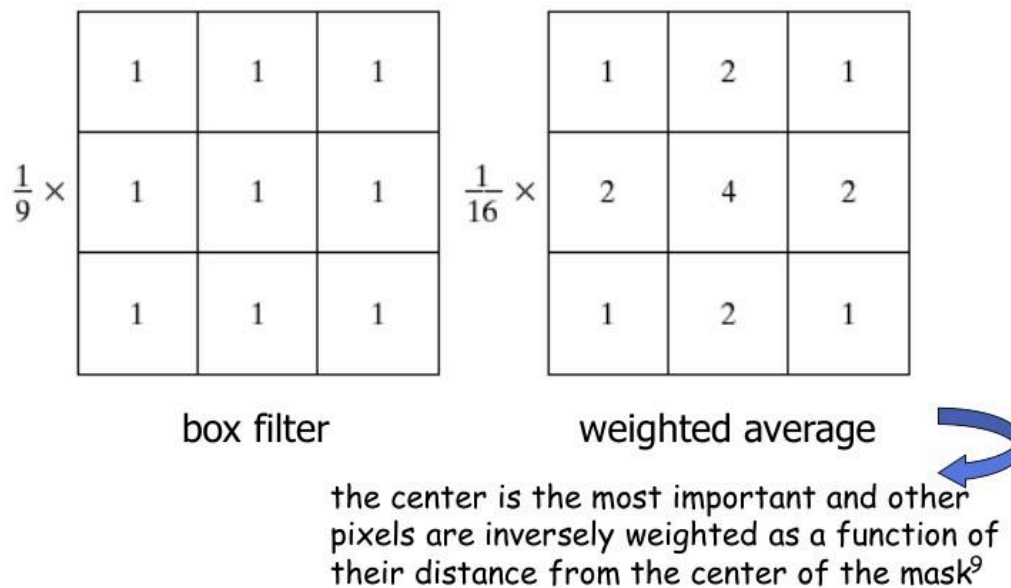
24.2 Averaging filters

The output is simply the **average of the pixels in the neighbourhood**. These are called **averaging filters** or **low-pass filters**, and the plain one with all coefficients equal is the **box filter**.

IN SIMPLE WORDS – WHY AVERAGING KILLS NOISE

Replacing every pixel by the average of its neighbourhood smooths out *sharp transitions* in gray level. Random noise is a sharp transition, so it goes. But so is an edge — so edges go soft too. Your slide states both halves honestly: smoothing **reduces noise (desirable)** and **blurs edges (undesirable)**. There is no averaging filter that does one without the other.

FROM YOUR LECTURE SLIDES



The two 3×3 smoothing masks you must be able to write down. Left: the box filter, every coefficient 1/9. Right: the weighted average, $1/16 \times [1\ 2\ 1; 2\ 4\ 2; 1\ 2\ 1]$.

24.3 The weighted average

In the weighted mask the centre is the most important coefficient and the others fall off with distance from it. Your slide gives the reason plainly: it is **an attempt to reduce blurring in the smoothing process**. You still average, but the pixel's own value is given the loudest vote, so it is dragged less far.

Once the coefficients are no longer all equal, dividing by 9 is wrong — you must divide by whatever the coefficients add up to. For the mask above, $1+2+1+2+4+2+1+2+1 = 16$, hence the $1/16$.

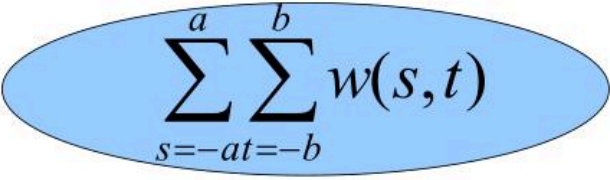
$$g(x, y) = \sum_s \sum_t w(s, t) f(x+s, y+t) \div \sum_s \sum_t w(s, t)$$

the denominator is the sum of all the coefficients of the mask

FROM YOUR LECTURE SLIDES

- filter of size $m \times n$ (m and n odd)

$$g(x, y) = \frac{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)}{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}$$



 summation of all coefficient of the mask

22

The general form. Dividing by the sum of the coefficients is what keeps the average brightness of the picture unchanged.

FOR THE EXAM

A smoothing mask's coefficients must **add up to 1** after the division. If they do not, the whole image gets brighter or darker. Contrast this with the sharpening masks in Chapter 26, where the coefficients add up to **0**. That one fact tells you at a glance which family a mask belongs to.

24.4 What the mask size does

FROM YOUR LECTURE SLIDES

Spatial Averaging



Original Image

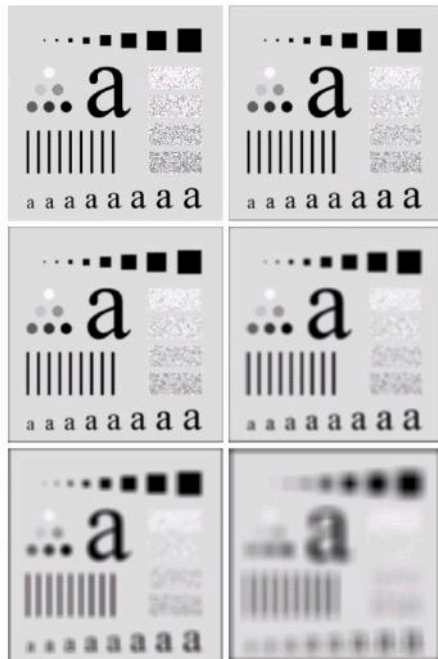
3×3
Smoothing
filter



Smoothed Image

A 3×3 average. The detail is softened but the subject is untouched.

FROM YOUR LECTURE SLIDES



Example

a	b
c	d
e	f

- a). original image 500x500 pixel
- b). - f). results of smoothing with square averaging filter masks of size $n = 3, 5, 9, 15$ and 35 , respectively.
- Note:
 - big mask is used to eliminate small objects from an image.
 - the size of the mask establishes the relative size of the objects that will be blended with the background.

20

The same 500x500 image with averaging masks of $n = 3, 5, 9, 15$ and 35 . Watch the small letters disappear first and the large black squares survive longest.

Your slide draws the conclusion for you, and it is the sentence examiners want:

THE RULE TO QUOTE

The size of the mask sets the size of the objects that get blended into the background. A big mask is used deliberately to eliminate small objects from an image. Anything noticeably smaller than the mask vanishes; anything much bigger survives.

Chapter 24 at a glance

- Smoothing = averaging = low-pass. It reduces noise and blurs edges, always both.
- Box filter: all coefficients equal, divide by $n \times n$.
- Weighted average: centre heaviest, divide by the *sum of the coefficients*; blurs less.
- Smoothing mask coefficients sum to 1. Bigger mask → bigger objects removed.

CHAPTER 25 LECTURE 21 UNIT 2

Order-statistic filters and the median

Every filter so far has been **linear** — multiply and add. **Order-statistic filters** are **non-linear**: the response is based on *ranking* the pixels inside the window, not on weighting them.

$$R = \text{median}\{ z_k \mid k = 1, 2, \dots, n \times n \}$$

$n \times n$ is the size of the mask

25.1 The median filter, step by step

Your slide gives exactly two steps:

1. All the pixels in the neighbourhood identified by the mask are **sorted**, ascending or descending — it makes no difference.
2. The **median** of the sorted list is taken, and that becomes the pixel value in the processed image.

One detail that is easy to lose a mark on: **the pixel's own original value is included** in the sort. You are ranking all nine values of a 3×3 window, not the eight neighbours.

Worked example from the lecture

Take the 3×3 window

$$\begin{vmatrix} 1 & 5 & 7 \\ 2 & 4 & 6 \\ 3 & 2 & 1 \end{vmatrix} \rightarrow \text{sort} \rightarrow 1, 1, 2, 2, 4, 5, 6, 7$$

There are nine values: 1, 1, 2, 2, 3, 4, 5, 6, 7. The fifth of nine is the middle one, so the median is **3** and the centre pixel's value 4 is replaced by 3.

FROM YOUR LECTURE SLIDES

18	22	33	25	32	24
34	128	24	172	26	23
22	19	32	31	28	26

compute the median value of the marked pixels as shown in above fig. using 3x3 mask

The exercise your lecturer wrote on the slide: compute the median for each circled pixel using a 3x3 mask. Do it on paper — it is the fastest kind of question to get right and the easiest to rush.

25.2 Why the median beats the average for salt-and-pepper noise

IN SIMPLE WORDS

Impulse noise — also called **salt-and-pepper noise** — is a scattering of pixels stuck at 0 (pepper) or 255 (salt). They are wildly wrong values.

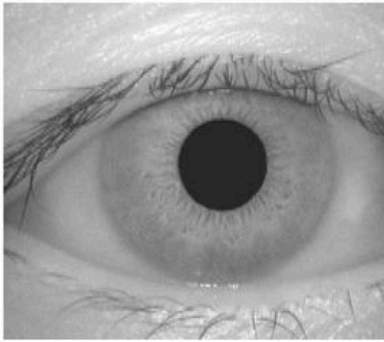
An *averaging* filter takes a 255 and spreads it across all nine output pixels. One bad pixel becomes nine slightly bad ones — a gray smudge.

A *median* filter sorts the nine values and takes the middle one. A 255 sits at the end of the sorted list, so it is **never chosen**. The noise pixel is not softened, it is **thrown away entirely** and replaced by a genuine neighbouring value.

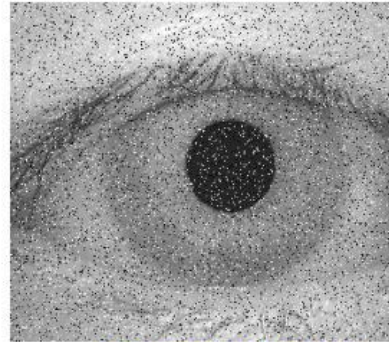
Your slide puts it as: the median filter **forces points with distinct gray levels to be more like their neighbours**. Any pixel whose value is very different from those around it is eliminated — forced to take the median intensity of its neighbours.

FROM YOUR LECTURE SLIDES

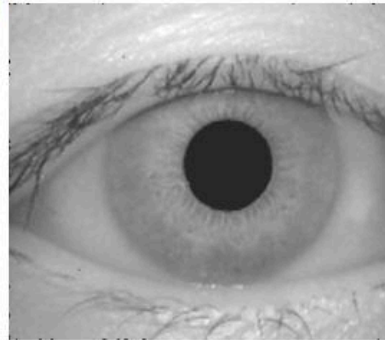
Median filter



Original Image



Corrupted Image

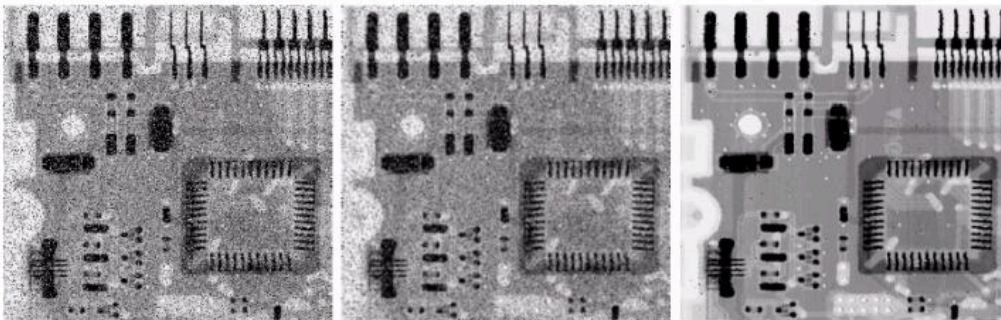


Median filtered Image

29

Original, corrupted, median-filtered. Almost every noise speck is gone and the eyelashes — genuine fine detail — are still there.

FROM YOUR LECTURE SLIDES



a b c

FIGURE 3.37 (a) X-ray image of circuit board corrupted by salt-and-pepper noise. (b) Noise reduction with a 3×3 averaging mask. (c) Noise reduction with a 3×3 median filter. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

30

Gonzalez & Woods Fig. 3.37. Centre: a 3×3 *average* — the noise is smeared into gray blobs and the whole board is blurred. Right: a 3×3 *median* — the noise is gone and the tracks are still sharp. This pair is the answer to "compare mean and median filtering".

FOR THE EXAM

Expect "what is the primary difference between an arithmetic mean filter and a median filter?" The answer in one sentence: **the mean filter is linear and blurs the noise into its neighbours, while the median filter is non-linear and removes the noise pixel outright, so for impulse noise it gives far better noise reduction with considerably less blurring at the same mask size.**

Chapter 25 at a glance

- Order-statistic filters rank the pixels; they are non-linear.
- Median filter: sort all $n \times n$ values (including the centre) and take the middle one.
- Outstanding against impulse / salt-and-pepper noise, with less blurring than averaging.
- For a 3×3 window the median is the 5th of the 9 sorted values.

CHAPTER 26 LECTURE 22 UNIT 2

Sharpening: derivatives of an image

Smoothing was averaging, which is a kind of *integration*. Sharpening is the opposite operation — **differentiation**. Your slide states the aim: to highlight fine detail, or to enhance detail that has been blurred, either by mistake or as a natural effect of how the image was acquired.

26.1 What a derivative must do in a digital image

Derivatives of a digital function are defined in terms of **differences**. There are several ways to define those differences, so the lecture first lays down what any sensible definition has to satisfy. Learn both lists — they are a favourite short question.

The requirements on a digital derivative

	FIRST DERIVATIVE	SECOND DERIVATIVE
In an area of constant intensity	must be zero	must be zero
At the onset of a step or ramp	must be non-zero	must be non-zero
At the end of a step or ramp	—	must be non-zero
Along a ramp of constant slope	must be non-zero	must be zero

That last row is the whole difference between the two. A ramp is a steady climb; the first derivative sees a steady non-zero slope all the way up it, while the second derivative sees no *change* in the slope and therefore reads zero except at the two ends.

26.2 The two definitions

First derivative: $\partial f / \partial x = f(x+1) - f(x)$

FROM YOUR LECTURE SLIDES

- Derivative of a digital pixel grid can be defined in terms of difference
- a basic definition of the first-order derivative of a one-dimensional function $f(x)$ is the difference

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

5

The first-order derivative: the next pixel minus this one.

Second derivative: $\partial^2 f / \partial x^2 = f(x+1) + f(x-1) - 2f(x)$

FROM YOUR LECTURE SLIDES

Second-order derivative

- similarly, we define the second-order derivative of a one-dimensional function $f(x)$ is the difference

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) - f(x) - [f(x+1-1) - f(x-1)]$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

7

The second-order derivative is the difference of two first differences, which collapses to $f(x+1) + f(x-1) - 2f(x)$. Memorise that line — it is the basis of every sharpening mask.

IN SIMPLE WORDS

The first derivative answers "*how fast is the brightness changing here?*" The second derivative answers "*is the rate of change itself changing?*" On a gentle ramp the answer to the first is "steadily", and to the second "no". That is why the second derivative ignores ramps and reacts only at their corners.

26.3 The scan-line example — do this one by hand

FROM YOUR LECTURE SLIDES

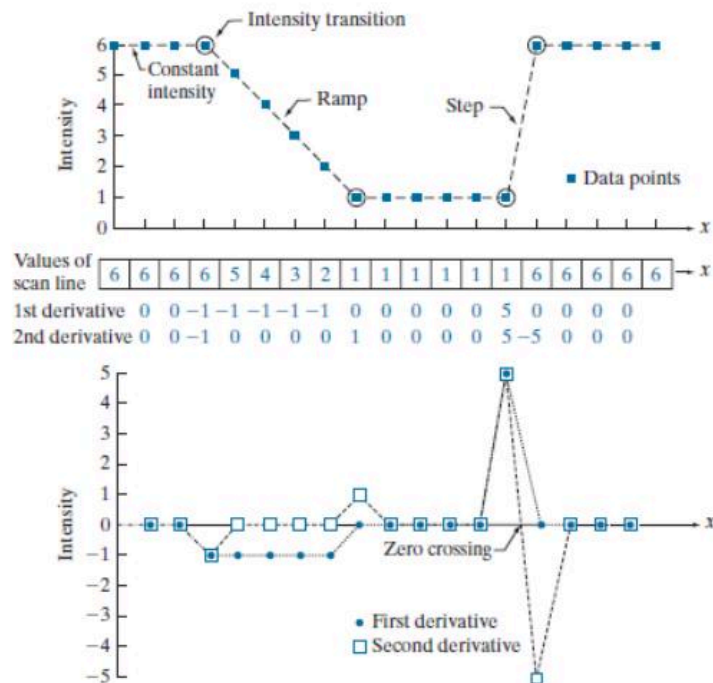
a
b
c

FIGURE 3.44

(a) A section of a horizontal scan line from an image, showing ramp and step edges, as well as constant segments.

(b) Values of the scan line and its derivatives.

(c) Plot of the derivatives, showing a zero crossing. In (a) and (c) points were joined by dashed lines as a visual aid.



10

Gonzalez & Woods Fig. 3.44: a horizontal gray-level profile through an image containing a ramp, an isolated point, a thin line and a step, with both derivatives plotted underneath.

Take the profile written along the bottom of that figure and work both rows out. The scan-line values are

6 6 6 6 5 4 3 2 1 1 1 1 1 1 6 6 6 6 6

First derivative, $f(x+1) - f(x)$, taken from the second value to the second-last so that both neighbours always exist:

0 0 -1 -1 -1 -1 -1 0 0 0 0 0 5 0 0 0 0

Second derivative, $f(x+1) + f(x-1) - 2f(x)$, over the same range:

0 -1 0 0 0 0 1 0 0 0 0 5 -5 0 0 0

THE COMMENTS THE QUESTION ASKS FOR

Constant regions. Both derivatives are 0 — correct for both.

The ramp (6→1). The first derivative holds a constant -1 all along it, so it produces a *thick* response covering the whole ramp. The second derivative is 0 in the middle of the ramp and non-zero only at the two ends, so it produces a *thin* response marking just the corners.

The step (1→6). The first derivative gives one spike of $+5$. The second gives a $+5$ immediately followed by a -5 — a double response of opposite sign. The point where it crosses between them is the **zero crossing**, and it locates the edge to the exact pixel.

Conclusion. First derivatives give thicker edges and respond more to gradual change; second derivatives give a stronger response to fine detail such as isolated points and thin lines, and their zero crossing pins an edge down precisely.

26.4 Why the second derivative is preferred for enhancement

Your slide gives three reasons:

- **Stronger response to fine detail** — isolated points and thin lines.
- **Simpler implementation** — one mask, no direction to worry about.
- The first-order derivative is not abandoned; it comes back in Chapter 29 as the gradient.

FOR THE EXAM

Derivative masks always have coefficients that **sum to zero**, so they give a response of 0 in an area of constant gray level. If you are asked to check whether a mask is a sharpening mask, add up its coefficients — zero means sharpening, one means smoothing.

Chapter 26 at a glance

- Sharpening = differentiation; the strength of the response is proportional to the degree of discontinuity.
- $\partial f / \partial x = f(x+1) - f(x)$; $\partial^2 f / \partial x^2 = f(x+1) + f(x-1) - 2f(x)$.
- First derivative: non-zero along ramps → thick edges. Second: zero along ramps → thin edges plus a zero crossing.
- Differentiation enhances edges and noise, and de-emphasises slowly varying areas.

CHAPTER 27 LECTURES 22 & 23 UNIT 2

The Laplacian

The first sharpening filter, and the one you are most likely to be asked to derive.

27.1 Isotropic — and why that matters

We want filters whose response is **independent of the direction** of the discontinuity. Such filters are called **isotropic**, and your slide gives the precise meaning: they are **rotation invariant** — rotating the image and then filtering gives the same answer as filtering and then rotating.

The simplest isotropic derivative operator is the **Laplacian**.

$$\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$$

FROM YOUR LECTURE SLIDES

The Laplacian

The Laplacian is defined as follows:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

where the partial 1st order derivative in the x direction is defined as follows:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

and in the y direction as follows:

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

The definition, with both partial derivatives written out underneath it.

27.2 Deriving the mask — the four-line proof

Write the two second derivatives from Chapter 26, one in each direction:

$$\begin{aligned}\partial^2 f / \partial x^2 &= f(x+1, y) + f(x-1, y) - 2f(x, y) \\ \partial^2 f / \partial y^2 &= f(x, y+1) + f(x, y-1) - 2f(x, y)\end{aligned}$$

Add them:

$$\nabla^2 f = [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] - 4f(x, y)$$

Read the coefficients straight off: **+1** on each of the four side neighbours, **-4** on the centre, 0 on the corners.

$$\text{Laplacian mask} = \begin{vmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{vmatrix} \quad \boxed{\text{SUMS TO } 0}$$

FROM YOUR LECTURE SLIDES

The Laplacian (cont...)

So, the Laplacian can be given as follows:

$$\begin{aligned} \nabla^2 f = & [f(x+1, y) + f(x-1, y) \\ & + f(x, y+1) + f(x, y-1)] \\ & - 4f(x, y) \end{aligned}$$

We can easily build a filter based on this

0	1	0
1	-4	1
0	1	0

The slide's own derivation, ending with the mask. If you can reproduce these four lines you can answer any Laplacian question.

The four masks in circulation

FROM YOUR LECTURE SLIDES

0	1	0	1	1	1
1	-4	1	1	-8	1
0	1	0	1	1	1

0	-1	0	-1	-1	-1
-1	4	-1	-1	8	-1
0	-1	0	-1	-1	-1

a b
c d

FIGURE 3.37

(a) Filter mask used to implement Eq. (3.6-6).
(b) Mask used to implement an extension of this equation that includes the diagonal terms.
(c) and (d) Two other implementations of the Laplacian found frequently in practice.

15

Gonzalez & Woods Fig. 3.37. (a) the basic mask. (b) the extension that also includes the diagonal terms, so the centre becomes -8 . (c) and (d) the same two with every sign reversed. All four are valid Laplacians; only the sign convention differs.

FOR THE EXAM — THE SIGN TRAP

Whether the centre is -4 or $+4$ decides which way you combine the result with the original image. Get this pair right and you cannot go wrong:

centre negative $\rightarrow g = f - \nabla^2 f$ centre positive $\rightarrow g = f + \nabla^2 f$

Both give the same sharpened picture. Say which convention you are using before you start.

27.3 The Laplacian alone is not an enhanced image

Apply the mask and you get a picture of edges on a dark, featureless background — because the operator has thrown away every area of slowly varying gray. Your slide is blunt about it:

"But that is not very enhanced!"

To get the sharpened image you must **put the edges back onto the original**:

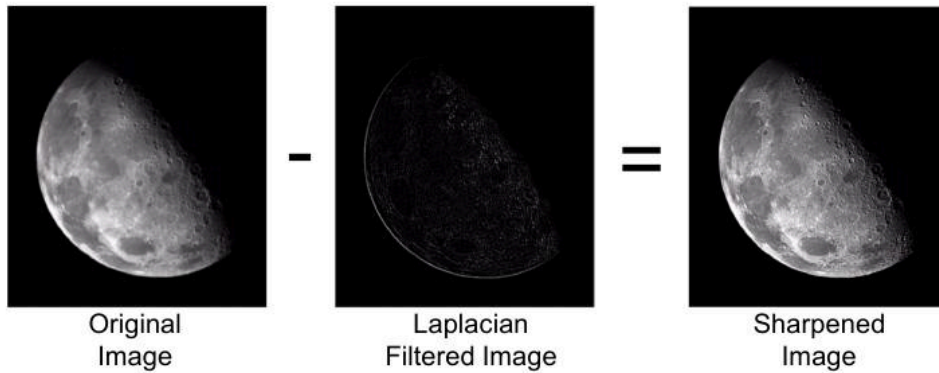
$$g(x, y) = f(x, y) + c \cdot \nabla^2 f(x, y)$$

$c = -1$ when the mask centre is negative, $c = +1$ when it is positive

FROM YOUR LECTURE SLIDES

Laplacian Image Enhancement

Images taken from Gonzalez & Woods, Digital Image Processing (2002)



In the final sharpened image edges and fine detail are much more obvious

Original – Laplacian = sharpened. In the final image the craters and the terminator are far more obvious, and the flat parts of the disc are unchanged.

27.4 The composite mask — doing it in one pass

Substituting the mask into $g = f - \nabla^2 f$ and collecting terms:

$$\begin{aligned}
 g(x, y) &= f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)] \\
 &= 5f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)]
 \end{aligned}$$

$$\text{composite mask} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad \boxed{\text{SUMS TO 1}}$$

FROM YOUR LECTURE SLIDES

Original-Mask of Laplacian

$$\begin{aligned}
 g(x, y) &= f(x, y) - [f(x+1, y) + f(x-1, y) \\
 &\quad + f(x, y+1) + f(x, y-1) - 4f(x, y)] \\
 &= 5f(x, y) - [f(x+1, y) + f(x-1, y) \\
 &\quad + f(x, y+1) + f(x, y-1)]
 \end{aligned}$$

0	-1	0
-1	5	-1
0	-1	0

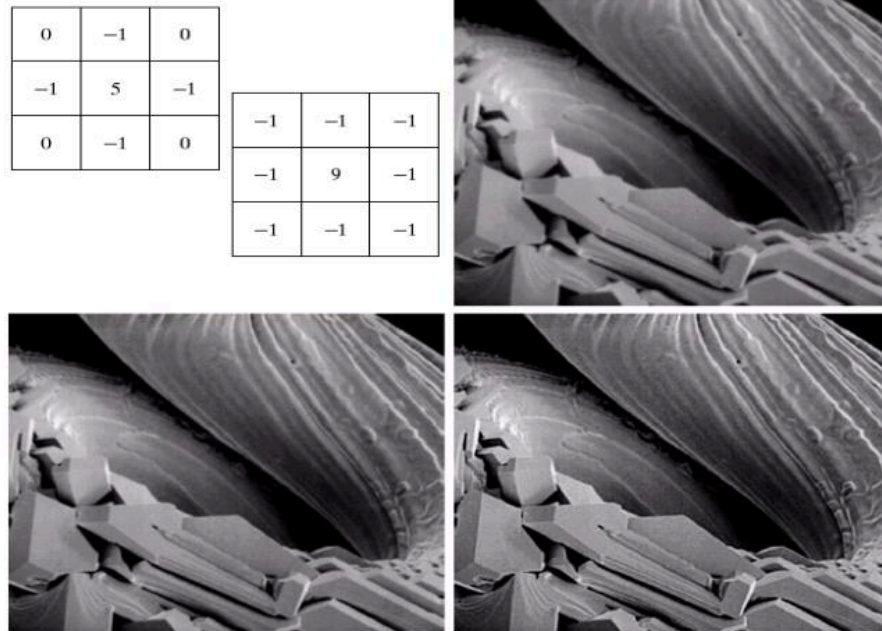
2

The composite mask. One pass over the image now does both the differentiation and the addition back onto the original.

IN SIMPLE WORDS – WHY THE CENTRE IS 5

The coefficients now add to **1**, not 0 — because this mask is no longer a pure derivative. It is "the original image, *plus* its edges", so it must preserve the average brightness. The 5 is the 1 from the original plus the 4 from the Laplacian's centre.

FROM YOUR LECTURE SLIDES



a b c
d e

FIGURE 3.41 (a) Composite Laplacian mask. (b) A second composite mask. (c) Scanning electron microscope image. (d) and (e) Results of filtering with the masks in (a) and (b), respectively. Note how much sharper (e) is than (d). (Original image courtesy of Mr. Michael Shaffer, Department of Geological Sciences, University of Oregon, Eugene.)

3

Gonzalez & Woods Fig. 3.41. Left column: the composite mask with 5 in the centre, and the version with 9 in the centre that also uses the diagonals. Notice how much sharper the 9-centre result is.

Chapter 27 at a glance

- Laplacian $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$ — isotropic, rotation invariant.
- Mask $[0 \ 1 \ 0; 1 \ -4 \ 1; 0 \ 1 \ 0]$, or $[1 \ 1 \ 1; 1 \ -8 \ 1; 1 \ 1 \ 1]$ with the diagonals.
- Laplacian alone = edges on a dark background, not an enhanced image.
- Sharpened $g = f \pm \nabla^2 f$; composite mask $[0 \ -1 \ 0; -1 \ 5 \ -1; 0 \ -1 \ 0]$ does it in one pass.

CHAPTER 28 LECTURE 23 UNIT 2

High-boost filtering

The composite Laplacian mask throws the original image in once. High-boost filtering asks a simple question: *what if we threw it in more than once?*

28.1 The idea

Also called the **high frequency emphasis filter**. Its purpose, from your slide: to **retain some of the low frequency components to aid in the interpretation of the image**. A pure sharpening filter kills the flat regions; high-boost keeps enough of them for the picture to still look like a picture.

$$\begin{aligned}
 \text{High boost} &= A \cdot f(x,y) - \text{low pass} \\
 &= (A - 1) f(x,y) + f(x,y) - \text{low pass} \\
 &= (A - 1) f(x,y) + \text{High pass}
 \end{aligned}$$

IN SIMPLE WORDS — FOLLOWING THE THREE LINES

Line 1 is the definition: take A copies of the original and subtract the blurred version.

Line 2 splits the A copies into "one copy" plus " $A-1$ spare copies".

Line 3 recognises that *original minus low-pass is exactly the high-pass*. So high boost = the sharpened image, **plus $A-1$ extra helpings of the original**. Turning A up fades the picture back in behind the edges.

28.2 The masks

FROM YOUR LECTURE SLIDES

0	-1	0	-1	-1	-1
-1	$A + 4$	-1	-1	$A + 8$	-1
0	-1	0	-1	-1	-1

- $A \geq 1$
- if $A = 1$, it becomes “standard” Laplacian sharpening⁶

The two high-boost masks. Left: the four-neighbour version with $A+4$ at the centre. Right: the eight-neighbour version with $A+8$.

$$\begin{vmatrix} 0 & -1 & 0 \\ -1 & A+4 & -1 \\ 0 & -1 & 0 \end{vmatrix} \quad \begin{vmatrix} -1 & -1 & -1 \\ -1 & A+8 & -1 \\ -1 & -1 & -1 \end{vmatrix}$$

The condition is $A \geq 1$, and your slide gives the check that proves you understand it:

FOR THE EXAM

If $A = 1$ the mask becomes the standard Laplacian sharpening mask. Put $A = 1$ into $A+4$ and you get 5 — the composite mask from Chapter 27. So high-boost is not a new filter, it is the Laplacian sharpener with a dial on it, and $A = 1$ is the setting where the dial does nothing.

As A grows the mask tends towards a plain multiple of the original image, so the sharpening effect gets weaker relative to the picture. Small $A \rightarrow$ strong edges, dark flats. Large $A \rightarrow$ nearly the original, gently sharpened.

Chapter 28 at a glance

- High boost = $A f - \text{low pass} = (A-1) f + \text{high pass}$, with $A \geq 1$.
- Masks: centre $A+4$ (four neighbours) or $A+8$ (eight neighbours), all others -1 .
- $A = 1$ gives back the ordinary Laplacian sharpening mask.
- It exists to keep some low-frequency content so the sharpened image is still readable.

CHAPTER 29 LECTURE 23 UNIT 2

The gradient: Roberts, Prewitt and Sobel

Chapter 26 set the first derivative aside. Here it comes back, because for **edge detection** it is what everybody actually uses.

29.1 The gradient vector

For every pixel, the change in gray value with respect to distance in x and in y is collected into one vector:

$$\nabla f = \begin{bmatrix} G_x & G_y \end{bmatrix}^T = \begin{bmatrix} \partial f / \partial x & \partial f / \partial y \end{bmatrix}^T$$

FROM YOUR LECTURE SLIDES

First derivatives in image processing are implemented using the magnitude of the gradient. For a function $f(x, y)$, the gradient of f at coordinates (x, y) is defined as the two-dimensional column *vector*

- Gradient is a 2-D vector that points to the direction in which the image intensity grows fastest

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

18

The gradient is a 2-D vector that points in the direction in which the image intensity grows fastest.

29.2 Magnitude, and the approximation everybody uses

Exact: $M(x, y) = \text{mag}(\nabla f) = [G_x^2 + G_y^2]^{1/2}$

Approx: $M(x, y) \approx |G_x| + |G_y|$

FROM YOUR LECTURE SLIDES

$$\begin{aligned} \nabla f = \text{mag}(\nabla f) &= [G_x^2 + G_y^2]^{1/2} \\ &= \left[\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2 \right]^{1/2} \end{aligned}$$

commonly approx.

$$M(x, y) = \nabla f \approx |G_x| + |G_y|$$

20

The square root is replaced by a sum of absolute values. It is much cheaper to compute and close enough for edge detection.

WORTH KNOWING — A SUBTLETY EXAMINERS LIKE

The *components* G_x and G_y are derivatives, so they are **linear** operators — but the **magnitude is not**, because of the squaring and the square root.

The other way round for rotation: the individual partial derivatives are **not** isotropic, but the **magnitude of the gradient vector is**. That is why we always work with the magnitude.

And the two useful readings: the **magnitude** gives the strength of the edge; the **orientation** gives the direction of greatest change, which is the direction across the edge.

29.3 The masks, smallest first

Label the 3×3 region the way the lecture does:

$$\begin{array}{|c|c|c|} \hline z_1 & z_2 & z_3 \\ \hline z_4 & z_5 & z_6 \\ \hline z_7 & z_8 & z_9 \\ \hline \end{array} \quad z_5 \text{ is the pixel being processed}$$

Simplest 2×2 approximation

$$G_x = (z_8 - z_5) \quad \text{and} \quad G_y = (z_6 - z_5)$$

$$|\nabla f| \approx |z_8 - z_5| + |z_6 - z_5|$$

FROM YOUR LECTURE SLIDES

Gradient Mask

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

- simplest approximation, 2x2

$$G_x = (z_8 - z_5) \quad \text{and} \quad G_y = (z_6 - z_5)$$

$$\nabla f = [G_x^2 + G_y^2]^{1/2} = [(z_8 - z_5)^2 + (z_6 - z_5)^2]^{1/2}$$

$$\nabla f \approx |z_8 - z_5| + |z_6 - z_5|$$

22

The plain difference: straight down minus centre, and across minus centre.

Roberts cross-gradient, 2x2

$$G_x = (z_9 - z_5) \quad \text{and} \quad G_y = (z_8 - z_6)$$

$$\nabla f \approx |z_9 - z_5| + |z_8 - z_6|$$

$$\begin{vmatrix} -1 & 0 \\ 0 & 1 \end{vmatrix} \quad \begin{vmatrix} 0 & -1 \\ 1 & 0 \end{vmatrix} \quad \text{the two Roberts masks — note they work on the } \textit{diagonals}$$

FROM YOUR LECTURE SLIDES

Gradient Mask

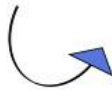
z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

- Roberts cross-gradient operators, 2x2

$$G_x = (z_9 - z_5) \quad \text{and} \quad G_y = (z_8 - z_6)$$

$$\nabla f = [G_x^2 + G_y^2]^{1/2} = [(z_9 - z_5)^2 + (z_8 - z_6)^2]^{1/2}$$

$$\nabla f \approx |z_9 - z_5| + |z_8 - z_6|$$



-1	0	0	-1
0	1	1	0

Roberts. The differences are taken across the two diagonals, which is where "cross-gradient" comes from.

Prewitt, 3x3

$$G_x = \begin{vmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{vmatrix} \quad G_y = \begin{vmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{vmatrix}$$

$$G_x = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$G_y = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

FROM YOUR LECTURE SLIDES

Prewitt operator

a) Prewitt Mask (3×3 Mask)

$$\begin{array}{ccc} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{array} \quad \begin{array}{ccc} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{array}$$

If the 3×3 region of the image is represented as $\begin{pmatrix} Z_1 & Z_2 & Z_3 \\ Z_4 & Z_5 & Z_6 \\ Z_7 & Z_8 & Z_9 \end{pmatrix}$, using Prewitt

mask, magnitude of partial derivative at the position labelled Z_5 is computed as

$$abs[(Z_7 + Z_8 + Z_9) - (Z_1 + Z_2 + Z_3)] + abs[(Z_3 + Z_6 + Z_9) - (Z_1 + Z_4 + Z_7)] \quad 27$$

Prewitt: bottom row minus top row, and right column minus left column. Every coefficient is ± 1 .

Sobel, 3×3

$$G_x = \begin{vmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{vmatrix} \quad G_y = \begin{vmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{vmatrix}$$

$$G_x = (Z_7 + 2Z_8 + Z_9) - (Z_1 + 2Z_2 + Z_3)$$

$$G_y = (Z_3 + 2Z_6 + Z_9) - (Z_1 + 2Z_4 + Z_7)$$

FROM YOUR LECTURE SLIDES

Gradient Mask

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

- Sobel operators, 3x3

$$G_x = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$G_y = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

$$\nabla f \approx |G_x| + |G_y|$$

the weight value 2 is to achieve smoothing by giving more important to the center point

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

24

Sobel is Prewitt with a 2 on the middle coefficient of each row or column. Your slide says exactly why: **the weight value 2 is to achieve smoothing by giving more importance to the centre point**. Sobel is therefore a bit less noise-sensitive than Prewitt.

FOR THE EXAM – THE FREE CHECK

The coefficients of every one of these masks add up to 0, which means they give a response of 0 in an area of constant gray level. Add up any gradient mask you are asked to write down. If it does not come to zero, you have copied it wrong.

The three named operators compared

OPERATOR	SIZE	COEFFICIENTS	CHARACTER
Roberts	2×2	±1 on a diagonal	Cheapest and fastest; very sensitive to noise; no centre pixel, so the edge is offset by half a pixel.
Prewitt	3×3	all ±1	Simple to compute by hand; averages three rows so it is steadier than Roberts.
Sobel	3×3	±1 and ±2	Extra weight on the centre gives some smoothing as well as differentiating; the usual choice in practice.

29.4 The lecture's worked example

The slides work one 8×8 image right through with all three operators. It is the model answer for "perform image sharpening using Prewitt, Roberts and Sobel".

FROM YOUR LECTURE SLIDES

Example.

Let us consider the matrix 'IMAGE' as mentioned below to perform image sharpening using Prewitt, Robert and sobel operator.

$$\text{IMAGE} = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 4 & 1 & 8 & 3 & 2 & 3 \\ 3 & 4 & 6 & 3 & 1 & 7 & 5 & 7 \\ 2 & 5 & 0 & 4 & 2 & 7 & 5 & 8 \\ 6 & 2 & 5 & 1 & 8 & 2 & 1 & 5 \\ 4 & 0 & 9 & 2 & 4 & 3 & 1 & 9 \\ 8 & 9 & 9 & 0 & 0 & 3 & 1 & 6 \\ 0 & 0 & 1 & 7 & 0 & 3 & 9 & 3 \\ 3 & 2 & 0 & 4 & 0 & 9 & 6 & 4 \end{pmatrix}$$

The image the example starts from.

The method for one pixel, using Prewitt on the 3×3 window sitting at that position:

1. Multiply the window by the G_x mask and add → one number.
2. Multiply the same window by the G_y mask and add → a second number.
3. Take $|G_x| + |G_y|$. That is the output pixel.

The slide's first result works out as $\text{abs}(0+3+4) + \text{abs}(0+0+4) = 7 + 4 = 11$, and the same three steps are then repeated for every position.

FROM YOUR LECTURE SLIDES

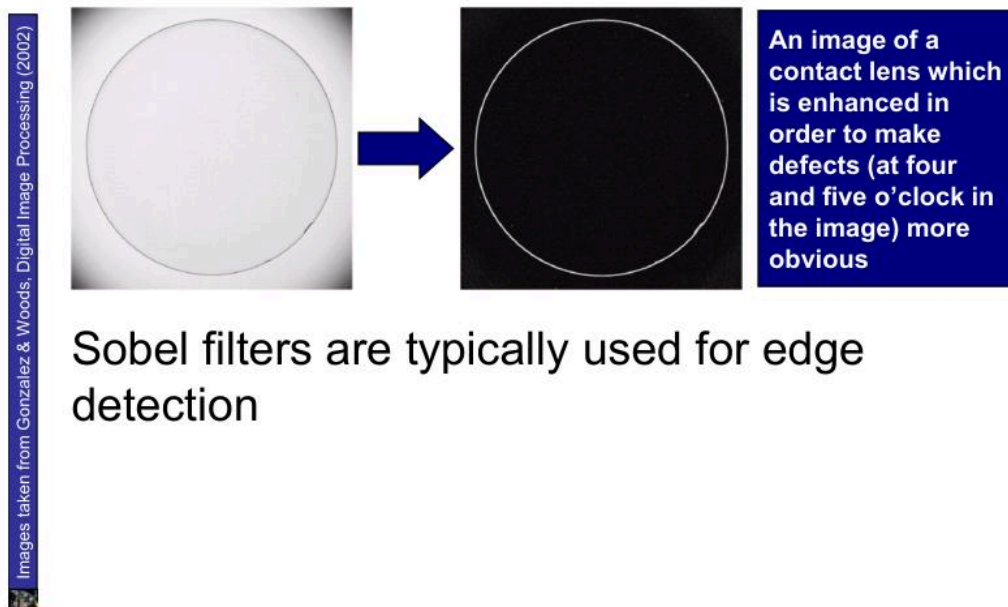
$$\text{IMAGE}(\text{sharp}(\text{SM})) = \begin{pmatrix} 14 & 26 & 20 & 16 & 20 & 28 & 24 & 28 \\ 20 & 14 & 2 & 8 & 18 & 10 & 16 & 30 \\ 20 & 4 & 10 & 4 & 18 & 10 & 20 & 24 \\ 10 & 2 & 12 & 10 & 8 & 24 & 24 & 10 \\ 22 & 30 & 20 & 22 & 22 & 18 & 20 & 6 \\ 26 & 20 & 20 & 26 & 6 & 12 & 22 & 16 \\ 28 & 28 & 28 & 12 & 10 & 42 & 16 & 26 \\ 4 & 6 & 20 & 16 & 16 & 36 & 34 & 36 \end{pmatrix}$$

34

The finished IMAGE(sharp) matrix for the Sobel mask. Work two or three entries of this yourself and check them against the slide before the exam.

FROM YOUR LECTURE SLIDES

Sobel Example



Sobel filters are typically used for edge detection

Why anyone bothers: a Sobel filter on an optical contact lens, where the defects at four and five o'clock become obvious only after the edges are pulled out.

Chapter 29 at a glance

- Gradient $\nabla f = [G_x, G_y]$, magnitude $[G_x^2 + G_y^2]^{1/2} \approx |G_x| + |G_y|$.
- Components are linear but the magnitude is not; the magnitude is isotropic but the components are not.

- Roberts 2×2 diagonal · Prewitt 3×3 all ± 1 · Sobel 3×3 with 2 in the middle for smoothing.
- All gradient mask coefficients sum to zero.

CHAPTER 30 LECTURE 23 UNIT 2

Combining the enhancement methods

The closing idea of the whole enhancement unit, and a favourite descriptive question. Your slide states it plainly: **successful image enhancement is typically not achieved using a single operation** — you combine a range of techniques to get the final result.

30.1 The three-step recipe

FROM YOUR LECTURE SLIDES

Example of Combining Spatial Enhancement Methods

- solve :
 1. Laplacian to highlight fine detail
 2. gradient to enhance prominent edges
 3. gray-level transformation to increase the dynamic range of gray levels

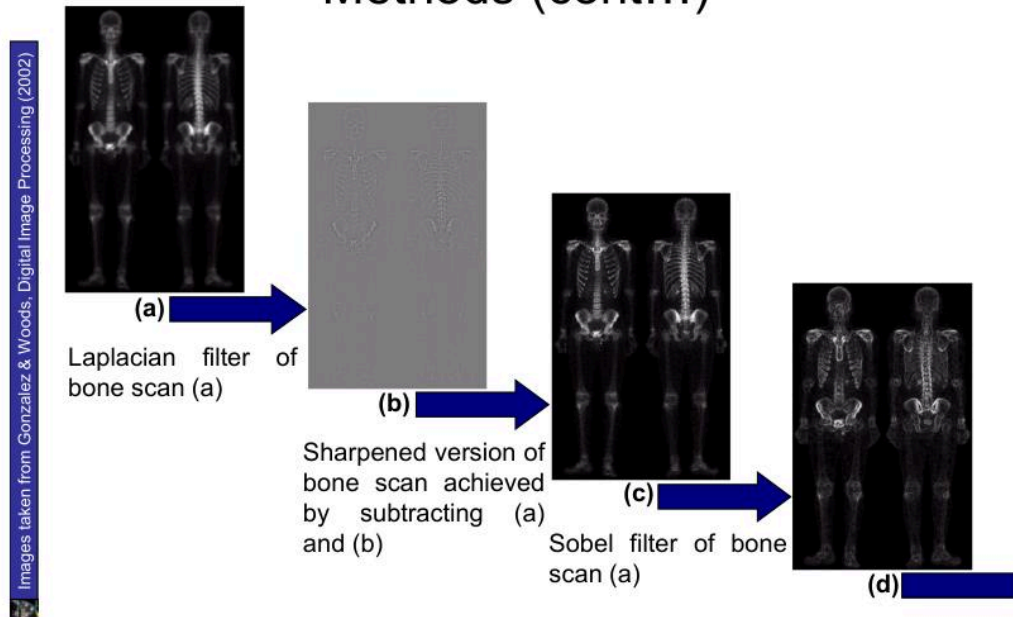
The plan for the whole-body bone scan.

1. **Laplacian** to highlight fine detail.
2. **Gradient** to enhance the prominent edges.
3. **Gray-level transformation** to increase the dynamic range of gray levels.

30.2 The eight steps on the bone scan

FROM YOUR LECTURE SLIDES

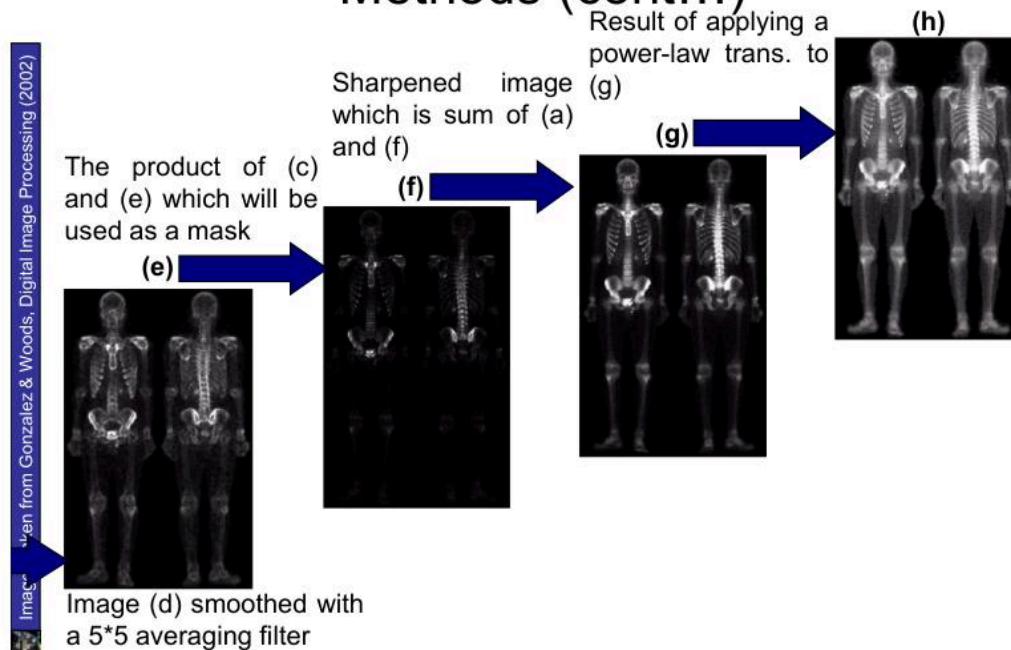
Combining Spatial Enhancement Methods (cont...)



(a) the original bone scan. (b) its Laplacian. (c) the sharpened version, (a) minus (b). (d) the Sobel gradient of (a).

FROM YOUR LECTURE SLIDES

Combining Spatial Enhancement Methods (cont...)



(e) (d) smoothed with a 5×5 averaging filter. (f) the product of (c) and (e), used as a mask. (g) the sharpened image, (a) plus (f). (h) the result of a power-law transformation applied to (g).

IN SIMPLE WORDS – WHY EACH STEP IS THERE

The Laplacian (b) finds the fine detail but is full of noise, because a second derivative amplifies noise badly.

The Sobel (d) finds strong edges and responds much less to noise, but it is thick and crude.

Smoothing the Sobel (e) turns it into a smooth *map of where the interesting structure is* — bright where there are edges, dark in the flat background.

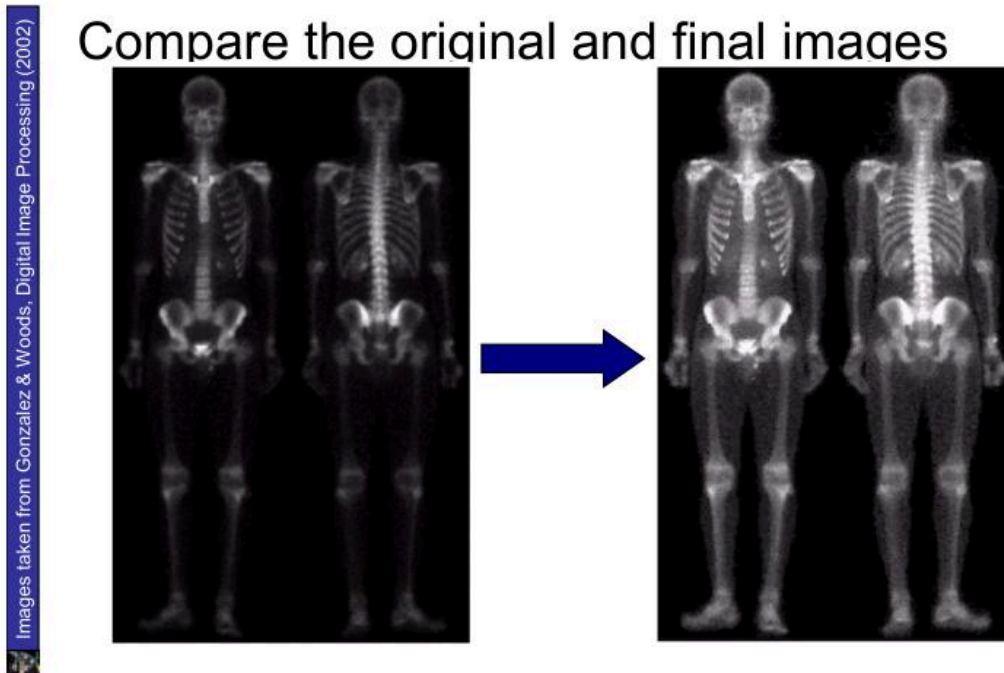
The product (f) is the clever step. Multiplying the noisy sharpened image by that map keeps the detail where there are real edges and suppresses it where there is only background noise. That is why it is described as a mask.

The power law (h) finally opens up the dark end of the scale so the faint structure becomes visible.

FROM YOUR LECTURE SLIDES

Combining Spatial Enhancement Methods (cont...)

Compare the original and final images



Original and final, side by side. Every technique in Chapters 16 to 29 is in that one arrow.

FOR THE EXAM

If you are asked to "explain how spatial enhancement methods can be combined", give the three-step recipe and then the bone scan as the example, naming the eight steps (a) to (h). The one sentence that earns the mark is: **the smoothed gradient is used as a mask, multiplied into the Laplacian-sharpened image, so that detail is kept where there are genuine edges and suppressed where there is only noise.**

Chapter 30 at a glance

- No single operation enhances a difficult image; you combine several.
- Laplacian → fine detail (noisy). Sobel → strong edges (robust). Smoothed Sobel → a mask.
- Multiply the mask into the sharpened image, add back to the original, then stretch with a power law.

CHAPTER 31 EXAM KIT

Formula sheet

Everything you might need to write down, in one place. If you learn nothing else the night before, learn this table.

WHAT	FORMULA	REMEMBER
Digital image	$f(x, y)$	x = row, y = column, origin top-left
Image formation	$f(x, y) = i(x, y) \cdot r(x, y)$	$0 < i < \infty \cdot 0 < r < 1$
Gray levels	$L = 2^k$	k = bits per pixel; values lie in $[0, L-1]$
Storage	$b = M \times N \times k$	answer in bits; $\div 8$ for bytes
Pixel reduction	$(\text{old} - \text{new}) / \text{old} \times 100 \%$	halving the side removes 75 % of pixels
4-neighbours	$N_4(p): (x \pm 1, y), (x, y \pm 1)$	share a side
Diagonal neighbours	$N_D(p): (x \pm 1, y \pm 1)$	share a corner
8-neighbours	$N_8(p) = N_4(p) \cup N_D(p)$	all eight
m-adjacency	$q \in N_4(p)$, or $q \in N_D(p)$ and $N_4(p) \cap N_4(q)$ has no pixel from V	removes the multiple-path ambiguity of 8-adjacency
Euclidean distance	$D_e = [(x-s)^2 + (y-t)^2]^{1/2}$	a disk
City-block	$D_4 = x-s + y-t $	a diamond
Chessboard	$D_8 = \max(x-s , y-t)$	a square
Distance order	$D_8 \leq D_e \leq D_4$	always true
Metric conditions	$D \geq 0 \cdot D(p, q) = D(q, p) \cdot D(p, z) \leq D(p, q) + D(q, z)$	all three needed
Linear operator	$H[a_i f_i + a_j f_j] = a_i H[f_i] + a_j H[f_j]$	additivity + homogeneity
2D convolution	$y(n_1, n_2) = \sum \sum x(k_1, k_2) h(n_1 - k_1, n_2 - k_2)$	product of x at (a, b) and h at (c, d) lands at $(a+c, b+d)$
Output size	$(M_1 + N_1 - 1) \times (M_2 + N_2 - 1)$	add and subtract 1, per direction
2D Z-transform	$X(z_1, z_2) = \sum \sum x(n_1, n_2) z_1^{-n_1} z_2^{-n_2}$	value $\rightarrow$ coefficient, position $\rightarrow$ powers
Convolution property	$y = x ** h \Leftrightarrow Y = X \cdot H$	convolution $\leftrightarrow$ multiplication

WHAT	FORMULA	REMEMBER
Frequency response	put $z_1 = e^{j\omega_1}$, $z_2 = e^{j\omega_2}$	$e^{j\theta} + e^{-j\theta} = 2\cos \theta$
Toeplitz matrix A	rows = number of output pixels, columns = number of kernel pixels	blocks from x, h as a column read row by row
Convolution checks	$y(0,0) = x(0,0)h(0,0) \cdot \Sigma y = (\Sigma x)(\Sigma h)$	use both, every time
Spatial enhancement	$g(x,y) = T[f(x,y)]$	1×1 neighbourhood $\rightarrow$ point processing, $s = T(r)$
Brightness / contrast	$g = f \pm c \cdot g = f \times c$	add shifts the histogram, multiply stretches it
Negative	$s = L - 1 - r$	white detail inside large dark regions
Log	$s = c \log(1 + r)$	compresses dynamic range; expands dark values
Power law	$s = c r^\gamma$	$\gamma < 1$ brightens $\cdot \gamma > 1$ darkens
Gamma correction	$s = c r^{1/\gamma}$	CRT γ is 1.8 to 2.5
Bit plane weight	plane n contributes 2^{n-1}	plane 8 $\rightarrow$ 128, plane 1 $\rightarrow$ 1
Histogram	$h(r_k) = n_k \cdot p(r_k) = n_k/MN$	the normalized one sums to 1
Equalization, continuous	$s = T(r) = (L-1) \int_0^r p_r(w) dw$	the CDF; gives $p_s(s) = 1/(L-1)$, uniform
Equalization, discrete	$s_k = ((L-1)/MN) \sum_{j=0}^k n_j$	round to the nearest integer
Matching	$G(z_q) = (L-1) \sum p_z(z_i) = s_k, z_q = G^{-1}(s_k)$	tabulate G, then look for the closest value
Mean and variance	$m = \sum r_i p(r_i) \cdot \sigma^2 = \sum (r_i - m)^2 p(r_i)$	brightness and contrast, as two numbers
Mask response	$R = \sum_i w_i z_i$	multiply the mask by what is underneath, then add
Spatial filtering	$g(x,y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s,t) f(x+s, y+t)$	$a = (m-1)/2, b = (n-1)/2$

WHAT	FORMULA	REMEMBER
Weighted smoothing	$g = \sum \sum w f \div \sum \sum w$	divide by the sum of the coefficients
Median filter	$R = \text{median}\{z_k\}$	sort all $n \times n$ values, take the middle one
First derivative	$\partial f / \partial x = f(x+1) - f(x)$	non-zero along a ramp $\rightarrow$ thick edges
Second derivative	$\partial^2 f / \partial x^2 = f(x+1) + f(x-1) - 2f(x)$	zero along a ramp $\rightarrow$ thin edges + zero crossing
Laplacian	$\nabla^2 f = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$	mask $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$
Laplacian sharpening	$g = f - \nabla^2 f$ (centre negative) · $g = f + \nabla^2 f$ (centre positive)	composite mask $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$
High boost	$(A-1)f + \text{high pass}$ · centre = $A+4$ or $A+8$	$A \geq 1$; $A = 1$ is plain Laplacian sharpening
Gradient magnitude	$M = [G_x^2 + G_y^2]^{1/2} \approx G_x + G_y $	Roberts 2×2 · Prewitt ± 1 · Sobel $\pm 1, \pm 2$
Mask coefficient sum	smoothing $\rightarrow 1$ · derivative $\rightarrow 0$	the fastest way to tell the two families apart
Adaptive enhancement	$g = C \cdot f$ if $m_{sxy} \leq k_0 m_G$ and $k_1 \sigma_G \leq \sigma_{sxy} \leq k_2 \sigma_G$	brighten only what is dark <i>and</i> faint

CHAPTER 32 EXAM KIT

Solved problems

Work through each one on paper first, then open the solution. These are written in the style your handouts and syllabus point to, and they cover every chapter above.

P1. An image of size 2048×2048 has 64 gray levels. Find the number of bits and the number of bytes needed to store it.

$64 = 2^6$, so $k = 6$ bits per pixel.

$b = M \times N \times k = 2048 \times 2048 \times 6 = 4,194,304 \times 6 = 25,165,824$ bits.

Bytes = $25,165,824 \div 8 = 3,145,728$ bytes = 3072 KB = 3 MB.

P2. An image is reduced from 1024×1024 to 256×256 . What percentage of the pixels is discarded? Comment on what is lost visually.

Old count = $1024 \times 1024 = 1,048,576$. New count = $256 \times 256 = 65,536$.

Discarded = $1,048,576 - 65,536 = 983,040$.

Percentage = $983,040 / 1,048,576 \times 100 = 93.75 \%$.

Visually: the side is divided by 4 but the pixel count by 16. Fine detail and texture go first, then edges begin to look like staircases, and the checkerboard effect (visible square pixels) appears. Brightness and contrast are untouched — only spatial detail is lost, because only sampling was changed.

P3. A surface has reflectance 0.8 and the illumination on it is 500 lm/m^2 . What is f ? What are the smallest and largest possible values of f under this illumination?

$f = i \cdot r = 500 \times 0.8 = 400 \text{ lm/m}^2$.

Reflectance is bounded by $0 < r < 1$, so with i fixed at 500 the value of f must lie between just above 0 (total absorption, like black velvet at $r = 0.01$ giving 5) and just below 500 (total reflection). f can never exceed the illumination, because a surface cannot return more light than it received.

P4. For the image segment below with $V = \{1\}$, list all 4-adjacent, 8-adjacent and m-adjacent pairs.

```
0 1 1
0 1 0
0 0 1
```

The pixels with value 1 are at (0,1), (0,2), (1,1) and (2,2), writing (row, column) from 0.

4-adjacent (must share a side): (0,1)–(0,2) and (0,1)–(1,1). *Two pairs.*

8-adjacent (side or corner): the two above, plus (0,2)–(1,1) and (1,1)–(2,2). *Four pairs.*

m-adjacent: start from the 8-adjacent list and test each diagonal pair.

- (0,2) and (1,1): $N_4(0,2) \cap N_4(1,1) = \{(0,1), (1,2)\}$. The value at (0,1) is 1, which is in $V \rightarrow$ **not** m-adjacent.
- (1,1) and (2,2): $N_4(1,1) \cap N_4(2,2) = \{(1,2), (2,1)\}$. Both are 0, none in $V \rightarrow$ m-adjacent. ✓

So the m-adjacent pairs are (0,1)–(0,2), (0,1)–(1,1) and (1,1)–(2,2). *Three pairs.*

P5. For the same segment, find the shortest 4-path, 8-path and m-path from (0,2) to (2,2), and say how many different paths exist in each case.

4-path: none exists. From (0,2) you can only reach (0,1) and then (1,1); from (1,1) the side neighbours (1,2) and (2,1) are both 0, so (2,2) can never be reached. The two pixels are not 4-connected.

8-path: $(0,2) \rightarrow (1,1) \rightarrow (2,2)$, length 2. But $(0,2) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,2)$ is also a valid 8-path, so **two** different paths exist and "the path" is ambiguous.

m-path: $(0,2) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,2)$, length 3, and it is the **only** one, because (0,2)–(1,1) is not m-adjacent. This is precisely the purpose of m-adjacency.

P6. For $p = (2, 1)$ and $q = (6, 4)$, compute D_e , D_4 and D_8 , and verify the standard inequality. Then find how many pixels satisfy $D_4 \leq 2$ and how many satisfy $D_8 \leq 2$ from a given centre.

$$|x - s| = |2 - 6| = 4 \text{ and } |y - t| = |1 - 4| = 3.$$

$$D_e = \sqrt{(4^2 + 3^2)} = \sqrt{25} = 5.0$$

$$D_4 = 4 + 3 = 7$$

$$D_8 = \max(4, 3) = 4$$

Check: $4 \leq 5 \leq 7$, so $D_8 \leq D_e \leq D_4$ holds. ✓

$D_4 \leq 2$: a diamond. 1 pixel at distance 0, 4 at distance 1, 8 at distance 2 → **13 pixels**.

$D_8 \leq 2$: a square of side $2 \times 2 + 1 = 5$, so $5 \times 5 = 25$ pixels.

P7. Show that the operator "take the maximum value in the neighbourhood" is nonlinear.

An operator is linear only if it satisfies additivity and homogeneity. One counter-example is enough to disprove it.

Let $H = \max$, and take two one-row images $f_1 = [0, 2]$ and $f_2 = [3, 0]$, with $a_i = a_j = 1$.

$$\text{Left side: } H[f_1 + f_2] = \max([3, 2]) = 3.$$

$$\text{Right side: } H[f_1] + H[f_2] = 2 + 3 = 5.$$

$3 \neq 5$, so additivity fails and the max operator is **nonlinear**. (Summing images, by contrast, satisfies both properties and is linear.)

P8. Perform the linear convolution of $x = [[4, 5, 6], [7, 8, 9]]$ with $h = [1, 1, 1]^T$ using the graphical method.

Size: x is 2×3 , h is $3 \times 1 \rightarrow y$ is $(2+3-1) \times (3+1-1) = 4 \times 3$.

Rule: x at (a,b) times h at (c,d) contributes to y at (a+c, b+d). Since h has three 1s at (0,0), (1,0) and (2,0), every value of x is simply copied into three rows below itself.

Column $n = 0$ receives 4 (from row 0) and 7 (from row 1):

$$y(0,0) = 4 \cdot y(1,0) = 4+7 = 11 \cdot y(2,0) = 4+7 = 11 \cdot y(3,0) = 7.$$

Doing the same for the other two columns:

$$y = \begin{bmatrix} 4 & 5 & 6 \\ 11 & 13 & 15 \\ 11 & 13 & 15 \\ 7 & 8 & 9 \end{bmatrix}$$

Check: $\Sigma x = 39$, $\Sigma h = 3$, so Σy should be 117. Adding the answer: $15 + 39 + 39 + 24 = 117$. ✓

P9. Find the 2D Z-transform of $x = [[4, 5, 6], [7, 8, 9]]$ and of $h = [1, 1, 1]^T$, and use them to obtain the convolution.

Each sample becomes coefficient $\times z_1^{-\text{row}} z_2^{-\text{column}}$:

$$\begin{aligned} X(z_1, z_2) &= 4 + 5z_2^{-1} + 6z_2^{-2} + 7z_1^{-1} + 8z_1^{-1}z_2^{-1} + 9z_1^{-1}z_2^{-2} \\ H(z_1, z_2) &= 1 + z_1^{-1} + z_1^{-2} \end{aligned}$$

Because convolution becomes multiplication, $Y = X \cdot H$. Multiplying, each term of X appears three times — once unchanged, once with z_1^{-1} and once with z_1^{-2} . Collecting equal powers:

- $z_1^0 z_2^0 \rightarrow 4$, so $y(0,0) = 4$
- $z_1^{-1} z_2^0 \rightarrow 4 + 7 = 11$, so $y(1,0) = 11$
- $z_1^{-2} z_2^0 \rightarrow 4 + 7 = 11$, so $y(2,0) = 11$
- $z_1^{-3} z_2^0 \rightarrow 7$, so $y(3,0) = 7$

and similarly for the z_2^{-1} and z_2^{-2} columns, giving the same 4×3 matrix as P8. The two methods must agree, because they are two ways of arranging the same multiply-and-add operations.

P10. Compute the linear convolution of $x = [[1, 2], [3, 4]]$ and $h = [[5, 6], [7, 8]]$ by the block Toeplitz matrix method.

Size: 3×3 , so the result column will have 9 entries.

Blocks from the rows of x (each 3×2 , first column is the row, then shifted down):

$$H_0 = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 2 \end{bmatrix} \quad H_1 = \begin{bmatrix} 3 & 0 \\ 4 & 3 \\ 0 & 4 \end{bmatrix}$$

Block Toeplitz matrix A $= [H_0 \ 0; H_1 \ H_0; 0 \ H_1]$, of size 9×4 .

h as a column, read row by row: $[5, 6, 7, 8]^T$.

Multiplying row by row: $5; 2(5)+1(6) = 16; 2(6) = 12; 3(5)+1(7) = 22;$
 $4(5)+3(6)+2(7)+1(8) = 60; 4(6)+2(8) = 40; 3(7) = 21; 4(7)+3(8) = 52; 4(8) = 32.$

Rearranged three per row:

$$y = \begin{bmatrix} 5 & 16 & 12 \\ 22 & 60 & 40 \\ 21 & 52 & 32 \end{bmatrix}$$

Check: $\sum x = 10$, $\sum h = 26$, so $\sum y$ should be 260. Adding: $33 + 122 + 105 = 260$. ✓

P11. A picture of a face and a picture of a dense crowd are both reduced to 4 bits per pixel. Which one looks worse, and why?

The **face** looks worse. A face is made largely of smooth, slowly changing areas — cheeks, forehead — and in a smooth area the quantization steps become visible as flat patches with hard edges between them, which is false contouring. The crowd picture is full of fine detail, and that busy texture hides the same steps, so far fewer gray levels are acceptable.

The general rule from the handout: images with a large amount of detail need fewer gray levels; images with large smooth areas need more. Spatial resolution and intensity resolution are therefore not independent — the right choice for each depends on the content.

P12. Explain how image subtraction is used in mask mode radiography, and why it works.

An X-ray image $h(x, y)$ of a region of the patient's body is captured first; this is the **mask**. A contrast medium is then injected into the bloodstream, and live images $f(x, y)$ of the same region are captured at TV rates. The enhanced image is

$$g(x, y) = f(x, y) - h(x, y)$$

It works because everything that did not move or change — bone, soft tissue, background — is identical in both pictures and subtracts to zero. Only the difference survives, which is the contrast medium travelling through the arteries. The result shows the flow clearly, and blockages of 50 % or 80 % can be measured. This is the technique called digital subtraction angiography.

P13. A monitor has a gamma of 2.5. What transformation must be applied to the image before display, and why? What happens if it is not applied?

A CRT's intensity-to-voltage response is a power function, so the displayed intensity is roughly the applied signal raised to the power $\gamma = 2.5$. Left alone, mid-gray values come out much darker than intended.

The correction is to pre-distort the image with the **inverse** power before sending it to the monitor:

$$s = c \, r^{1/\gamma} = c \, r^{1/2.5} = c \, r^{0.4}$$

The two non-linearities then cancel: $(r^{0.4})^{2.5} = r$. Without it the whole picture looks too dark, with the shadows crushed.

P14. Perform histogram equalization on a 3-bit ($L = 8$) image of size 64×64 whose gray levels occur 790, 1023, 850, 656, 329, 245, 122 and 81 times.

$MN = 4096$, and $L - 1 = 7$.

Step 1 — divide by 4096: $p_r = 0.19, 0.25, 0.21, 0.16, 0.08, 0.06, 0.03, 0.02$.

Step 2 — running totals: 0.19, 0.44, 0.65, 0.81, 0.89, 0.95, 0.98, 1.00.

Step 3 — multiply each by 7: 1.33, 3.08, 4.55, 5.67, 6.23, 6.65, 6.86, 7.00.

Step 4 — round: $s_k = 1, 3, 5, 6, 6, 7, 7, 7$.

Output histogram. Levels 3 and 4 both landed on 6, and levels 5, 6 and 7 all landed on 7, so add their probabilities:

$$p_s(1)=0.19 \cdot p_s(3)=0.25 \cdot p_s(5)=0.21 \cdot p_s(6)=0.24 \cdot p_s(7)=0.11$$

Eight levels became five. The histogram is spread out but not uniform, which is exactly what the discrete case predicts.

P15. Using the s_k from P14, match the image to the specified histogram 0, 0, 0, 0.15, 0.20, 0.30, 0.20, 0.15.

Step 1 — cumulative of the specified histogram: 0, 0, 0, 0.15, 0.35, 0.65, 0.85, 1.00.

Step 2 — multiply by 7 and round: $G(z_q) = 0, 0, 0, 1, 2, 5, 6, 7$.

Step 3 — for each s_k , find the closest value in that list and read off its q :

$$\begin{array}{llll} s=1 \rightarrow G=1 \rightarrow z=3 & s=3 \rightarrow G=2 \rightarrow z=4 & s=5 \rightarrow G=5 \rightarrow z=5 & s=6 \\ \rightarrow G=6 \rightarrow z=6 & s=7 \rightarrow G=7 \rightarrow z=7 & & \end{array}$$

Final mapping: $0 \rightarrow 3, 1 \rightarrow 4, 2 \rightarrow 5, 3 \rightarrow 6, 4 \rightarrow 6, 5 \rightarrow 7, 6 \rightarrow 7, 7 \rightarrow 7$.

Note that you never had to invert G . You built a table of all its values and searched it, which is how it is always done in practice.

P16. For the matrix $A = [167 \ 133 \ 111; 144 \ 140 \ 135; 159 \ 154 \ 148]$, find the 8th bit plane, and reconstruct the image using only planes 8 and 7.

Write each value in 8-bit binary, for example $167 = 10100111$ and $144 = 10010000$.

Plane 8 (MSB) is the leading bit of each: values of 128 and above give 1, below 128 give 0. Since 111 is the only value under 128:

```

1 1 0
1 1 1
1 1 1

```

Reconstruction from planes 8 and 7. Plane 8 is worth 128 and plane 7 is worth 64. For 167 = **10100111** the two leading bits are 1 and 0, giving $128 + 0 = 128$. For 111 = **01101111** they are 0 and 1, giving $0 + 64 = 64$. Doing all nine:

```

128 128 64
128 128 128
128 128 128

```

Only two planes were kept, so only a few distinct values can appear — the picture keeps its overall shape but loses all fine shading.

P17. State the two conditions a histogram equalization transformation $T(r)$ must satisfy, and give the reason for each.

(a) $T(r)$ must be single-valued and monotonically increasing on $0 \leq r \leq L-1$.

- Monotonically increasing means output values never fall below the corresponding input values, which prevents artifacts caused by reversals of intensity, and it preserves the order from black to white so the result is not a negative image.
- Single-valued means one input maps to exactly one output, which guarantees the inverse transformation exists.

(b) $0 \leq T(r) \leq L-1$ for $0 \leq r \leq L-1$, which guarantees the output range is the same as the input range.

If the inverse is needed as well, (a) is strengthened to *strictly* monotonically increasing, so the mapping back from s to r is also one-to-one and free of ambiguity.

P18. Why is local histogram equalization sometimes necessary, and how does the adaptive rule using mean and variance decide which pixels to change?

Why local. Histogram processing is global — one transformation built from the whole image. Small details that differ only slightly from their surroundings, and cover few pixels, cannot influence a histogram built from the entire picture, so they stay hidden. Local processing defines a small neighbourhood, computes the equalization for the points inside it, maps only the centre pixel, and moves on.

The adaptive rule. $g(x,y) = C \cdot f(x,y)$ when both conditions hold, otherwise $g(x,y) = f(x,y)$:

- $m_{Sxy} \leq k_0 m_G$ — the neighbourhood is darker than the image as a whole.
- $k_1 \sigma_G \leq \sigma_{Sxy} \leq k_2 \sigma_G$ — it has some variation, but not much. The lower limit rejects completely flat patches that contain nothing to reveal; the upper limit rejects patches that already have good contrast.

So only regions that are both dark and low in contrast get multiplied by the gain C , which is how faint shapes hidden inside dark boxes are brought out without disturbing the rest of the picture.

P19. A 3×3 image is $\begin{bmatrix} 7 & 9 & 11 \\ 10 & 50 & 8 \\ 9 & 5 & 6 \end{bmatrix}$. Filter it with a 3×3 box mask using zero padding, and find the values at $(0,0)$ and $(0,1)$.

The box mask is nine coefficients of $1/9$, so each output is the average of the nine values under the mask.

$g(0,0)$. The mask hangs off the top and the left, so five of the nine values are padded zeros:

$0, 0, 0, 0, 7, 9, 0, 10, 50 \rightarrow \text{sum } 76 \rightarrow 76/9 = 8.44$.

$g(0,1)$. Only the row above is missing, so three zeros come in:

$0, 0, 0, 7, 9, 11, 10, 50, 8 \rightarrow \text{sum } 95 \rightarrow 95/9 = 10.56$.

Notice that the padding pulls the edge pixels down towards zero. That darkening of the border is the price of zero padding, and it is why other schemes — replicating the edge, or mirroring it — exist.

P20. Compute the median of the centre pixel for the window $\begin{bmatrix} 18 & 22 & 33 \\ 34 & 128 & 24 \\ 22 & 19 & 32 \end{bmatrix}$ using a 3×3 mask, and say why the answer is better than the average here.

Sort all nine values, the centre included: 18, 19, 22, 22, **24**, 32, 33, 34, 128.

Nine values, so the median is the fifth: **24**. The centre pixel 128 is replaced by 24.

The average would have been $(18+22+33+34+128+24+22+19+32)/9 = 332/9 = 36.9$ — dragged up by the single bad pixel, and that 128 would also have leaked into all eight neighbouring outputs. The median never selects an extreme value, so the impulse is removed outright instead of being spread around.

P21. A scan line reads 6 6 6 6 5 4 3 2 1 1 1 1 1 6 6 6 6. Find the first and second derivatives and comment on the results.

First derivative, $f(x+1) - f(x)$, taken from the second sample to the penultimate one:

0 0 -1 -1 -1 -1 -1 0 0 0 0 0 5 0 0 0

Second derivative, $f(x+1) + f(x-1) - 2f(x)$, over the same range:

0 -1 0 0 0 0 1 0 0 0 0 5 -5 0 0 0

Comments. In the constant regions both are zero, as required. Along the *ramp* the first derivative holds a steady -1 the whole way down, producing a thick response, while the second derivative is zero in the middle of the ramp and non-zero only at its two ends — a thin response marking just the corners. At the *step* the first derivative gives a single

spike of +5; the second gives +5 immediately followed by −5, and the point between them is the **zero crossing**, which fixes the edge to the exact pixel.

So: first derivatives give thicker edges and react to gradual change; second derivatives react more strongly to fine detail and locate edges precisely.

P22. Derive the 3×3 Laplacian mask from the definition, and write the mask that sharpens an image in a single pass.

The Laplacian is $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$. Write each second derivative as a difference:

$$\partial^2 f / \partial x^2 = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

$$\partial^2 f / \partial y^2 = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

Adding gives $\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$, so the coefficients are +1 on the four side neighbours and −4 on the centre:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

For sharpening in one pass, substitute into $g = f - \nabla^2 f$ (the centre is negative):

$$g = 5f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] \rightarrow \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Check: the Laplacian mask sums to 0 (a pure derivative), the composite mask sums to 1 (a derivative *plus* the original), which is exactly what you want.

P23. For the window [1 2 3; 4 5 6; 7 8 9], compute the gradient magnitude using the Sobel operator with the $|G_x| + |G_y|$ approximation.

Label the window $z_1 \dots z_9$ row by row: $z_1=1, z_2=2, z_3=3, z_4=4, z_5=5, z_6=6, z_7=7, z_8=8, z_9=9$.

$$G_x = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) = (7 + 16 + 9) - (1 + 4 + 3) = 32 - 8 = 24$$

$$G_y = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) = (3 + 12 + 9) - (1 + 8 + 7) = 24 - 16 = 8$$

Magnitude $\approx |24| + |8| = 32$. The exact value would be $\sqrt{(24^2 + 8^2)} = \sqrt{640} = 25.3$, so the absolute-value form overestimates a little — the usual price of avoiding a square root.

The large G_x against the small G_y says the brightness climbs much faster down the columns than across the rows, so the edge here runs roughly horizontally.

P24. A mask is given as [−1 −1 −1; −1 9 −1; −1 −1 −1]. What kind of filter is it, and what does it do?

Add the coefficients: eight -1 s and one $+9$ give $+1$. A sum of 1 means the mask preserves average brightness, so it is *not* a pure derivative.

It is the **composite Laplacian sharpening mask that includes the diagonals**. Start from the eight-neighbour Laplacian $\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$, which sums to 0, and form $g = f - \nabla^2 f$: the centre becomes $1 + 8 = 9$ and every neighbour becomes -1 .

What it does: it adds the image's own fine detail back onto itself, so edges and texture are strengthened while flat regions are left alone. It is also the high-boost mask with centre $A+8$ at $A = 1$.

CHAPTER 33 EXAM KIT

Word list

One line each. If a question uses a word you cannot define, you cannot answer it — so read this list the morning of the exam.

Pixel	One element of a digital image — one position and its intensity value.
Intensity / gray level	The brightness number stored at a pixel; 0 is black, 255 is white in an 8-bit image.
Sampling	Digitising the <i>coordinates</i> — deciding how many points are measured. Fixes $M \times N$.
Quantization	Digitising the <i>amplitude</i> — deciding how many brightness levels are allowed. Fixes L .
Spatial resolution	The smallest detail that can still be made out; decided by the sampling. Often quoted as DPI.
Intensity resolution	The number of gray levels, $L = 2^k$; decided by the quantization.
False contouring	Visible bands with hard edges in smooth areas, caused by too few gray levels.
Checkerboard effect	Individual pixels showing as squares, caused by too few samples.
Radiance	Total energy flowing from a light source, in watts.
Luminance	The amount of that energy an observer perceives, in lumens.
Brightness	A subjective impression of intensity; cannot be measured.
Illumination $i(x,y)$	Light falling on the scene; $0 < i < \infty$.
Reflectance $r(x,y)$	Fraction of light the object sends back; $0 < r < 1$.
Enhancement	Improving appearance for a purpose; subjective, judged by the viewer.
Restoration	Reversing a known degradation; objective, based on a model.
Indexed image	Pixels store index numbers into a colour table instead of colours themselves.
Raster image	A fixed grid of sampled pixels; degrades when zoomed.
Vector image	Shapes defined by formulas; scales to any size with no loss.
V (the set)	The set of intensity values that count as "the object" when deciding adjacency.
Adjacency	Two pixels are adjacent if both values are in V and their positions touch in the required way.
m-adjacency	Mixed adjacency; allows a corner step only when no route exists through a common side neighbour. Removes the ambiguity of 8-adjacency.

Path	A sequence of distinct pixels in which each is adjacent to the next; its length is the number of steps.
Connected component	All the pixels of a set that are connected to one given pixel.
Region	A connected set of pixels in an image.
Boundary	The pixels of a region that have at least one neighbour outside the region.
Foreground / background	The union of all object regions / everything else, its complement.
Metric	A distance function satisfying non-negativity, symmetry and the triangle inequality.
Linear operator	An operator with both additivity and homogeneity.
Array vs matrix product	Array product multiplies pixel by pixel ($A \cdot B$); matrix product is rows-times-columns ($A * B$).
Convolution	Sliding a kernel over an image, multiplying the overlap and summing; written $x ** h$.
Kernel / mask	The small matrix h that is slid over the image.
Toeplitz matrix	A matrix in which each column is the previous one shifted down by one place; every diagonal is constant.
Frequency response	What you get by putting $z = e^{i\omega}$ in the Z-transform; tells you which spatial frequencies a filter keeps.
Enhancement	Manipulating an image so the result is more suitable than the original for a specific application. Problem-oriented, and judged by the viewer.
Point processing	A 1×1 neighbourhood: the new value of a pixel depends only on its own old value, through $s = T(r)$.
Mask processing	A neighbourhood bigger than 1×1 : the new value depends on a window of neighbours. Also called filtering.
Gamma	The exponent in $s = c r^\gamma$. Also the exponent of a display's own response curve, typically 1.8 to 2.5 for a CRT.
Gamma correction	Pre-distorting an image with $s = c r^{1/\gamma}$ so that the display's non-linearity cancels out.
Contrast stretching	A piecewise linear transformation that widens the range of gray levels actually used.
Gray-level slicing	Highlighting one range $[A, B]$ of gray levels, either flattening everything else (without background) or preserving it (with background).

Bit-plane slicing	Splitting an 8-bit image into eight 1-bit images; plane 8 is the MSB and holds most of the visible information.
Histogram	$h(r_k) = n_k$, the count of pixels at each gray level. It says nothing about where those pixels are.
Histogram equalization	Using the cumulative distribution as the transformation, which spreads the histogram towards uniform. Automatic, with no parameters.
Histogram matching	Also called specification: forcing the output histogram into a shape you choose, via $z = G^{-1}[T(r)]$.
Local enhancement	Running a histogram method inside a small moving window and mapping only the centre pixel, so small details are not drowned by the global statistics.
Variance	$\sigma^2 = \sum (r_i - m)^2 p(r_i)$. A measure of contrast; low in smooth regions, high at edges, in texture and in noise.
Mask / kernel / template	The small array of coefficients slid over the image in neighbourhood processing.
Coefficient	A number inside a mask. A pixel is data; a coefficient is a weight you chose.
Zero padding	A border of zeros added round the image so the mask can be centred on edge pixels.
Box filter	A smoothing mask whose coefficients are all equal.
Order-statistic filter	A non-linear filter whose output comes from ranking the pixels, not weighting them.
Impulse noise	Salt-and-pepper noise: scattered pixels stuck at 0 or at the maximum value.
Isotropic filter	One whose response does not depend on the direction of the discontinuity; rotation invariant.
Zero crossing	The point where the second derivative changes sign; it locates an edge to the exact pixel.
High-boost filter	High frequency emphasis: $(A-1)f + \text{high pass}$, keeping some low frequency content.
Gradient	The vector of first partial derivatives; it points where intensity grows fastest.

A suggested plan for reading this

1. **First pass, one sitting.** Read Chapters 1 to 6 without stopping at the numbers. You are only building the picture: an image is a table, sampling sets its size, quantization sets its shades.
2. **Second sitting.** Chapters 7 to 10 with a pen. Draw the N_4 / N_D / N_8 grids and the D_4 diamond and D_8 square yourself, from memory, until they come out right.
3. **Third sitting.** Chapters 11 to 14. Do the graphical convolution of Chapter 12 on paper before reading the answer, then do the same problem again by the Z-transform and by the Toeplitz matrix, and confirm all three agree.
4. **Fourth sitting.** Chapters 15 to 22, the enhancement half. Do not just read the histogram chapters — copy out the 3-bit table of Chapter 20 and fill in the s_k column yourself, then do the matching table of Chapter 21 the same way. Those two tables are the most likely numerical questions in the whole unit.
5. **Before the exam.** Chapter 23 formula sheet, then all eighteen problems in Chapter 24 with the solutions covered, then the word list.

The five assignments you have already written are the practical half of exactly this material — Assignment 1 for image reading and representation, Assignment 2 for resolution, Assignment 3 for convolution, Assignment 4 for pixel relationships and distances, Assignment 5 for image types and colour. If a chapter feels abstract, open the matching program and print the numbers.

Prepared for Nikhil Pathak (U24EC012) from the EC341 lecture handouts and course syllabus.